THEORETICAL FRAMEWORK AND RESEARCH PROPOSAL

Persistence Limited Interference

of Therapeutic Interfering Particles in

Orthoebolavirus zairense

(Ebola virus, EBOV)

A free, offline tool for ranking candidate Ebola antivirals before they enter high-containment laboratories.

A zero dependency, multi-scale deterministic C++20 simulation engine with complete annotated source code for modeling Ebola virus within host dynamics and mechanistically prioritizing Therapeutic Interfering Particle (TIP) profiles under cytokine and autonomic safety constraints, incorporating antibody enhanced clearance and asymmetric hijacking

https://github.com/rein-baremetal/ebola-within-host-model

Part I: theory, model, engine and core results. Part II: mechanism and robustness, adaptive immunity and vaccination, stochastic boundaries, extended specification.

Author

REINFORD NYACHIO

Maasai Mara University, Kenya

November 2026

Preface

This document records a computational idea and the program that implements it. It was written in November 2026 by a researcher at Maasai Mara University, Kenya, working from the open literature. It is meant to be understood without its author: the equations are printed in full and explained in words, every parameter carries units, every result states how it was produced, and the complete annotated C++ source is included. Nothing is hidden: the document states what was verified, what was only assumed, and what would prove the work wrong. If the approach proves mistaken, the document should still show exactly why. If it proves useful, it prioritises candidate interfering particle designs and reduces the experimental burden, including scarce biosafety level-4 time, before any wet-laboratory work begins. It makes no claim of patient benefit.

Epistemic status

Computational conjecture, not established biology. All parameters are illustrative placeholders until they are calibrated against published data (Section 9.1). To the author's knowledge no in vivo data on engineered Ebola TIPs existed in November 2026, so every TIP-specific statement here is a model prediction that experiments must test.

Verified: numerical correctness of the solver (fourth-order convergence, determinism, invariants). **Not verified:** biological realism of the TIP behaviour. The theory is stated so that it can be falsified (Section 4.4). The six falsifiers F1–F6 carry numeric thresholds (Table 3a), and calibration to published data is the first aim of the work plan, with a stop rule: if the calibrated model cannot reproduce published EBOV kinetics within tolerance, the quantitative claims are withdrawn (Section 14.3).

How to read this document

- **In 30 minutes:** the Abstract (Section 1), the theory (Section 4), the results figures (Sections 7 and 10) the falsifier register (Table 3a), the measurement targets (Table 20) and the synthesis (Section 15). You will know the claim and the evidence.
- **In 3 hours:** add the model (Section 5), the methods (Section 6) and Sections 10–12. You will be able to re-derive every number in the results.
- **In 3 days:** add Appendices C–E, the annotated source code and analysis scripts. You will be able to rebuild the engine, extend it, or port it to another language.

1 Abstract

Aim. We will deliver an offline, open-source, reproducible decision tool that ranks candidate Ebola therapeutic interfering particles before they consume scarce BSL-4 time, and a pre-registered test that will say clearly whether the underlying theory survives contact with data.

Ebola virus disease (EVD), caused by *Orthoebolavirus zairense* (Ebola virus, EBOV), remains a high mortality infection. A licensed vaccine and two licensed monoclonal antibody therapeutics have improved outcomes, but gaps remain in rapid post exposure protection and in control of cytokine driven pathology. Therapeutic Interfering Particles (TIPs) are non-autonomous, replication defective particles that keep the packaging signals of the virus but lack its essential genes. A TIP replicates only inside a cell that is also infected with wild type (WT) virus, where it diverts the viral replication machinery to its own genome. TIPs are therefore a candidate self-propagating, within host protective strategy. Choosing among candidate TIP designs experimentally is slow and costly, and no practical tool exists to rank designs beforehand.

This work proposes persistence limited interference (short form RPT-EBOV; Section 2) as a mechanistic theory of when TIP interference can clear EBOV from a host, together with the simulation engine that embodies it. The engine couples (i) a target cell, eclipse phase, infected cell virus model extended to WT/TIP co-infection with TIP-primed cells; (ii) a pro-inflammatory cytokine subsystem; and (iii) an autonomic pulse rate feedback acting through the cholinergic anti-inflammatory pathway. It is written in zero-dependency C++20, uses a fixed step fourth order Runge–Kutta (RK4) integrator, and exposes a text in, text-out C interface designed for a thin Kotlin/JNI Android layer, so that it can be used in laboratories without reliable internet access. A screening module searches TIP profiles defined by entry rate (β_T), hijacking factor (α), TIP production (p_T), TIP clearance (c_T) and antibody-like effects, and ranks them by

peak WT viral load, cytokine and pulse rate safety bounds and time to extinction of WT virus, with parameter sensitivity analysis.

Central result. On illustrative parameters, WT extinction occurs only when the free TIP persists for days. With TIP-primed cells tracked, free-TIP clearance must fall below roughly 0.26–0.46 d^{-1} (half life of about 1.5–2.7 days or more), against 5–15 d^{-1} for WT virus. A variant that ignores primed cells overstates the TIP benefit by a factor of about 2.5. This is reported as a design requirement for experiments to check, not as an established property of TIPs. The prototype reproduces the expected RK4 convergence order (4.04), is bitwise identical on 1 and 4 threads, and its principal results were regenerated from a clean build (Sections 7.4 and 7.6).

Continuation. Part II tests the theory further. The persistence threshold is mapped over dose and clearance, shown to be controlled chiefly by R_0 and the infected-cell lifetime, and found independent of TIP production at fixed dose. An adaptive immunity extension (antibody, B-cell and CTL states, cross validated against the core) shows that vaccine induced antibody relaxes the requirement from 0.26 to above 1 d^{-1} at moderate titres, provided the TIP is antigenically distinct from the vaccine antigen; if antibody clears the TIP additively the combination fails. A stochastic analysis derives and validates the establishment probability of an infection and shows that the deterministic threshold is conservative. A specification for pan-filovirus, tropism and compartment extensions is given, with its formulation checks.

Verification tests RK4 convergence order, return to the infection free equilibrium, threshold behaviour at $R_{eff} = 1$, determinism and fuzz robustness. Validation will test reproduction of published EBOV within-host kinetics. The engine is a research decision support instrument. It is not a diagnostic or therapeutic device, and its outputs are not patient level predictions or manufacturing instructions. Calibration to published EBOV kinetics is the first aim of the work plan; it carries a stated tolerance and a stop rule, and a calibration failure is reported as a result (Section 14.3).

Keywords: Orthoebolavirus zairense; Ebola virus; therapeutic interfering particles; vaccination; adaptive immunity; stochastic extinction; within host modelling; ordinary differential equations; cytokine storm; persistence limited interference; C++20; Android.

2 Naming, Terminology and Abbreviations

Virology names change over time. To keep this work searchable, it uses the naming conventions of the International Committee on Taxonomy of Viruses (ICTV) for the virus and the World Health Organization (WHO) for the disease, and it records older synonyms. Readers should check the current ICTV taxonomy release, because names can be revised again.

Table 1. Standard names used in this document

Level	Name used here	Note
Order	*Mononegavirales*	Non-segmented negative-strand RNA viruses
Family	*Filoviridae*	Filamentous viruses (Ebola, Marburg and relatives)
Genus	*Orthoebolavirus*	Genus of the ebolaviruses
Species	*Orthoebolavirus zairense*	Binomial species name. Older synonym: *Zaire ebolavirus* (use for searching older papers)
Virus	Ebola virus (EBOV)	Virus names are not italicised; the species name is
Disease	Ebola virus disease (EVD)	Older name “Ebola haemorrhagic fever”
Intervention	Therapeutic Interfering Particle (TIP)	Engineered, non-autonomous, replication-defective particle
Natural analogue	Defective interfering particle (DIP)	Arises spontaneously; documented for EBOV (Calain *et al.*, 1999)
This theory	Rein Persistence Theory (RPT-EBOV)	Eponym on the theory only; see the convention below

Eponym convention. The name “Rein” is attached to the theory only. ICTV and WHO practice discourages naming viruses, diseases or particles after people, so this work never writes “Rein virus”, “Rein TIP” or similar. The theory name has three parts: the author’s eponym (**Rein**), the mechanism it asserts (**Persistence-Limited Interference**) and the formal taxon it applies to (*Orthoebolavirus zairense*). The short form **RPT-EBOV** is for tables and code comments. In this document and in grant-facing material the

theory is called persistence limited interference; the eponym is explained here and survives only in the short form.

Table 2. Abbreviations

Term	Meaning
EBOV / EVD	Ebola virus / Ebola virus disease
TIP / DIP	Therapeutic Interfering Particle / defective interfering particle
WT	Wild type virus
RPT-EBOV	Rein Persistence Theory for EBOV
ODE	Ordinary differential equation
RK4	Classical fourth-order Runge–Kutta method
R_0 / R_{eff}	Basic / effective reproduction number of WT virus
c_T, c_W	Clearance rates (d^{-1}) of free TIP and free WT virions
LHS / PRCC	Latin hypercube sampling / partial rank correlation coefficient
JNI / NDK	Java Native Interface (Kotlin–C++ bridge) / Android Native Development Kit
ICTV / WHO	International Committee on Taxonomy of Viruses / World Health Organization

3 Background, Problem and Purpose

3.1 Background

Ebola virus (*Orthoebolavirus zairense*) causes a severe haemorrhagic disease whose case fatality rate has varied widely between outbreaks, from roughly a quarter to about nine in ten of those infected (World Health Organization, 2024). The licensed vaccine and the two monoclonal antibody therapeutics (Inmazeb and Ebanga) have improved survival (Mulangu *et al.*, 2019). Even so, mortality in treated patients remains substantial. Treatment depends on early presentation, cold-chain supply and specialist care. Severe disease is driven both by uncontrolled viral replication and by a dysregulated, cytokine-mediated inflammatory response (Fajgenbaum and June, 2020).

TIPs offer a different approach. A TIP is a non-autonomous, defective genome that is packaged and amplified only when the same cell is infected with the wild-type virus. Inside a co-infected cell the TIP competes for polymerase and structural proteins, and its shorter genome gives it a replication advantage. WT output falls while TIP output rises, so the intervention scales with the infection it targets (Tanner, Kirkegaard and Weinberger, 2016; Notton *et al.*, 2014). Naturally arising defective interfering particles of Ebola virus have been described (Calain, Monroe and Nichol, 1999), and engineered TIPs have been proposed for other viruses (Rouzine and Weinberger, 2013; Chaturvedi *et al.*, 2021).

3.2 Problem statement

Candidate TIP designs differ in entry efficiency, replication advantage, interference strength and clearance. Their combined effect on WT viral load, on cytokine-driven pathology and on the host's autonomic response is nonlinear and cannot be judged by intuition. Testing every combination in cell culture or animal models is slow, expensive and, for a biosafety level 4 pathogen, available to very few laboratories. No simple, offline, reproducible tool exists that lets a laboratory in a resource-limited setting rank candidate TIP profiles on mechanistic grounds before committing to experiments.

3.3 Purpose: prioritising candidate interfering particle designs and reducing experimental burden

The engine does not cure anything and makes no claim of patient benefit. Its purpose is to make the early, cheap part of therapeutic design faster and more honest: to say which design parameters matter, which regions of design space are hopeless, and which experiments are worth the scarce biosafety-level-4 time. Any route from this document to clinical benefit is long and lies outside the scope of this work. Table 3 shows where the work sits and what others would have to do next.

Table 3. From simulation to experiment: where this work sits (stages after 0 are for others)

Stage	Question	Contribution of this work	What must come next
0 (here)	Which TIP profiles are worth testing?	Ranked profiles, parameter sensitivities, the persistence requirement	Calibrate to published kinetics (Phase 1)
1	Do the model and published EBOV kinetics agree?	Validation plan and tolerances	Fit and cross-validate on non-human-primate and patient data
2	Does a real particle persist and interfere?	Measurable targets: free-particle half-life, entry rate, hijacking factor	In vitro experiments in qualified BSL-4 laboratories
3	Does it work in a living host?	Predicted dose and timing windows	Animal studies under ethical and biosafety oversight
4	Is it safe and combinable?	Cytokine and pulse safety bounds; antibody-like terms to model combinations with licensed antibodies	Safety studies; extend the model with adaptive immunity for vaccine combinations
5	Is clinical evaluation warranted?	None. Outside the scope of this work	Regulated clinical evaluation during or between outbreaks

3.4 Justification and significance

- **Efficiency of experimental effort.** Ranking candidates in silico narrows the number of constructs that need testing. Whether this raises laboratory success rates can only be judged by later experimental comparison, which is outside this study.
- **Mechanistic insight.** The engine shows which parameters (entry, hijacking strength, production advantage, clearance) limit TIP success, and so which measurements to make first.
- **Access.** An on-premise, internet free Android and workstation tool suits laboratories in East Africa, where connectivity, computing budgets and licensed software are constraints.
- **Capacity building.** The usability study with laboratory staff, local training and open-source ownership by a Kenyan institution are deliverables of the work plan, not by-products (Sections 9.1 and 9.2).
- **Safety framing.** Cytokine and pulse-rate bounds are built into the screening objective, so profiles that clear virus but provoke pathological inflammation are penalised.

3.5 Scope

The work is entirely computational. It models within-host dynamics only. It involves no handling of pathogens, no wet laboratory work and no design of genetic sequences. Inputs are published kinetic parameters; outputs are abstract parameter profiles with simulated trajectories. No wet laboratory partner is named and no experimental validation is claimed: Aims 1–3 (Section 9.1) are computational only. The experiments that would test the model are specified in Table 20; running them is outside this proposal and belongs to a partner led application.

4 The Theory of Persistence-Limited Interference

4.1 Statement of the theory

Persistence-limited interference (RPT-EBOV)

In one sentence: in a host infected with *Orthoebolavirus zairense*, a Therapeutic Interfering Particle can drive wild-type virus to extinction only if the free TIP persists in the body for days, because the TIP is amplified only inside cells that WT virus has already infected and so loses its source of supply as it succeeds.

Why "persistence limited": the limiting factor for TIP success is not how strongly the TIP interferes inside a cell, but how long free TIP survives outside cells.

The intuition is as follows. Inside a co-infected cell the TIP out-replicates and suppresses WT virus. But the TIP can only multiply there. As WT virus falls, the number of co-infected cells falls with it, and so does the TIP's production. Residual WT virus, even a few copies, can regrow once the TIP supply is gone. A TIP that is cleared quickly (as WT virions are, at 5–15 per day) cannot bridge this gap. A TIP that circulates for days can intercept the residual infection until it is extinct.

4.2 Postulates

P1. **P1 Interference.** In a co-infected cell the TIP suppresses WT output to a fraction α of normal ($0 \le \alpha \le 1$) and is itself produced faster than WT ($p_T > p_W$). (Equations 6–7.)

P2. **P2 Primed cells.** A TIP that enters an uninfected cell cannot replicate alone, but that cell (a "primed" cell, P) stays open to later WT entry. TIP entry is therefore not a pure loss of target cells. Models that treat it as one (the abortive-entry variant, primed = 0) overstate TIP benefit. (Equations 4 and 4b.)

P3. **P3 Persistence.** For a TIP profile θ there is a critical free-TIP clearance rate $c_T^*(\theta)$ above which WT extinction does not occur. In the reference model $c_T^*(\theta)$ is far below the WT clearance rate c_W (5–15 d^{-1}): about 0.26–0.46 d^{-1} for the profiles tested, and no primed-cell extinction for a weaker profile.

P4. **P4 Safety coupling.** Stimulating the innate response with TIP ($\rho_T > 0$) can raise c_T^* only by pushing cytokine far past its safety bound, so it does not rescue a fast-cleared TIP safely. (Equations 8–10.)

Postulate P3 is stated precisely by defining the critical clearance as the supremum of the free-TIP clearance rates that still permit extinction, and the free-TIP half-life that corresponds to it:

$$c_T^*(\theta) = \sup\{c_T > 0 : \; V_W(t) < V_{\text{ext}} \quad \text{for all } t \ge t_{\text{ext}}\}, \qquad t_{1/2} = \frac{\ln 2}{c_T} \tag{15}$$

In words: $c_T^*(\theta)$ is the largest free-TIP clearance for which WT virus falls below V_{ext} and stays below it. The half-life is ln 2 divided by the clearance rate. The theory asserts that c_T^*/c_W is of order a few hundredths (Section 10).

4.3 Quantitative predictions of the reference model

- Extinction requires c_T below about 0.5 d^{-1} (free-TIP half-life of about 1.4 days or more). Thresholds found: 0.46 d^{-1} (strongest profile), 0.26 d^{-1} (best screened profile). Half-life = ln 2 / c_T, so these are 1.5 and 2.7 days.
- Ignoring primed cells raises the apparent thresholds to 1.15 and 0.69 d^{-1}, a factor of about 2.5–2.7 too generous.
- With c_T = 5–15 d^{-1}, 0 of 2000 sampled profiles reach extinction: the TIP is cleared within about 2 days and residual virus regrows.
- An entry-blocking effect alone must exceed $1 - 1/R_0$ (about 88.5% for R_0 = 8.73) to bring R_{eff} below 1; TIP hijacking and production advantage reduce this requirement (hypotheses H1–H2).

4.4 How the theory can be falsified

A theory is useful only if it can fail. RPT-EBOV is rejected, or must be revised, if any of the following is observed:

F1. **F1.** A particle of the TIP type is measured with an effective free half-life well under about 1.4 days in a relevant host, yet still achieves sustained WT clearance in a faithful experimental model. Then persistence is not the limiting factor, and some mechanism missing from the model (for example interferon-mediated protection of primed cells) dominates.

F2. **F2.** Cells that have only been entered by a TIP are shown to be protected against later WT entry. Postulate P2 fails, and the true thresholds are higher than reported here.

F3. **F3.** Calibrated parameter sets (Phase 1) give c_T^* inside the WT clearance range of 5–15 d^{-1} when the structural assumptions of Section 8.1 are relaxed. Hypothesis H4 is rejected as a general property.

F4. **F4.** Stochastic (Gillespie-type) simulation of the low-count phase shows that extinction obtained with the deterministic floor does not survive demographic noise.

F5 and F6 are introduced with the vaccination extension in Section 11.7. Table 3a collects all six with the numeric rule that rejects the theory, the measurement that decides it, and its present status. Thresholds marked "proposed" are fixed here so that they can be pre-registered before any calibrated run (Section 14.3); the others come from the results of Sections 7 to 12.

Table 3a. Falsifier register F1–F6: numeric rejection rules, deciding measurements and status in November 2026

ID	Claim at risk	The theory is rejected or revised if	Decided by	Status
F1	P3, persistence	Effective free-TIP half-life is below 1.4 d (cT above about 0.5 d^{-1}) and yet WT clearance is sustained in a faithful experimental model	Free-TIP half-life in a relevant host or ex vivo system (Table 20, row 1)	Open; needs experiment. The 1.4 d bound is from Section 4.3
F2	P2, primed cells	TIP-only cells are protected from later WT entry by a fraction π of 0.5 or more (proposed threshold; π = 0 is the reference model, π = 1 the abortive variant with cT* of 0.696 against 0.256 d^{-1})	Superinfection assay on TIP-only cells (Table 20, primed-cell row)	Open; needs experiment; threshold proposed
F3	P3 and H4, cT* far below cW	Calibrated parameter sets give cT* inside the WT clearance range of 5–15 d^{-1} once the structural assumptions of Section 8.1 are relaxed	Aim 1 calibration with local and global sensitivity (Sections 10.5 and 14.3)	Pending Aim 1. On uncalibrated ±25% changes cT* never exceeds 0.4 d^{-1} (largest 0.393, Figure 11)
F4	Extinction under demographic noise	Stochastic tail simulation gives an extinction probability below 0.5 at cT = 0.9 × cT* (200 replicates; proposed threshold)	Gillespie tail simulation (Section 12.2)	Not triggered: P = 0.995 to 1.00 for cT up to 0.25 d^{-1} (Table 17)
F5	P6, antigenic distinctness	Post-vaccination sera clear a TIP at several d^{-1} (proposed: 3 d^{-1} or more, at least 0.3 cW) at titres where WT infectivity is also reduced	Neutralisation and clearance assays with post-vaccination sera (Table 20, ξT row)	Open; needs experiment; threshold proposed
F6	Vaccine alone versus TIP	Measured antibody in vaccinated hosts exceeds Acrit of 68.6 µg mL^{-1} (Eq 25; 94.0 µg mL^{-1} for 60-day extinction)	Serology in a vaccinated cohort; KA and εmax (Table 20)	Open; needs data. Acrit is conditional on illustrative constants

4.5 Hypotheses tested with the engine

- **H1.** Under the reference parameter set, at least one TIP profile drives WT virus to extinction with an antibody-like entry effect below the entry-only benchmark $1 - 1/R_0$, so the TIP adds benefit beyond entry blocking alone.
- **H2.** Profiles combining high β_T, low α and elevated p_T reach extinction at substantially lower individual parameter values than any single-mechanism profile.
- **H3.** In the monostable regime, profiles that reduce peak V_W also reduce peak cytokine C relative to the untreated case. Exceptions are expected where weakened cytokine-mediated killing of infected cells lets virus rise, and will be tested explicitly.
- **H4 (exploratory).** Extinction of WT virus requires the free TIP to persist for days (this is Postulate P3). It is rejected as a general property under falsifier F3.

4.6 Objectives and research questions

General objective. To develop and validate a deterministic, zero-dependency C++20 simulation engine that predicts EBOV–TIP within-host dynamics and ranks candidate TIP profiles by predicted protective efficacy and safety. **Specific objectives:** (1) formulate a ten-state ODE model of EBOV–TIP co-infection with primed cells and cytokine and autonomic feedback, documenting the provenance of every parameter; (2) implement it in an own-coded fixed-step RK4 solver and verify it against Table 8; (3) build a screening module that samples and optimises TIP profiles and reports Pareto-optimal candidates with sensitivity analysis; (4) validate against published EBOV within-host kinetics to a stated tolerance on peak value and peak timing; (5) deliver a JNI bridge and Kotlin interface and assess usability with laboratory staff.

Research questions. Which combinations of β_T, α, p_T and c_T drive WT virus to extinction within the cytokine and pulse-rate safety bounds? How large an entry-only effect is needed to bring R_{eff} below 1, and how much does hijacking and production advantage reduce it? Which parameters dominate the uncertainty in the ranking? Does the dependence of extinction on free-TIP clearance persist when primed cells are included?

4.7 Literature context

Target-cell-limited models with an eclipse phase are the standard framework for acute within host infection (Nowak and May, 2000; Perelson, 2002) and have been applied to Ebola virus with in vitro, non-human-

primate and patient data (Nguyen *et al.*, 2015; Martyushev *et al.*, 2016; Madelain *et al.*, 2018); these studies supply the kinetic ranges used here and show that several parameters are weakly identifiable from viral load alone, which motivates the global sensitivity analysis. Mathematical treatments of interfering particles show that coexistence and suppression depend on entry, relative replication and packaging advantage (Metzger, Lloyd-Smith and Weinberger, 2011; Rouzine and Weinberger, 2013); experimental TIP work has focused on HIV and respiratory viruses. Excess cytokine release drives severe disease (Fajgenbaum and June, 2020) and the vagal inflammatory reflex restrains it (Tracey, 2002). **Gap:** to the author's knowledge, and subject to confirmation in Phase 0, no published tool combines EBOV–TIP co-infection dynamics, cytokine and autonomic feedback and automated multi-objective profile screening in an offline, deterministic, portable engine.

5 The Mathematical Model

5.1 State variables and assumptions

Ten states are integrated. In the code (Appendix C) they are the first ten entries of the state vector; four further running integrals accumulate the objective function.

Table 4. State variables

Symbol	Code	Meaning	Units
T	iT	Uninfected target cells (macrophages, monocytes, dendritic cells)	cells mL^{-1}
E_W, I_W	iEW, iIW	Eclipse and productively infected WT-only cells	cells mL^{-1}
E_{Co}, I_{Co}	iEC, iIC	Eclipse and productively infected co-infected cells (WT + TIP)	cells mL^{-1}
P	iP	TIP-only "primed" cells (entered by a TIP, no WT)	cells mL^{-1}
V_W	iVW	Free WT virions	copies mL^{-1}
V_T	iVT	Free TIP virions	copies mL^{-1}
C	iC	Cytokine surrogate (IL-6-like)	pg mL^{-1}
H	iH	Host pulse rate	beats min^{-1}
J_V, J_C, J_H, U	iJV…iU	Running integrals for the objective J and TIP exposure	mixed

The model rests on the following assumptions, all stated in the interface and in reports:

- A TIP entering an uninfected cell cannot replicate. The cell leaves T and becomes a primed cell P (Equation 4b), which releases no virions and dies at the natural rate d_T (abortive infection).
- A primed cell can later be entered by WT virus at the same rate β_W as an uninfected cell, and then becomes a co-infected eclipse cell (Equation 4). Primed cells are not protected against WT. The abortive-entry variant (primed = 0) is kept as an ablation. Interferon-mediated protection of primed cells, which would favour the TIP, is not represented.
- Cytokine synthesis is driven by infected cells, WT virus and, through the term $\rho_T \cdot V_T$ in Equation 8, free TIP. The default is $\rho_T = 0$ (no innate stimulation by the TIP); a positive ρ_T is a sensitivity parameter because TIP immunogenicity is unknown.
- A WT-productive cell entered by a TIP becomes productively co-infected directly (no new eclipse phase). A WT-eclipse cell entered by a TIP moves to the co-infected eclipse state.
- Mass-action infection, with loss of virions by cell entry neglected relative to clearance (standard in this model class).

5.2 Governing equations

Entry of both WT and TIP is reduced by an antibody-like entry-blocking effect $\varepsilon_{Ab}(t)$. Enhanced WT clearance is represented by $\varepsilon_{Clr}(t)$ with gain κ. Infected-cell death is increased by a saturating cytokine-mediated term $\delta_N \cdot C/(K_F + C)$. The equations below are typeset as vector graphics (SVG), and each is followed by a reading in words.

Cells

$$\frac{dT}{dt} = \lambda - d_T\,T - (1-\varepsilon_{\mathrm{Ab}})(\beta_W V_W + \beta_T V_T)T \tag{1}$$

In words: Uninfected target cells are supplied at rate λ, die naturally at rate d_T, and are lost whenever WT or TIP virus enters them (entry reduced by the factor $1 - \varepsilon_{Ab}$).

$$\frac{dE_W}{dt} = (1-\varepsilon_{\mathrm{Ab}})\,\beta_W\,T\,V_W - k_W E_W - (1-\varepsilon_{\mathrm{Ab}})\,\beta_T E_W V_T \tag{2}$$

In words: WT-eclipse cells are created when WT enters a target cell, leave eclipse at rate k_W, and move to the co-infected eclipse state if a TIP enters first.

$$\frac{dI_W}{dt} = k_W E_W - \left(\delta_W + \delta_N \frac{C}{K_F + C}\right) I_W - (1-\varepsilon_{\mathrm{Ab}})\,\beta_T I_W V_T \tag{3}$$

In words: Productive WT-only cells come from the eclipse pool, die at δ_W plus an extra cytokine-driven term, and become co-infected productive cells if a TIP enters.

$$\frac{dE_{\mathrm{Co}}}{dt} = (1-\varepsilon_{\mathrm{Ab}})(\beta_T E_W V_T + \beta_W P\,V_W) - k_{\mathrm{Co}} E_{\mathrm{Co}} \tag{4}$$

In words: Co-infected eclipse cells arise two ways: a TIP enters a WT-eclipse cell, or WT superinfects a primed cell (β_W P V_W). They exit eclipse at k_{Co}.

$$\frac{dP}{dt} = (1-\varepsilon_{\mathrm{Ab}})(\beta_T\,T\,V_T - \beta_W P\,V_W) - d_T P \tag{4b}$$

In words: Primed cells form when a TIP enters an uninfected cell, and are lost when WT enters them or when they die at d_T. Setting primed = 0 deletes the WT-entry term and gives abortive entry.

$$\frac{dI_{\mathrm{Co}}}{dt} = k_{\mathrm{Co}} E_{\mathrm{Co}} + (1-\varepsilon_{\mathrm{Ab}})\,\beta_T I_W V_T - \left(\delta_{\mathrm{Co}} + \delta_N \frac{C}{K_F + C}\right) I_{\mathrm{Co}} \tag{5}$$

In words: Productive co-infected cells come from E_{Co} or from TIP entry into productive WT cells, and die at δ_{Co} plus the cytokine-driven term.

Virus: asymmetric hijacking

Inside co-infected cells WT production is suppressed by the hijacking factor α ($0 \le \alpha \le 1$), and TIP production is enhanced by its shorter genome ($p_T > p_W$):

$$\frac{dV_W}{dt} = p_W I_W + \alpha\,p_W I_{\mathrm{Co}} - c_W(1 + \kappa\,\varepsilon_{\mathrm{Clr}})V_W \tag{6}$$

In words: Free WT virions are produced at p_W by WT-only cells but only at α p_W by co-infected cells; they are cleared at c_W, increased by ($1 + \kappa\ \varepsilon_{Clr}$) when antibody-like clearance is applied.

$$\frac{dV_T}{dt} = p_T I_{\mathrm{Co}} - c_T V_T \tag{7}$$

In words: Free TIP virions are produced only by co-infected cells at p_T and cleared at c_T. The size of c_T relative to c_W is the subject of the theory.

Cytokine and autonomic feedback

Cytokine synthesis has a basal rate s_0, infection-driven terms, and a second-order Hill positive-feedback term representing cytokine self-amplification. The whole synthesis term is scaled by a vagal gain m(H); clearance is first order at rate d_C. The gain is bounded ($0 \le \sigma_v < 1$). Anti-inflammatory suppression is strongest at resting pulse rate and fades as pulse rises toward H_{max}, which closes the cholinergic feedback loop:

$$\frac{dC}{dt} = m(H)\left[s_0 + \rho_I(I_W + I_{\mathrm{Co}}) + \rho_V V_W + \rho_T V_T + \gamma \frac{C^2}{K_C^2 + C^2}\right] - d_C\,C \tag{8}$$

In words: Cytokine is made at a basal rate, plus terms proportional to infected cells, WT virus and (optionally) TIP, plus a saturating self-amplification; the whole bracket is scaled by m(H) and removed at rate d_C.

$$m(H) = 1 - \sigma_v\,\nu(H), \qquad \nu(H) = \mathrm{clip}\left(\frac{H_{\max} - H}{H_{\max} - H_0},\ 0,\ 1\right) \tag{9}$$

In words: The vagal gain m(H) equals $1 - \sigma_v$ at resting pulse (maximum suppression) and rises to 1 as the pulse approaches H_{max}; the clip keeps it in [$1 - \sigma_v$, 1].

Pulse rate relaxes with time constant τ_H toward a set point that rises with cytokine concentration through a saturating Hill function, with maximal increment ΔH, half-effect constant K_H and cooperativity n:

$$\frac{dH}{dt} = \frac{1}{\tau_H}\left[H_0 + \Delta H \frac{C^n}{K_H^n + C^n} - H\right] \tag{10}$$

In words: Pulse rate is pulled toward a resting value H_0 plus an increment that grows with cytokine and saturates at ΔH, on a time scale τ_H (0.02 d, about 29 minutes, in the reference set).

5.3 Equilibrium and the monostability condition

Because the feedback and pulse-rate terms are saturating, the loop is bounded. At the infection-free equilibrium (T*, C*, H*), T* = λ/d_T and C* balances basal synthesis against clearance (the code chooses s_0 so that C* equals the input C_rest exactly). For the cytokine subsystem to have a single equilibrium, the steepest slope of the feedback term must not exceed the clearance slope. The maximum slope of $C^2/(K_C^2 + C^2)$ is $3\sqrt{3}/(8K_C)$, so with m at its upper bound of 1 a sufficient condition for monostability is:

$$\frac{3\sqrt{3}}{8}\frac{\gamma}{K_C} < d_C \tag{11}$$

In words: The steepest rate at which cytokine can amplify itself must stay below its first-order clearance rate. With the reference values γ = 3, K_C = 50, d_C = 5 the left side is about 0.04 against 5, so the condition holds with large margin.

Parameter sets that violate Equation 11 are not discarded: they are labelled as a bistable, “storm-class” regime and reported separately (the code issues a WARNING). The vagal loop adds gain that Equation 11 does not capture, so the full-system equilibrium is also checked numerically from multiple initial states. The unique-equilibrium criterion applies to the monostable regime.

5.4 Reference parameters and provenance

Kinetic parameters are to be drawn from published EBOV within-host studies. Cytokine and autonomic parameters are illustrative and will be calibrated against published clinical time courses in Phase 1. A parameter provenance register will record for each parameter its value, units, source, whether it was measured or assumed, and its plausible range. The defaults below are those of the file input.txt (Appendix B).

Table 5. Key parameters: code name, default, units and plausible range (provisional)

Symbol	Code name	Meaning	Default	Units	Plausible range
λ	lambda	Target-cell production	5×10^4	cells mL^{-1} d^{-1}	10^4–10^5
d_T	d_T	Target-cell death	0.07	d^{-1}	0.05–0.1
β_W	beta_W	WT infection rate	5×10^{-7}	mL $virion^{-1}$ d^{-1}	10^{-7}–10^{-6}
β_T	beta_T	TIP infection rate	5×10^{-7}	mL $virion^{-1}$ d^{-1}	0.5–2 × β_W
k_W, k_{Co}	k_W, k_Co	Eclipse exit rates	6	d^{-1}	4–8
δ_W, δ_{Co}	delta_W, delta_Co	Infected-cell loss	2	d^{-1}	1–3
p_W	p_W	WT production	500	virions $cell^{-1}$ d^{-1}	10^2–10^3
p_T	p_T	TIP production	1500	virions $cell^{-1}$ d^{-1}	2–5 × p_W
α	alpha	Hijacking factor	0.2	dimensionless	0.05–0.4
c_W	c_W	WT clearance	10	d^{-1}	5–15
c_T	c_T	Free-TIP clearance	10	d^{-1}	0.1–15 (unknown for TIPs)
δ_N, K_F	delta_N, K_F	Cytokine-driven killing (max, half-effect)	0.5, 50	d^{-1}, pg mL^{-1}	illustrative
C*	C_rest	Resting cytokine	5	pg mL^{-1}	illustrative
σ_v, H_0, H_{max}	sigma_v, H_0, H_max	Vagal gain, resting and maximum pulse	0.5, 70, 130	–, beats min^{-1}	illustrative
C_{crit}, H_{crit}	C_crit, H_crit	Safety bounds	500, 130	pg mL^{-1}, beats min^{-1}	to be set clinically
V_{W0}	V_W0	Initial WT inoculum	100	copies mL^{-1}	scenario

Units are fixed throughout: cells mL^{-1}; virions mL^{-1}; time in days. The baseline set is a single point inside these ranges chosen so that $R_0 > 1$ for WT alone; combinations with $R_0 \leq 1$ do not describe acute EBOV infection. In the reference set $R_0 = 8.73$ (the mid-range illustration in Section 5.5 gives 8.9).

5.5 Reproduction number and the extinction criterion

For WT alone, linearisation at the infection-free state gives the basic reproduction number. Eclipse phase cells are not lost in this model, so every infected cell eventually becomes productive and k_W does not appear in R_0:

$$R_0 = \frac{\beta_W p_W T^*}{c_W \delta_{\text{eff}}}, \qquad T^* = \frac{\lambda}{d_T}, \qquad \delta_{\text{eff}} = \delta_W + \delta_N \frac{C^*}{K_F + C^*} \tag{12}$$

In words: R_0 is the number of new infected cells one infected cell causes in a fully susceptible host: virus production per infected cell lifetime, times infection rate, times the susceptible pool T*, divided by virus clearance. δ_{eff} is the infected-cell loss rate at the resting cytokine level.

With an entry-blocking effect and enhanced clearance acting on WT alone, the effective reproduction number is:

$$R_{\text{eff}} = \frac{(1-\varepsilon_{\text{Ab}})\beta_W p_W T^*}{c_W(1+\kappa\varepsilon_{\text{Clr}})\delta_{\text{eff}}} = \frac{(1-\varepsilon_{\text{Ab}})R_0}{1+\kappa\varepsilon_{\text{Clr}}} \tag{13}$$

In words: Entry blocking multiplies R_0 by $(1 - \varepsilon_{Ab})$; enhanced clearance divides it by $(1 + \kappa\, \varepsilon_{Clr})$. Reff below 1 means the infection cannot sustain itself.

As an illustration only, mid-range values ($\lambda = 5\times10^4$, $d_T = 0.07$, $\beta_W = 5\times10^{-7}$, $\delta_W = 2$, $p_W = 500$, $c_W = 10$, cytokine term neglected) give $R_0 \approx 8.9$. An entry-only reduction would then have to exceed $1 - 1/R_0 \approx 89\%$ to bring R_{eff} below 1. This is the analytic benchmark for Hypothesis H1. R_{eff} in the presence of a TIP dose, which includes the hijacking effect, will be computed from the next-generation matrix of the infected subsystem (E_W, I_W, E_{Co}, I_{Co}, V_W, V_T) (van den Driessche and Watmough, 2002; Heffernan, Smith and Wahl, 2005). This is planned work; the current code computes only the entry-plus-clearance value.

A deterministic ODE never reaches exactly zero virus, so “sterilising clearance” needs an explicit definition. **Extinction is declared when V_W falls below V_{ext}**, set at one virion per total host blood volume (about 2×10^{-4} copies mL^{-1} for an adult of about 5 L), and stays below it for the rest of the run. Results are to be reported for a range of V_{ext} to show sensitivity to this choice. The engine offers an optional **extinction floor** that sets WT to zero once fewer than one virion and one infected cell per host remain. This is a deterministic proxy for stochastic extinction. Without it, residual virus below one copy per host can regrow, which is an artefact of the continuous model. A Gillespie-type stochastic cross-check is planned in Phase 2.

6 Computational Methods

6.1 Numerical scheme

Time stepping uses the classical fixed-step fourth-order Runge–Kutta method (Hairer, Nørsett and Wanner, 1993), written by hand in about ten lines (function rk4). Fixed-step RK4 is chosen for bit-level reproducibility: the same inputs give the same bits on any machine. An adaptive high-order solver is to be used offline, outside the engine, only to generate reference solutions for validation. The zero-dependency commitment applies to the shipped engine, not to those offline test tools.

Stability guard. RK4 is stable only if the step h times the fastest linearised rate stays below the real-axis stability limit of 2.785. Clearance and eclipse rates are about 10 d^{-1}, but infection terms such as $\beta_W \cdot V_W$ can reach tens to hundreds of d^{-1} at peak viraemia. Before each integration segment the engine therefore bounds the spectral radius of the Jacobian by its infinity norm (finite differences, function jac_bound), and reduces the step if h times that bound would exceed 2.5. The step is also justified by a step-halving study (the self-test, Section 7.1).

Invariants. After every step the engine checks that all states are finite and not significantly negative; otherwise the run stops with an error rather than returning a plausible-looking wrong answer.

6.2 One simulation

A run (function run) starts from the infection-free state (T = T*, C = C*, H = H*) plus a WT inoculum V_{W0}. At time t_{dose} (tip_time) a dose of free TIP is added instantaneously to V_T. The integration is split so that the dose lands exactly on its scheduled time. Peaks of V_W, C and H are tracked at every internal step, not only at output times, so coarse output grids do not hide peaks. Four running integrals (J_V, J_C, J_H, U) accumulate the terms of the objective so that weights can be changed afterwards without re-simulating.

6.3 The screening objective

A TIP profile θ comprises β_T, α, p_T, c_T, dose, dosing time and optional antibody-like effects. The weighted objective, minimised over $[0, t_f]$, is:

$$J(\theta) = \int_0^{t_f} \left[w_1 \log_{10}(1 + V_W) + w_2 \left(\frac{C - C^*}{C_{\text{crit}}} \right)^2 + w_3 \left(\frac{H - H^*}{H_{\text{crit}}} \right)^2 + w_4 \, u(\theta) \right] dt \quad (14)$$

In words: The cost J adds four things over the run: how much WT virus is present (on a log scale), how far cytokine and pulse stray from their resting values (scaled by the safety bounds, squared), and how much TIP is used. Smaller is better; the weights w_1–w_4 express priorities.

Hard constraints require peak C ≤ C_{crit} and peak H ≤ H_{crit}. In the code a profile that violates a bound is classed C (infeasible); a feasible profile that reaches extinction is class A (high priority); feasible without extinction is class B. The scalar score is F = J + penalty, where the penalty is 1000 times the relative excess over each bound plus w_ext times $\log_{10}$ of the final V_W relative to V_{ext}, which rewards approaching extinction smoothly.

6.4 Screening pipeline

1. **Latin hypercube sampling** (McKay, Beckman and Conover, 1979) of the profile space: each dimension is divided into equal slices, each used exactly once, so the space is covered evenly with a fixed seed.
2. **Evaluation** of every sample with the full simulator, in parallel on a worker pool.
3. **Ranking** by class, then by F, with ties broken by sample index so the order is total and reproducible.
4. **Pareto front** (non-dominated sorting) over peak V_W, peak C and TIP exposure among feasible samples.
5. **Nelder–Mead refinement** (Nelder and Mead, 1965) of the best few sampled profiles, in the unit cube with clamping.
6. **Weight sensitivity:** each weight is scaled by ×2 and ÷2 and the ranking recomputed, to see whether the winner is robust to subjective weights.
7. **Global sensitivity** by partial rank correlation coefficients (Marino *et al.*, 2008) of each swept parameter against $\log_{10}$ peak V_W, peak C and TIP exposure.
8. **Outputs:** a text report, a CSV of every sample for later analysis, and two SVG figures (Pareto scatter and best-profile trajectories).

6.5 Architecture: what exists and what is planned

Table 6 compares the planned architecture with what the code in Appendix C actually does. The distinction matters: a reader should not assume an optimisation exists merely because it was planned.

Table 6. Architecture status

Element	Planned design	Status in code.cpp
Language / dependencies	C++20, standard library only	**Implemented.** No third-party code
State layout	64-byte-aligned flat aggregate	Plain std::array of 14 doubles; alignment is a possible later optimisation
Integrator	Own-coded fixed-step RK4 + stability pre-flight	**Implemented** (rk4, jac_bound, advance)
Parallelism	Worker pool; thread-count-independent results	**Implemented** with std::thread and an atomic work counter (not jthread); verified identical for 1 and 4 threads
Ensemble layout	Structure-of-arrays tiling	**Not implemented**; one trajectory at a time
Android bridge	Zero-copy direct-buffer JNI	C API (text in, text out, malloc-owned strings) **implemented**; JNI/Kotlin layer is Phase 3 work
R_{eff} with TIP	Next-generation matrix	Not implemented (entry + clearance value only)
Stochastic check	Gillespie simulation of low-count phase	Tail-hybrid version **implemented** in Python (Section 12); C++ port pending
Adaptive immunity	Antibody, B-cell and CTL states	**Implemented** (rpt_adapt.cpp, Appendix D); cross-validated against the core

Element	Planned design	Status in code.cpp
Provenance register	Value, units, source, range per parameter	Provenance register to be completed in Aim 1; sources and calibrated values to be added

6.6 Input and output formats

Everything is plain text, so that the data outlive any particular software. Parameters are “name = value” lines (Appendix B, input.txt); a screening campaign is a list of “name = min max [log]” search dimensions plus control keys (screen.txt). Outputs are a whitespace-separated trajectory table, a text report, a CSV and SVG vector figures, all readable in a text editor. The C API for embedding is:

Table 7. C interface (the stable boundary for any future wrapper)

Function	Purpose
ebov_version()	Returns a version string
ebov_run(params_text, format)	One run from text parameters. format 0 = summary, 1 = trajectory table, 2 = SVG dashboard. Returns ERROR lines if input is invalid
ebov_screen(params, screen, prefix)	Full screening; writes four files with the given prefix and returns the report text
ebov_free(ptr)	Releases any string returned by the engine

6.7 Computational budget and how to rebuild everything

A 30-day trajectory at a step of 0.005 d needs about 6×10^3 steps, so 10^6 profiles need about 6×10^9. At a measured rate of about 2×10^6 RK4 steps per second per core (Section 7.4, one cloud core), that is roughly 50 minutes on one core, so campaigns of this size are feasible on a workstation, and plausibly on a mid-range phone (untested). To rebuild: compile code.cpp with any C++20 compiler (g++ -std=c++20 -O2 -pthread), run the self-test, run one trajectory with input.txt, then the screening with screen.txt; the 2000-profile reference campaign takes about 7 seconds on one core. Appendix C begins with the exact commands.

7 Verification, Validation and Results

7.1 Acceptance criteria

Table 8. Acceptance criteria and their status in November 2026

Test	Criterion	Evidence	Status
Convergence order	Observed RK4 order 3.8–4.2; agreement with independent reference to 10^{-8}	Step-halving; offline reference	Order **met**: 3.90 to 4.00 over six step sizes (Figure 5a). Offline reference pending
Equilibrium	Monostable regime returns to infection-free equilibrium; non-negativity and boundedness	Multi-start runs; invariant checks	**Met**: six perturbed starts return to equilibrium (Figure 5b)
Threshold behaviour	Hijacking just below/above $R_{eff} = 1$ gives growth vs monotone decay	Growth-rate sweep against linear theory	**Met**: zero crossing matches $1 - 1/R_0$ and $R_0 - 1$ (Figure 6)
Determinism and scaling	Bitwise-identical for all thread counts; efficiency ≥ 0.85 on four cores	Hash comparison; scaling report	Determinism **met** (identical SHA-256); scaling not yet measured; it will be measured in Aim 2 (M2)
Robustness	10^7 random inputs and sanitiser builds with zero crashes	Fuzz and sanitiser logs	Sanitiser build clean on self-test and a trajectory run; large fuzz campaign pending; it will run against the C++ port of the stochastic modules
Biological validation	Published EBOV and DIP kinetics reproduced within stated tolerances	Comparison to literature curves	Aim 1 (Phase 1 of Table 13; Table 14a)

Because no published TIP data exist for EBOV in vivo, TIP-specific behaviour will be validated qualitatively against published defective-interfering-particle kinetics and theoretical results, and this limitation will be stated in all reports.

7.2 Preliminary results from the prototype engine

The prototype engine was run on an illustrative baseline set inside the ranges of Table 5 (R_0 = 8.73; 30-day horizon; V_{ext} = 2×10^{-4} copies mL^{-1}; extinction floor on; primed cells tracked unless stated). The results demonstrate the screening workflow. They are not predictions, because cytokine and autonomic parameters are not yet calibrated.

Table 9. Prototype verification and screening results (prototype run)

Item	Result
RK4 convergence order	4.04 (step halving; criterion 3.8–4.2)
Determinism and robustness	Screening output bitwise identical on 1 and 4 threads; sanitiser build clean
Search space	Latin hypercube, 2000 profiles over β_T, α, p_T, c_T, dose and dosing day; about 5 s on two cores
Untreated reference	Peak V_W 1.3×10^{7} copies mL^{-1}, peak cytokine about 207 pg mL^{-1}, peak pulse about 110 beats min^{-1}
c_T = 5–15 d^{-1}	0 of 2000 profiles reach WT extinction; TIP cleared within about 2 days and residual virus regrows
c_T = 0.5–15 d^{-1}, abortive-entry variant	49 of 2000 profiles (2.5%) reach extinction, all with c_T between 0.50 and 0.94 d^{-1}
c_T = 0.5–15 d^{-1}, primed cells tracked (reference)	0 of 2000 profiles reach extinction over the same range and seed
c_T = 0.1–15 d^{-1}, primed cells tracked	33 of 2000 (1.7%) reach extinction, all with c_T between 0.10 and 0.38 d^{-1} and dose between 2.7×10^{5} and 10^{9} copies mL^{-1}
Direct clearance sweep (three profiles): primed vs abortive thresholds for c_T	Strongest (dose 10^{9}, day 1): 0.46 vs 1.15 d^{-1}. Screened best: 0.26 vs 0.69 d^{-1}. Weaker (dose 10^{8}, day 1): no primed-cell extinction down to 0.003 vs 0.53 d^{-1} (Figure 2)
TIP-driven cytokine stimulation (ρ_T)	Raising ρ_T from 0 to 10^{-3} raises the c_T threshold of the screened best profile from 0.26 to 0.36 d^{-1}, but peak cytokine rises from about 5 to about 6×10^{3} pg mL^{-1}, far above the 500 pg mL^{-1} bound
Best profile (Nelder–Mead refined)	β_T 2.0×10^{-6}, α 0.05, p_T 1779, c_T 0.10 d^{-1} (at the lower bound), dose 2.2×10^{6} at day 0.03: WT extinct at day 20; peak cytokine about 5 pg mL^{-1}, pulse about 70 beats min^{-1}
Dominant parameter (PRCC)	Dosing day is strongest for peak V_W (+0.87); c_T is strongest for TIP exposure (−0.91)
Weight sensitivity	Top-ranked profile stays first in 6 of 8 weight-scaling scenarios (worst rank 2); still weakly informative while the cytokine constraint is inactive

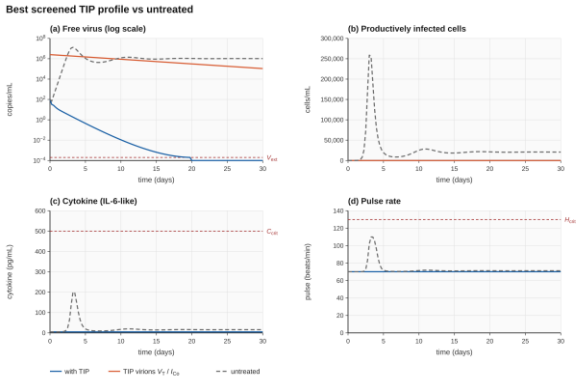


Figure 1. Best screened TIP profile (solid) against the untreated run (dashed), primed cells tracked. Illustrative parameters.



Figure 2. Time to WT extinction against free-TIP clearance rate c_T for three TIP profiles. Strongest: refined profile (analysis_scripts.REFINED) at dose 10^9, day 1; Best: the screened best profile (screen_report.txt); Weaker: the WEAK profile of make_figures_analysis.py ($\beta_T = 5\times10^{-7}$, $\alpha = 0.2$, $p_T = 1500$, dose 10^8, day 1). Solid: primed cells tracked (reference); dashed: abortive-entry variant; triangles: thresholds c_T^* (primed 0.456 and 0.256; abortive 1.151, 0.696 and 0.525; the weaker profile has no primed extinction); shaded: WT clearance range 5–15 d^{-1}. Reproduce: make_figures_analysis.py fig2; data in figure_data/.

7.3 Interpretation

First, tracking TIP-primed cells makes the extinction requirement stricter. If a TIP entering an uninfected cell is treated as a pure loss of that cell (abortive entry), 49 of 2000 sampled profiles clear WT with c_T between 0.5 and 0.94 d^{-1}. When primed cells are tracked and remain open to WT, no sampled profile cleared WT with c_T of at least 0.5 d^{-1}, and extinction needed c_T below about 0.4 d^{-1} (half-life about 1.7 days or more), against 5–15 d^{-1} for WT virus. The threshold depends on the profile. Because TIPs are virion-like, so large a difference is not obviously realistic; the result is a design requirement that experiments must check, not a general property of TIPs.

Second, TIP-driven cytokine stimulation raises the threshold only by pushing cytokine far past its bound, so it does not rescue a fast-cleared TIP safely. **Third,** no extinct profile at $\rho_T = 0$ caused a cytokine excursion, so the safety constraint is not yet discriminating; it becomes informative only after cytokine parameters are calibrated in Phase 1. **Fourth,** extinction here relies on the deterministic floor, and the stochastic cross-check will test whether it survives low-count noise.

7.4 Reproduction of the reference campaign

The annotated source of Appendix C was compiled from a clean build (g++, C++20, -O2) and the principal results were regenerated. The annotated source is identical to the unannotated program once comments are removed (checked by comparing the comment-stripped preprocessor output), and every output file was byte-identical to that of the unannotated program. The environment was a single cloud CPU core, so absolute timings are indicative only.

Table 10. Reproduction of the reference campaign (input.txt, screen.txt, seed 12345)

Item	Result
RK4 self-test	Order estimate 4.037 ($e_1 = 2.37\times10^{-10}$, $e_2 = 1.44\times10^{-11}$): PASS
Reference R_0; untreated peaks	$R_0 = 8.73$; peak V_W 1.311×10^7, peak C 207, peak H 110.5
Single run, default input (TIP dose 10^7, c_T = 10 d^{-1})	Peak V_W 1.87×10^6 at day 2.8 (86% below untreated); no extinction by day 30
Screening, 2000 profiles	6.7 s on one thread; all 2000 valid and feasible; **33 feasible with WT extinction**; Pareto front of 11 profiles
Best profile before / after refinement	F = 211 (sampled) → 25.7 (Nelder–Mead): β_T 2.0×10^{-6}, α 0.050, p_T 1779, c_T 0.10, dose 2.24×10^6, day 0.029; extinct (day 20.0 in the report)
Extinction threshold, screened-best sampled profile	Bisection on c_T: **0.256 d^{-1} with primed cells, 0.696 d^{-1} abortive, matching Section 7.2 (0.26 vs 0.69)**
Determinism	CSV of all 2000 samples has identical SHA-256 (8408cc08…) on 1 and 4 threads

Item	Result
Sanitiser build	AddressSanitizer and UndefinedBehaviorSanitizer: self-test and a trajectory run clean
C-API test (test_api.cpp)	Summary, CSV, SVG and screening calls all return; invalid input returns an ERROR line

A short sweep for the Nelder–Mead-refined best profile illustrates the persistence requirement directly (Table 11). The values are re-entered from the report rounded to four significant figures; this moved the extinction day from 20.0 to 19.5 and shows how sensitive trajectories near the threshold are. The screen bound was $c_T \in [0.1, 15]$; the optimizer sits at the floor, so the true optimum may lie below 0.1 d^{-1}. This does not change c_T*, which is computed by bisection, but it means the ranked shortlist is conservative on the clearance axis.

Table 11. Free-TIP clearance sweep, refined best profile (dose 2.24×10^6 at day 0.029)

c_T (d^{-1})	Primed cells tracked (reference)	Abortive-entry variant
0.05	Extinct, day 14.6	Extinct, day 6.0
0.1	Extinct, day 19.5	Extinct, day 6.0
0.2	No extinction (peak C 5.6)	Extinct, day 6.4
0.5	No extinction	Extinct, day 27.9 (peak C 29)
0.7	No extinction	No extinction
10 (default)	No extinction (peak C 20.7)	No extinction (peak C 115)

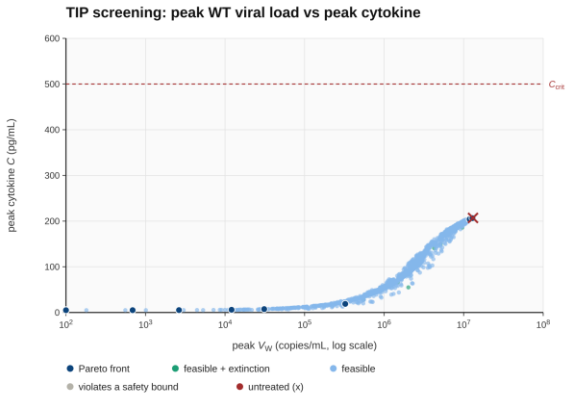


Figure 3. Engine-generated screening output (screen_pareto.svg, 2000 profiles): peak WT load vs peak cytokine. Dark blue: Pareto front; green: feasible with extinction; red cross: untreated. No profile nears the cytokine bound, so the safety constraint is not yet discriminating.



Figure 4. Untreated infection against a weaker TIP profile (WEAK in make_figures_analysis.py: $\beta_T = 5\times10^{-7}$, $\alpha = 0.2$, $p_T = 1500$, dose 10^8, day 1) and the best screened profile ($c_T = 0.18\ d^{-1}$; screen_report.txt), illustrative parameters: (a) WT virus over time; (b) peak load (untreated 1.3×10^7, weaker 4.7×10^5, best 100 = inoculum, i.e. extinct); (c) peak cytokine against its safety bound (207, 31 and 5.0 against $C_{crit} = 500$). Reproduce: make_figures_analysis.py fig4; data in figure_data/.

7.5 Extended verification of the numerical core

Three acceptance criteria that were pending in Table 8 have been tested with the compiled engine. The adaptive-immunity extension (Section 11) was first cross-validated against the core: with its adaptive block frozen it reproduces the core peaks and extinction outcomes at every parameter set tried (seven sets, agreement to the printed four significant figures) and the same persistence thresholds (0.2560 d^{-1} with primed cells, 0.6964 d^{-1} in the abortive-entry variant). All figures in Sections 7.5 and 10–12 were then generated with that engine.

Convergence order. The final state after 6 days of untreated infection was computed with six step sizes from h = 0.05 d to 0.0016 d and compared with a reference at h = 0.05/128. The maximum relative error falls from 2.7×10^{-5} to 2.9×10^{-11} and the local convergence order rises monotonically from 3.90 to 4.00 (Figure 5a), as expected for a fourth-order method. The stability guard is switched off for this study (guard = 0), otherwise it would override the coarse steps.

Return to the infection-free equilibrium. Six runs were started from strongly perturbed states (target cells from 0.05 to 2.5 times T*, cytokine from 0 to 300 pg mL^{-1}, pulse from 50 to 130 beats min^{-1}) without any virus. All return to (T*, C*, H*) (Figure 5b). The slowest mode is the target-cell turnover, with rate $d_T = 0.07$

d^{-1}, so deviations fall by a factor e^{−0.07 t}, reaching about 10^{-12} after 400 days. When only cytokine and pulse start away from equilibrium, the fast modes decay to machine precision (10^{-15}) within about ten days.

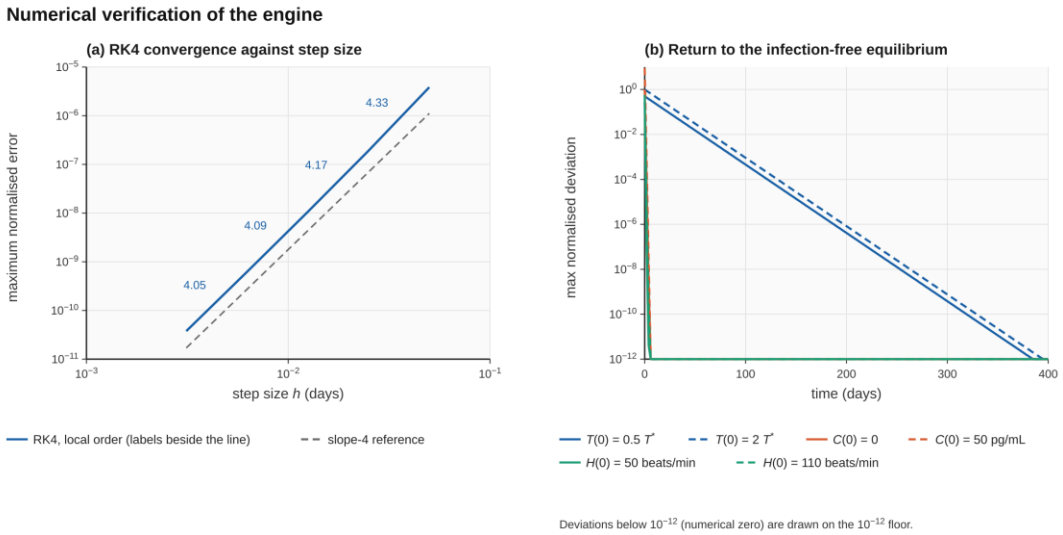


Figure 5. *Numerical verification. (a) Maximum error of RK4 against step size h = 0.05 to 0.003125 d (reference h = 0.0004 d, 30 days, extinction floor and guard off), with the slope-4 reference; local orders 4.33, 4.17, 4.09 and 4.05 between successive steps. (b) Maximum normalised deviation from the infection-free equilibrium over 400 days for six perturbed starting states (T_0 factor 0.5 or 2, C_0 0 or 50, H_0 50 or 110), no virus and no TIP; deviations below 10^{-12} are drawn on the floor. Reproduce: make_figures.py fig5; data in figure_data/.*

Threshold behaviour at R_{eff} = 1. Near the threshold the criterion "peak V_W ≤ inoculum" is unreliable because a slowly growing infection first dips and later recovers. The test was therefore made on the asymptotic exponential growth rate r of V_W, measured in the linear regime ($V_{W0} = 10^{-12}$, so that no target cells are depleted), and compared with the largest eigenvalue of the linearised infected subsystem, which satisfies the characteristic equation

$$(r + k_W)(r + \delta_{\text{eff}})(r + c_{\text{eff}}) = k_W p_W \tilde{\beta} \tag{16}$$

In words: The growth rate r is the largest real root of this cubic. At r = 0 it reduces to $k_W\, \delta_{eff}\, c_{eff} = k_W\, p_W\, \tilde{\beta}$, which is exactly $R_{eff} = 1$.

$$\tilde{\beta} = (1 - \varepsilon_{\text{Ab}})\, \beta_W\, T^{*}, \qquad c_{\text{eff}} = c_W(1 + \kappa\, \varepsilon_{\text{Clr}})$$

In words: The effective infection rate and clearance entering Eq 16.

Simulated and theoretical growth rates agree to within 8×10^{-5} d^{-1} over the whole grid (Figure 6). Bisection on the sign of r gives the critical entry-blocking effect 0.88545 against the analytic $1 - 1/R_0 = 0.88545$, and the critical clearance gain 7.7302 against $R_0 - 1 = 7.7302$. Growth versus decay therefore switches exactly where Equation 13 says it should, which closes the threshold-behaviour criterion of Table 8.

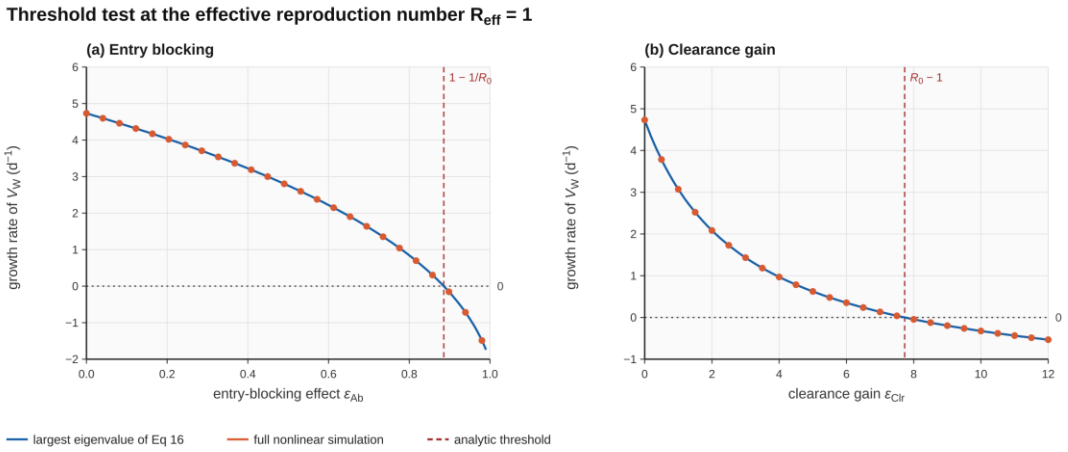


Figure 6. *Threshold test. Asymptotic growth rate of WT virus against (a) entry-blocking effect and (b) clearance gain. Lines: largest eigenvalue of Eq 16; points: full nonlinear simulation. Dashed red lines mark the analytic thresholds $1 - 1/R_0$ and $R_0 - 1$; the simulated zero crossings coincide with them.*

7.6 Verification record

The verification evidence is collected in one place because it is the part of this work that does not depend on illustrative parameters. Every line is regenerated by a script in the repository and is re-run on each push by continuous integration; none is a hand-typed number. These checks establish that the code solves the stated equations correctly. They say nothing about whether the equations describe a real TIP, which is the question of Aim 1 and the experiments of Table 20.

Table 11a. Verification record

Check	Result	Reproduced by
RK4 convergence order	Order 4.037 (e1 = 2.37×10^{-10}, e2 = 1.44×10^{-11}): PASS against the 3.8–4.2 criterion	./engine --selftest input.txt
Bitwise determinism	Screening CSV of 2000 profiles has the same SHA-256 (8408cc08…) on 1 and 4 threads	check_determinism.sh
Threshold behaviour	Bisection on the sign of the largest eigenvalue gives $1 - 1/R0$ = 0.885454545 and $R0 - 1$ = 7.730158730, matching the analytic values to seven or more digits (differences below 10^{-14}); full nonlinear runs agree in growth rate to better than 8×10^{-5} d^{-1}	verify_threshold.py; make_figures_analysis.py fig6
Equilibrium	Six strongly perturbed starts return to the infection-free equilibrium; no drift without perturbation	make_figures.py (Figure 5b)
Adaptive extension against core	With the adaptive block frozen, seven parameter sets agree to four significant figures and give the same thresholds (0.2560 and 0.6964 d^{-1})	Section 7.5; rpt_adapt.cpp
Sanitisers	AddressSanitizer and UndefinedBehaviorSanitizer report nothing on the self-test and a trajectory run	check_sanitisers.sh
Figures	17 of 17 figures regenerate from code; every plotted point is converted back to data units and compared with engine output (trajectory points within 2×10^{-5} of the axis range, 2000 of 2000 Pareto circles match)	run_all.sh; verify_figures.py
Continuous integration	Engine build and self-test, sanitiser and determinism checks, figure and manuscript rebuild, and the Android build run on every push	.github/workflows/build.yml

Not yet measured: parallel scaling (the efficiency criterion of Table 8) and a large fuzz campaign. Both remain open and are listed as such in Table 8. Parallel scaling will be measured in Aim 2 (M2); the fuzz campaign will run against the C++ port of the stochastic modules.

8 Limitations, Risks, Ethics and Safety

8.1 Limitations

The model is a compact mechanistic abstraction. TIP-primed cells and TIP-driven innate stimulation (as a sensitivity parameter, default zero) are represented. It still does not capture adaptive immune memory in detail, tissue-compartment heterogeneity, interferon-mediated protection of primed cells, or co-infection with other pathogens. It models within-host dynamics only and does not address transmission or evolution of resistance. Mass-action mixing treats the body as a single well-stirred compartment. Outputs are simulated trajectories under stated assumptions; they are not patient-level predictions, dosing instructions or manufacturing recommendations, and they require experimental confirmation.

Specific cautions for the next researcher: (i) all numbers in Section 7 are conditional on illustrative parameters; (ii) the extinction result depends on the deterministic floor and on V_{ext}; (iii) results near the clearance threshold are sensitive to rounding (Section 7.4); (iv) the cytokine safety constraint did not bind in any tested profile, so safety rankings are currently uninformative. It would bind in either of two ways: if free TIP stimulates the innate response above about $\rho T = 10^{-4}$ (pg mL^{-1} d^{-1} per copy mL^{-1}), at which point peak cytokine passes the 500 pg mL^{-1} bound (Figure 12 and Table 20), or if calibrated cytokine parameters violate the monostability condition of Equation 11, which places the host in the storm-class regime. Calibration of the cytokine block in Aim 1 decides which, if either, applies.

8.2 Risks and mitigation

Table 12. Risks and mitigation

Risk	Mitigation
Parameters poorly identifiable from published data	Use ranges, global sensitivity analysis, and report ranking robustness rather than point predictions
Published kinetics cannot be reproduced within tolerance	Fix the tolerance in Phase 1; document discrepancies and recalibrate cytokine and autonomic parameters
No in vivo EBOV TIP data for validation	State the limitation; validate against defective-interfering-particle literature and theory
Fixed-step RK4 inaccurate at extreme parameters	Step-halving checks; constrained parameter bounds; stability pre-flight; offline adaptive solver for comparison
Schedule slippage (single part-time researcher)	Phases 3–4 are decoupled from the science, and the engine remains usable from the command line if the interface is delayed
Free-TIP clearance dependence reflects model structure	Primed cells are included and results with and without them are reported; because the dependence is stricter with them, it is treated as a design requirement until tested against further structural extensions and experiments

8.3 Ethical, biosafety and dual-use considerations

- **No pathogen work.** The study is purely in silico. Biosafety review is required only to confirm scope.
- **Dual use.** Inputs and outputs are abstract kinetic parameters. The engine produces no genetic sequences, construct designs or protocols, and this document contains none. Parameter libraries are restricted to published therapeutic and vaccine-research contexts. Anyone extending the engine toward sequence-level design should seek independent biosafety and dual-use review first.
- **Human participants.** The usability study will be run only after institutional ethics approval and, where applicable, a research permit from the National Commission for Science, Technology and Innovation (NACOSTI). Any personal data will be handled under the Data Protection Act, No. 24 of 2019 (Republic of Kenya, 2019).

9 Work Plan, Outputs and Conclusion

9.1 Work plan

Durations assume part-time effort by a single researcher.

Table 13. Phases, deliverables and exit criteria

Phase	Months	Activities and deliverables	Exit criteria
0. Preparation	1	Literature review; ethics and biosafety statement; finalise parameter ranges	Plan finalised; draft provenance register
1. Mathematical core	1–3	RK4 library; co-infection kernels; equilibrium analysis; reproduction of published kinetics; implementation and calibration of the TIP-primed cell compartment and test of the free-TIP clearance dependence (H4); TIP immunogenicity sensitivity (ρ_T, default 0); numerical stress suite	Order, equilibrium, invariant and fuzz criteria met; C++ ≥ 99% of lines
2. Screening module	4–5	LHS and Nelder–Mead search; Pareto front; weight and global sensitivity analysis; stochastic extinction check; scaling report	Reproducible ranked output; efficiency ≥ 0.85
3. JNI bridge	6–7	Native-owned aligned buffers; versioned layouts; lifecycle tests	Correct under 10^5 bind–run–close cycles
4. Kotlin interface	8–10	Constrained input forms; ranked-profile display; assumption statements; local save/load; usability study	Usability study completed
5. Reporting	11–12	Final analysis of H1–H4; final report; manuscript draft	Report and manuscript completed

Table 14. Indicative timeline (months counted from the start of the project)

Phase / month	1	2	3	4	5	6	7	8	9	10	11	12
0. Preparation	■											
1. Mathematical core	■	■	■									
2. Screening module				■	■							
3. JNI bridge						■	■					
4. Kotlin interface								■	■	■		
5. Reporting											■	■

Funding-aligned structure. The phases above give the engineering order. For review, the same work is organised as three aims over 24 months, and the first aim is calibration (Table 14a). A version of 12 to 18 months, sized for a Grand Challenges-type award, would consist of Aim 1, the C++ port of the stochastic and establishment modules from Aim 2, the usability study and an interim shortlist. A two-year version, sized for an R21-type award, adds the tropism and compartment extensions of Aim 2 and the full deliverables of Aim 3. “Phase 1” elsewhere in this document means Aim 1.

Table 14a. Aims, months, deliverables and stop rules

Aim	Months	Work	Exit criterion and stop rule
1. Calibrate	1–8	Calibrate the core and adaptive constants to published EBOV within-host kinetics and vaccine titres; complete the provenance register; publish the calibrated parameter set	Peaks and timings within the pre-registered tolerance (Table 21a). If not met, the quantitative claims are withdrawn and the project stops at the structural results
2. Extend	6–16	Port the stochastic and establishment modules to C++; add the tropism and compartment extensions with conservation checks; recompute cT* with sanctuary compartments	C++ output identical to Appendix E; conservation checks pass; cT* with sanctuary compartments reported
3. Deliver	14–24	Deliver the ranked, uncertainty-bounded shortlist and the pre-registered falsification protocol; open release; methods paper	Shortlist with uncertainty bounds; protocol time-stamped before the final runs; engine, scripts, parameter files, manuscript and APK released under the MIT licence

9.2 Resources and costs

Environment. The host institution is Maasai Mara University, Kenya. The work runs on a workstation or laptop with a C++20 toolchain and the Android NDK, with a mid-range Android device for performance testing, and every result in this document regenerates on such a machine with run_all.sh. No wet-laboratory partner is named: Aims 1–3 are computational and no experimental validation is claimed.

Budget scope. Personnel is the dominant cost, including a part-time statistician or research assistant to support rigour. The remaining items are a workstation, an Android test device, open-access publication fees, ethics approval and participant compensation for the usability study, and small travel. No wet-laboratory costs are included because none of the aims requires them. Table 14b gives indicative amounts in US dollars for a lean version and a fuller version of the 24-month project. They are planning estimates, to be replaced by the research office's rates at the start of the project.

Table 14b. Indicative budget over 24 months, lean and fuller versions (US dollars)

Item	Basis	Lean (USD)	Fuller (USD)
Personnel: PI	Part-time over 24 months (about 20% effort lean, 50% fuller)	8,000	22,000
Personnel: statistician / RA	Part-time support for calibration, uncertainty analysis and the usability study (about 6 months lean, 15 fuller)	3,500	12,000
Workstation	C++20 and Android NDK development and compute	1,800	3,500
Android test device	Mid-range handset for performance testing (one lean, three fuller)	300	900
Open-access fees	Methods paper (one lean, two fuller)	1,500	4,500
Usability study	Ethics review and participant compensation for laboratory staff (about 10 lean, 25 fuller)	1,200	4,500
Travel	Partner visits; one conference in the fuller version	1,500	6,000
Subtotal		**17,800**	**53,400**
Overhead (if applicable)	Indicative 10% of subtotal; actual rate set by the university and the funder	1,780	5,340
Total	**Lean about $19.6k; fuller about $58.7k**	**19,580**	**58,740**

The lean version funds Aim 1, the C++ port from Aim 2, the usability study and an interim shortlist. The fuller version funds all three aims, the full usability study and dissemination. No wet-laboratory costs are included. Subaward or foreign-institution rules and indirect-cost caps differ by funder and are applied once the funder is chosen.

9.3 Expected outputs

- A validated open C++20 engine and Android application, with a parameter provenance register.
- A shortlist of high-ranking theoretical TIP profiles that reach extinction of WT virus within the safety bounds, with predicted R_{eff}, time to extinction and sensitivity analysis.
- A quantitative test of H1–H4 and of the six falsifiers F1–F6 (Table 3a), including whether single-mechanism profiles fail the $R_{eff} < 1$ threshold while combined profiles succeed.
- A decisive answer. Either the calibrated model reproduces published EBOV kinetics within the pre-registered tolerance and the shortlist is issued with uncertainty bounds, or it does not and the quantitative claims are withdrawn. Both outcomes are reported as results (Section 14.3).
- Sensitivity maps showing which parameters most affect ranking, so experimental groups can prioritise the highest-leverage measurements.
- Plain-language assumption statements suitable for laboratory reports and ethics submissions, a final report, and a manuscript for peer reviewed submission.

9.4 Open questions

The following questions, if answered, would most change the standing of this theory. They are listed in the order in which they would be cheapest to settle.

1. What is the effective in vivo half-life of an engineered filovirus-derived TIP-type particle? (Decides P3.)
2. Are cells entered by a TIP, but not yet infected by WT virus, protected by interferon or other innate mechanisms? (Decides P2.)
3. Do calibrated cytokine and pulse parameters make the safety bounds bind for any effective profile? (Decides whether P4 matters in practice.)
4. Does extinction survive demographic noise in a stochastic model? (Falsifier F4; addressed in Section 12.2.)
5. Can the model be extended with adaptive immunity so that vaccination and TIP therapy can be modelled together, and does the persistence requirement relax in a vaccinated host? (Addressed in Section 11.)

9.5 Conclusion of Part I

This work sets out a verifiable, offline simulation engine for EBOV–TIP co-infection that links viral dynamics to cytokine and autonomic safety constraints and searches the TIP design space systematically, and it names and states the theory the engine supports: that TIP success against *Orthoebolavirus zairense* is limited by the persistence of the free particle. The value of the work lies in narrowing the field of candidate profiles, showing which parameters matter most, and doing so in a form that laboratories with limited connectivity can use. The work plan, acceptance criteria and stated limitations are designed so that each claim made by the engine can be checked, and the theory can be refuted. The clearance requirement on the free TIP is the main result for experiments to test. If it survives, it points toward the measurements that matter most for prioritising candidate designs; if it fails, it will have saved effort that would otherwise have gone to the wrong particle. Part II tests these claims further.

10 Mechanism and Robustness of Persistence-Limited Interference

Part II of this document continues the work of Sections 1–9. It asks four questions of the core theory: where exactly is the boundary between extinction and regrowth, what happens near it, which design parameters and biological parameters move it, and does it survive the addition of adaptive immunity and of demographic noise? All calculations use the compiled engine and the fixed profile of Section 7.3 (the best sampled profile: $\beta_T = 8.13\times10^{-7}$, $\alpha = 0.050$, $p_T = 2877$, dose 3.8×10^{7} copies mL^{-1} at day 0.26) unless a quantity is being swept.

10.1 Structure of the extended model

Figure 7 shows the structure that the rest of the document works with. The core model has ten states: target cells, the four infected-cell classes, primed cells, the two virus pools, cytokine and pulse. The adaptive extension of Section 11 adds three states, the antibody-secreting pool B, the neutralising antibody concentration A and the cytotoxic T-cell pool Z. Antibody acts on the core only through the two effect functions ε_{Ab} and ε_{Clr} that already appear in Equations 1–7, and the CTL pool acts as an extra infected-cell death term, so the core equations keep their form.

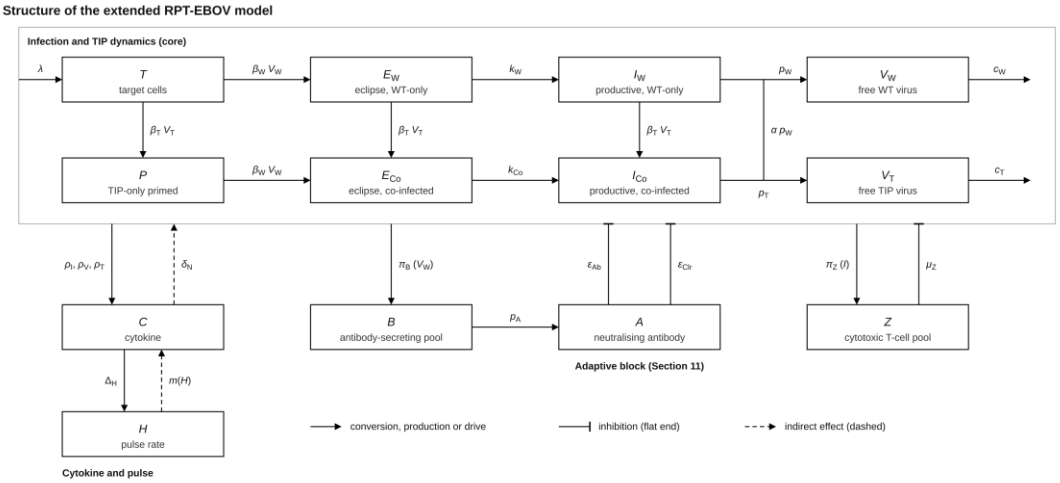


Figure 7. *Structure of the extended model. Top: the ten-state core (target cells T, WT-only infected cells E_W and I_W, co-infected cells E_{Co} and I_{Co}, primed cells P, free WT and TIP virus, with clearance c_W and c_T). Bottom left: cytokine C and pulse H with vagal feedback. Bottom right: the adaptive block B, A, Z, acting on the core through entry blocking, enhanced clearance and CTL killing.*

10.2 The persistence boundary in dose clearance space

Figure 8 shows, for 1728 combinations of free-TIP clearance and dose, whether WT virus is driven to extinction within 30 days. The extinction region is a compact set at small c_T: no combination with c_T above about 0.55 d^{-1} succeeds with primed cells tracked (above about 1.4 d^{-1} in the abortive variant), and the whole WT clearance range of 5–15 d^{-1} is far outside the region. Ignoring primed cells roughly doubles the extent of the region along the clearance axis, which is the graphical form of Postulate P2.

The boundary is not a simple monotone curve. Between doses of about 5×10^{5} and 6×10^{6} copies mL^{-1} it contains a notch, in which the critical clearance falls to a minimum of 0.04 d^{-1} near 1.8×10^{6} copies mL^{-1} (Section 10.4). Outside the notch the critical clearance rises slowly with dose, from 0.25 d^{-1} at 10^{5} to 0.53 d^{-1} at 10^{9}, so a four-orders-of-magnitude increase in dose buys only a factor of about two in tolerable clearance. The persistence requirement cannot be bought off with dose.

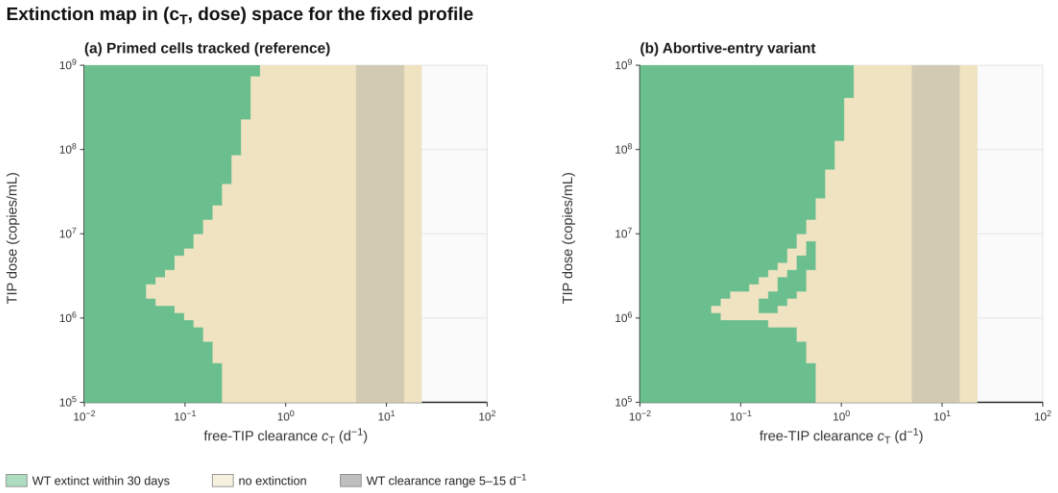


Figure 8. *Extinction map in (c_T, dose) space for the fixed profile, a 36 × 48 grid (c_T 0.01–20 d^{-1}, dose 10^5–10^9). Green: WT extinct within 30 days; beige: no extinction. (a) Primed cells tracked (reference model); (b) abortive-entry variant. The grey band is the WT clearance range, 5–15 d^{-1}. Reproduce: make_figures_analysis.py fig8; data in figure_data/.*

10.3 Dynamics near the boundary

Figure 9 follows seven trajectories that straddle the threshold of 0.256 d^{-1}. For c_T = 0.05 and 0.1 d^{-1} the virus falls roughly log-linearly, one decade every 2.4 days, and crosses V_{ext} at day 13–14. For 0.2 d^{-1} and above, V_W falls to a minimum and then regrows, and the trough is deeper the smaller c_T is: about 10^{-3} copies mL^{-1} (five virions per host) at 0.2 d^{-1}, above V_{ext}, after which the virus regrows. Free TIP decays smoothly when it is not being amplified, and oscillates in anti-phase with WT virus once regrowth starts, as expected for an interfering particle that feeds on its target (panels b and c). Section 12 shows that the regrowth seen here is partly an artefact of treating one-copy quantities as continuous.

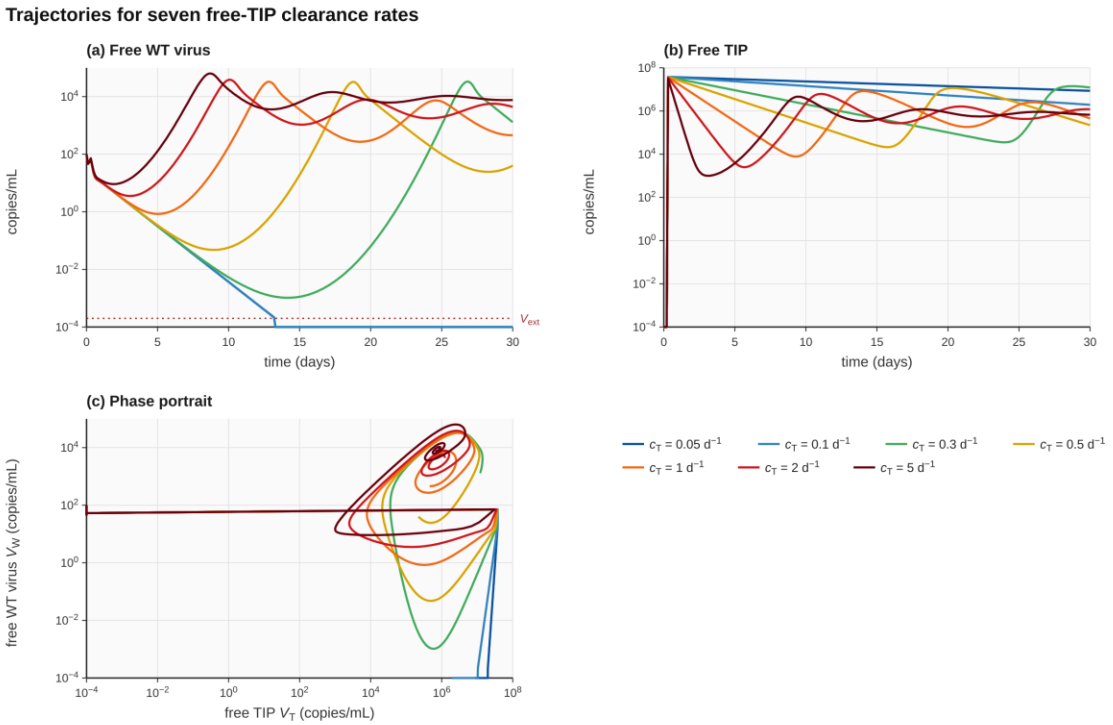


Figure 9. *Trajectories for seven values of c_T (0.05, 0.1, 0.3, 0.5, 1, 2 and 5 d^{-1}; colour key below the panels) with the fixed profile and the extinction floor on. (a) Free WT virus with the extinction threshold V_{ext}; (b) free TIP; (c) phase portrait of V_W against V_T. Extinction occurs for c_T = 0.05 and 0.1 (day 13.3); all larger values regrow, consistent with c_T^* = 0.256 d^{-1}. Reproduce: make_figures_analysis.py fig9; data in figure_data/.*

10.4 Dependence on TIP design parameters

Figure 10 shows how the critical clearance c_T^* changes when one design parameter is varied and the others are held at the profile values. Four findings stand out.

- **Hijacking is the strongest design lever.** With primed cells tracked, extinction at α = 0.02 needs c_T < 0.51 d^{-1}, at 0.045 it needs c_T < 0.29 d^{-1}, at 0.07 it needs c_T < 0.15 d^{-1}, and at α = 0.095 and above (the next grid point) no extinction is possible for any c_T down to 0.003 d^{-1}. The abortive variant hides this limit completely (it still predicts extinction at α = 0.6). A TIP that merely halves WT output cannot clear the virus in this model.
- **TIP production advantage is irrelevant at this dose.** The threshold is constant to three digits (0.256 d^{-1}) as p_T/p_W is varied from 1 to 10. The injected dose of 3.8×10^7 copies mL^{-1} dominates the free-TIP pool, so amplification inside co-infected cells does not change when extinction succeeds. Production advantage is expected to matter at low doses, where the TIP has to propagate itself, and this is not covered by the fixed-dose scan.
- **Entry efficiency helps weakly.** Raising β_T from 0.5 to 8 times β_W raises c_T^* from 0.16 to about 0.4 d^{-1}, a factor 2.5 for a factor 16.
- **Dose has a resonance.** c_T^* drops to 0.04 d^{-1} near a dose of 1.8×10^6 copies mL^{-1} and is higher on both sides (0.25 d^{-1} at 10^5, 0.53 d^{-1} at 10^9). The cause has not been analysed; the notch is also visible in Figure 8 and survives in the abortive variant, so it is not a primed-cell effect. It is listed as an open question in Section 14.

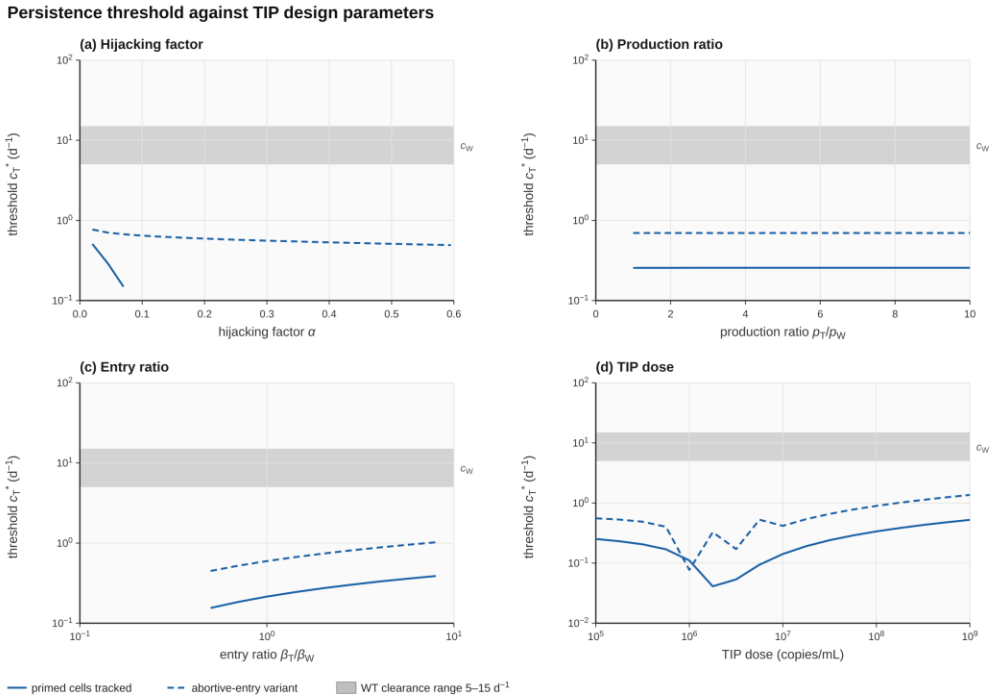


Figure 10. *Persistence threshold c_T^* against (a) hijacking factor α, (b) production ratio p_T/p_W, (c) entry ratio β_T/β_W and (d) TIP dose, with primed cells tracked (solid) and in the abortive variant (dashed). Grey band: WT clearance range. In (a) the reference model has no extinction for α of 0.095 and above (last extinct grid point α = 0.07). Reproduce: make_figures_analysis.py fig10; data in figure_data/.*

10.5 Local sensitivity to biological parameters

The sensitivity of c_T^* to ±25% changes of fifteen parameters is shown in Figure 11. The result is simple. Everything that enters R_0 shifts the threshold by a similar amount in the same direction: λ, β_W and p_W are exactly equivalent (they appear only as the product), each +25% moves c_T^* by −33% and −25% by +48%; the infected-cell loss rate $\delta_W = \delta_{Co}$ is the strongest parameter (+25% gives +54%, −25% gives −48%); the WT clearance c_W and target-cell death d_T follow. TIP-side parameters matter much less (β_T and dose about ±10%, p_T not at all), and the cytokine parameters δ_N and K_F, the inoculum V_{W0} and the dosing time move c_T^* by less than 3%.

Two consequences follow. First, the persistence requirement is controlled mainly by how aggressive the infection itself is (R_0), not by TIP engineering, so a calibrated R_0 is the single most valuable measurement for sharpening the prediction. Second, the conclusion that c_T^* is far below c_W is robust: no ±25% change in any parameter moves c_T^* above 0.4 d^{-1}.

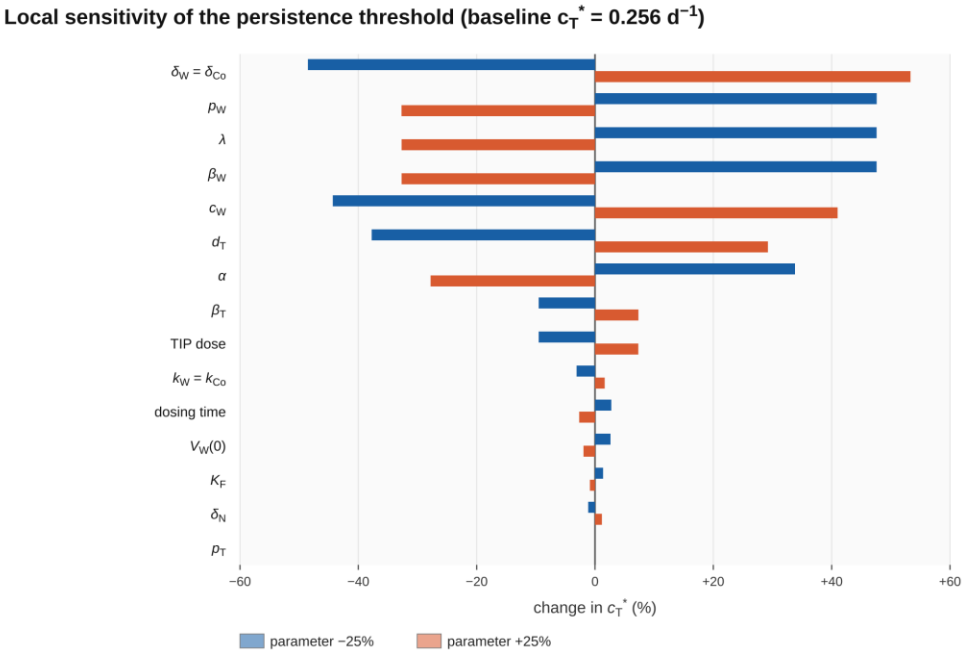


Figure 11. *Local sensitivity of the persistence threshold c_T* (baseline 0.256 d⁻¹) to ±25% changes in each parameter, one at a time. Parameters entering R_0 dominate; TIP production p_T and the cytokine parameters have almost no effect.*

10.6 Safety coupling (Postulate P4)

If the TIP itself stimulates the innate response ($\rho_T > 0$), a larger fraction of regrowth is suppressed and c_T* rises, but cytokine rises faster. Figure 12 shows that raising ρ_T from 0 to 3×10^{-3} increases c_T* only from 0.256 to 0.373 d^{-1} (+46%), while the peak cytokine at the threshold climbs from 5 pg mL^{-1} to 1.7×10^4 pg mL^{-1}. The cytokine safety bound of 500 pg mL^{-1} is crossed at $\rho_T \approx 10^{-4}$, where c_T* has increased by only 25%. Innate stimulation cannot be used to rescue a fast-cleared TIP without breaching the safety constraint, which is Postulate P4 stated quantitatively.

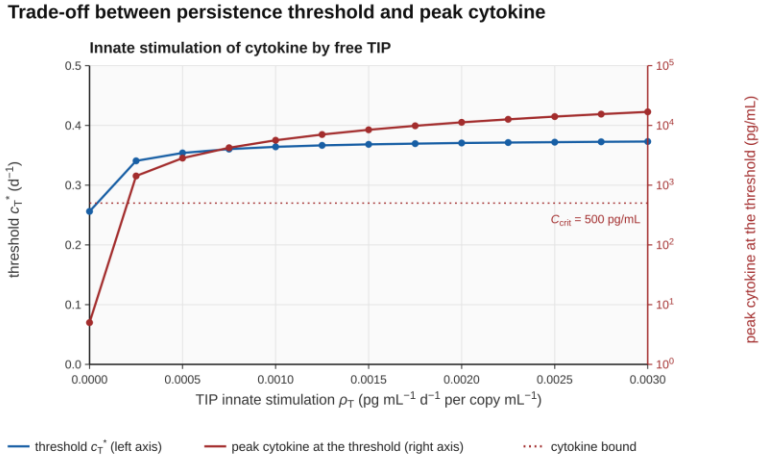


Figure 12. *Trade-off between persistence threshold (blue, left axis) and peak cytokine at the threshold (red, right axis) as TIP innate stimulation ρ_T increases over 13 values from 0 to 3×10^{-3}; c_T^* rises from 0.256 to 0.373 d^{-1} and peak C from 5 to 1.7×10^4. The dotted line is the cytokine bound C_{crit}. The leftmost point is $\rho_T = 0$. Reproduce: make_figures_analysis.py fig12; data in figure_data/.*

11 Adaptive Immunity and Vaccination

Section 10 describes a host with no adaptive response. Two things change that picture. First, a naive host mounts its own response during infection, with antibody appearing after one to two weeks. Second, a vaccinated host begins with antibody and memory cells already in place. Vaccination and TIP therapy are not competitors in this model, because the antibody acts on the same two quantities (entry and clearance) that the TIP interferes with, and because antibody may also act on the TIP. This section extends the engine with three states and examines both questions. The implementation is rpt_adapt.cpp (Appendix D).

11.1 Formulation

Antibody concentration A (μg mL⁻¹) acts through a saturating effect a(A) with half-effect concentration K_A. It adds to the constant entry-blocking and clearance effects of Equations 1–7 in the form of independent probabilities:

$$a(A) = \frac{A}{K_A + A} \tag{17}$$

In words: a(A) rises from 0 to 1 as antibody increases; half of the maximal effect is reached at A = K_A.

$$\varepsilon_{\mathrm{Ab}}^{\mathrm{tot}} = 1 - (1 - \varepsilon_{\mathrm{Ab}})(1 - \varepsilon_{\max}\, a(A)), \qquad \varepsilon_{\mathrm{Clr}}^{\mathrm{tot}} = \varepsilon_{\mathrm{Clr}} + (1 - \varepsilon_{\mathrm{Clr}}) a(A) \tag{18}$$

In words: The total entry-blocking effect combines the constant effect ε_{Ab} with the antibody effect (maximum ε_{max}); the total clearance effect combines ε_{Clr} with a(A). These replace ε_{Ab} and ε_{Clr} in Equations 1–7 and 13.

Three further states complete the extension: an antibody-secreting memory pool B that expands in the presence of free virus and relaxes to its naive level, antibody A produced by B and cleared at rate d_A, and a cytotoxic pool Z that expands with the infected-cell load.

$$\frac{dB}{dt} = \pi_B\, B\, \frac{V_W}{K_B + V_W}\left(1 - \frac{B}{B_{\max}}\right) - \mu_{\mathrm{decay}}(B - B_{\mathrm{naive}}) \tag{19}$$

In words: B grows logistically (cap B_{max}) with a saturating dependence on free WT antigen V_W (half-saturation K_B), and relaxes to B_{naive} at rate μ_{decay} when antigen is absent.

$$\frac{dA}{dt} = p_A\, B - d_A\, A \tag{20}$$

In words: Antibody is produced in proportion to the antibody-secreting pool and cleared at rate d_A. Antibody consumed by binding virus is neglected: at 10^7 copies mL⁻¹ and about 10^3 antibody molecules per virion, the bound antibody is about 2.5×10^{-3} μg mL⁻¹, under 0.01% of 50 μg mL⁻¹.

$$\frac{dZ}{dt} = \pi_Z\, Z\, \frac{I_W + I_{\mathrm{Co}}}{K_Z + I_W + I_{\mathrm{Co}}}\left(1 - \frac{Z}{Z_{\max}}\right) - \mu_{Z,\,\mathrm{decay}}(Z - Z_{\mathrm{naive}}) \tag{21}$$

In words: The cytotoxic pool Z expands with the load of infected cells $I_W + I_{Co}$ (half-saturation K_Z) under a logistic cap, and relaxes to Z_{naive}.

$$\delta_W \;\rightarrow\; \delta_W + \mu_Z Z, \qquad \delta_{\mathrm{Co}} \;\rightarrow\; \delta_{\mathrm{Co}} + \mu_Z Z \qquad (\text{Eqs 3 and 5}) \tag{22}$$

In words: CTL killing adds the term $\mu_Z Z$ to the infected-cell death rate in Equations 3 and 5.

$$\frac{dV_T}{dt} = p_T\, I_{\mathrm{Co}} - c_T\left(1 + \xi_T\, \kappa\, \varepsilon_{\mathrm{Clr}}^{\mathrm{tot}}\right) V_T \qquad (\text{replaces Eq 7}) \tag{23}$$

In words: The TIP is cleared faster when antibody recognises it. $\xi_T \in [0, 1]$ is the degree of antibody cross-recognition of the TIP (0: invisible to the vaccine antibody; 1: recognised like WT). A worst-case additive form, in which antibody clearance of the TIP adds $\kappa\, \xi_T\, \varepsilon_{Clr}\, c_W$ to c_T, is also examined.

$$A(0) = A_{\mathrm{vac}}, \qquad B(0) = \max\left(B_{\mathrm{naive}},\ \frac{d_A}{p_A} A_{\mathrm{vac}}\right) \tag{24}$$

In words: Vaccination is represented only by initial conditions. A host with antibody level A_{vac} starts with a memory pool B(0) that exactly sustains that level at equilibrium ($B(0) = d_A\, A_{vac} / p_A$), and CTL memory scaled alike. A naive host has A(0) = 0, $B(0) = B_{naive}$, $Z(0) = Z_{naive}$.

Table 15. Parameters of the adaptive extension (illustrative; to be calibrated)

Symbol	Code name	Meaning	Default	Units
K_A	K_A	Antibody half-effect concentration	10	µg mL^{-1}
ε_{max}	eps_max	Maximal antibody entry-blocking fraction	0.9	dimensionless
ξ_T	xi_T	Antibody cross-recognition of the TIP	1 (varied)	dimensionless
π_B, K_B, B_{max}	pi_B, K_B, B_max	B-pool expansion rate, half-saturation, cap	0.5, 10^3, 200	d^{-1}, copies mL^{-1}, cells mL^{-1}
μ_{decay}, B_{naive}	mu_decay, B_naive	Relaxation rate and naive level of B	0.005, 1	d^{-1}, cells mL^{-1}
p_A, d_A	p_A, d_A	Antibody production per B cell, antibody clearance	0.0165, 0.033	µg cell^{-1} d^{-1}, d^{-1}
π_Z, K_Z, Z_{max}	pi_Z, K_Z, Z_max	CTL expansion rate, half-saturation, cap	0.4, 10^3, 10^4	d^{-1}, cells mL^{-1}, cells mL^{-1}
μ_Z	mu_Z	CTL killing coefficient	10^{-5}	mL cell^{-1} d^{-1}
A_{vac}	A0	Initial antibody (vaccinated host)	0 or 50	µg mL^{-1}

The antibody half-life is ln 2/d_A = 21 days. p_A is chosen so that B = 100 cells mL^{-1} sustains A = 50 µg mL^{-1} ($p_A = d_A \times 50/100$); it is an effective constant, not a measured secretion rate. All values are placeholders pending calibration to published antibody kinetics and post-vaccination titres.

11.2 Antibody as a rate modifier: how the effect is scaled

The scale of the antibody effect determines every result in this section, so it is worth stating how it is set. Entry blocking and clearance enhancement act on rates of order 10 d^{-1} (c_W) and on R_0 = 8.73. A neutralisation rate that is a mass-action product γ_A A V with a small coefficient (for example 10^{-4} per µg mL^{-1} per day) gives only 5×10^{-3} d^{-1} at 50 µg mL^{-1}, under 0.1% of c_W, and could not change any outcome. The saturating form of Equation 17 instead lets A = 50 µg mL^{-1} block 75% of entry and nearly double clearance, giving R_{eff} = 1.19 for the vaccinated host (Equation 13), a vaccine that is strongly but not completely protective. The two constants K_A and ε_{max} are the quantities that experiments on post-vaccination sera must supply.

11.3 Naive and vaccinated hosts: time courses

Figure 13 compares four situations: naive and vaccinated (A_{vac} = 50 µg mL^{-1}) hosts, each without treatment and with the TIP profile at c_T = 0.5 d^{-1}. The naive untreated host peaks at 1.3×10^7 copies mL^{-1} with a cytokine peak of 206 pg mL^{-1}, and its own antibody appears from day 10, reaching 45 µg mL^{-1} by day 30: the adaptive response arrives too late to prevent the peak. The vaccinated untreated host never develops a peak (virus peaks at 3.5×10^3, a 3.6 log10 reduction) and cytokine stays at 5.1 pg mL^{-1}, but it does not clear the virus within 30 days because R_{eff} remains above 1. The TIP at c_T = 0.5 d^{-1} fails in the naive host (peak 3.2×10^4, no extinction) since this clearance is above the naive threshold of 0.26 d^{-1}; in the vaccinated host the same TIP clears WT virus at day 5.3. The combination achieves what neither does alone.

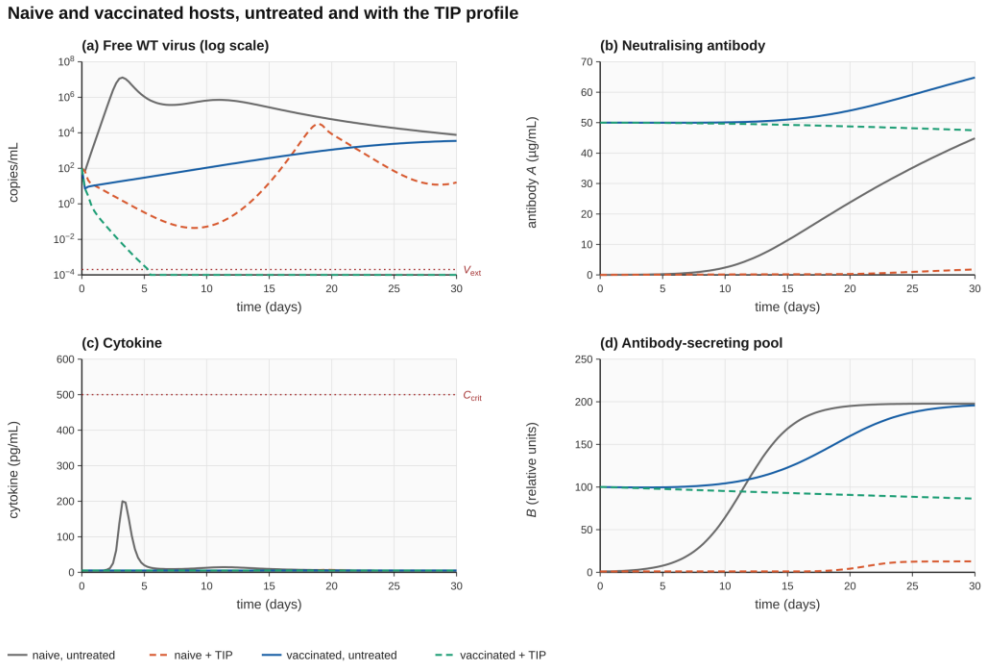


Figure 13. *Naive (grey, orange) and vaccinated (blue, green) hosts, untreated (solid) and with the TIP profile at c_T = 0.5 d⁻¹ (dashed). (a) Free WT virus, with V_{ext} dotted; (b) neutralising antibody; (c) cytokine, with the safety bound dotted; (d) antibody-secreting pool B. Only the vaccinated host with TIP reaches extinction.*

The host's own adaptive response changes the persistence threshold only slightly: $c_T^* = 0.261$ d⁻¹ with the adaptive states active in a naive host, against 0.256 d⁻¹ without. The reason is timing. The threshold is decided in the first week, while antibody in the naive host is still of the order of 1 μg mL⁻¹ or less.

11.4 When the vaccine alone is enough

Equation 13 applied to the antibody-modified effects gives the effective reproduction number as a function of antibody level, and its zero crossing gives the antibody level at which a vaccinated host can clear the virus without any TIP:

$$a_{\mathrm{crit}} = \frac{R_0 - 1}{\kappa + \varepsilon_{\max} R_0}, \qquad A_{\mathrm{crit}} = \frac{K_A\, a_{\mathrm{crit}}}{1 - a_{\mathrm{crit}}} \tag{25}$$

In words: Setting $R_{eff}(A) = 1$ in Equation 13 with Equation 17 gives the critical antibody effect a_{crit} and hence A_{crit}. For the parameters of Table 15, $a_{crit} = 0.873$ and $A_{crit} = 68.6$ μg mL⁻¹.

The simulated time to extinction without a TIP shows that A_{crit} is necessary but not sufficient: within a 60-day window extinction requires 94 μg mL⁻¹ (Figure 14b), because just above $R_{eff} = 1$ the decay is so slow that 13 e-folds (from 100 to 2×10^{-4} copies mL⁻¹) cannot be completed. The relevant range for a combination therapy is therefore A between about 5 and 90 μg mL⁻¹, where the vaccine reduces R_{eff} but does not clear the infection and the TIP has something to add.

11.5 Persistence under vaccination: the antibody cross-recognition problem

The central question for combination therapy is how the persistence threshold c_T^* moves with vaccine titre. Figure 14a shows the answer for four assumptions about antibody cross-recognition of the TIP. In each case antibody lowers the WT burden that the TIP must clear, but it may also clear the TIP.

- **TIP invisible to antibody ($\xi_T = 0$).** c_T^* rises steeply with titre: 0.86 d⁻¹ at 12.5 μg mL⁻¹, 1.96 at 25, 4.8 at 50 and 9.7 at 75 μg mL⁻¹, entering the WT clearance range. At $A_{vac} = 20$ μg mL⁻¹ the threshold has already reached 1.2 d⁻¹, which is a free-TIP half-life of only 0.58 days.
- **Fully recognised, multiplicative clearance ($\xi_T = 1$).** The benefit is reduced but remains large: 0.55 d⁻¹ at 12.5 μg mL⁻¹, 1.14 at 25, 2.6 at 50 μg mL⁻¹. c_T^* reaches 1.2 d⁻¹ at $A_{vac} = 27.5$ μg mL⁻¹ ($\xi_T = 0.5$: 22.5).

- **Fully recognised, additive clearance (worst case).** Antibody-coated TIP is cleared at the antibody clearance rate regardless of its baseline stability. Extinction then fails at every titre between 2.5 and 70 µg mL⁻¹ even for c_T as low as 0.003 d⁻¹; only an unvaccinated host (c_T^* = 0.17 d⁻¹) and titres of 72 µg mL⁻¹ and above, close to the vaccine-alone limit, succeed. The TIP is useless in a vaccinated host in this case.

The finding is a design requirement. A TIP intended for use alongside, or after, vaccination or antibody therapy must be antigenically distinct from the vaccine antigen, or antibody-mediated clearance of the TIP must be shown to be slow. Whether antibody clears a TIP additively (as immune complexes) or only multiplicatively is an experimental question that can be answered with post-vaccination sera and an engineered particle.

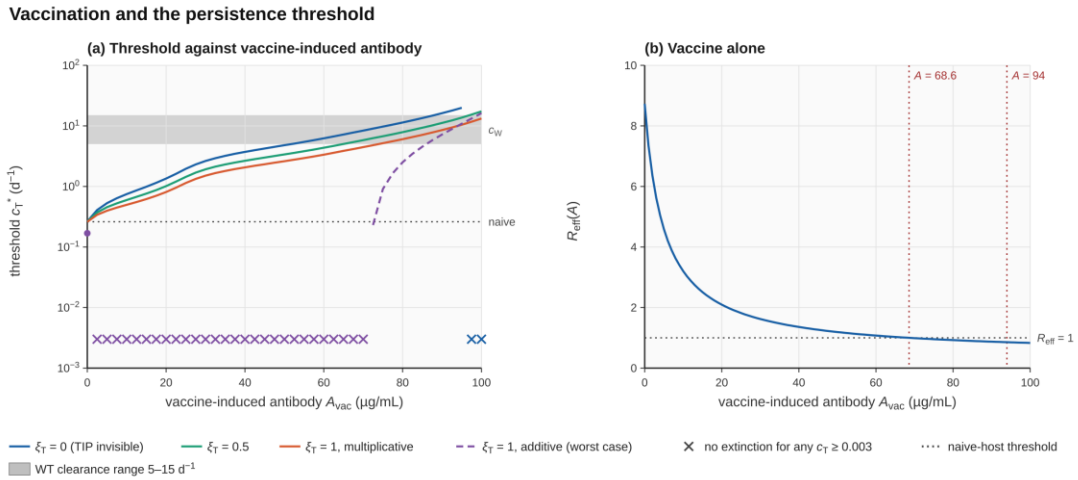


Figure 14. *Vaccination and the persistence threshold. (a) c_T^* against vaccine-induced antibody for four assumptions on antibody recognition of the TIP (ξ = 0, 0.5, 1 multiplicative and 1 additive); the grey band is the WT clearance range and the dotted line the naive-host threshold. (b) $R_{eff}(A)$ for vaccine alone (Eq 13); dotted lines: R_{eff} = 1 at A = 68.6 µg mL⁻¹ and 60-day extinction at 94.0 µg mL⁻¹. Reproduce: make_figures_analysis.py fig14; data in figure_data/.*

11.6 Peak load and cytokine across titres

Figure 15 gives peak virus and peak cytokine as vaccine-induced antibody increases. Without TIP, peak viraemia falls steadily from 1.3×10⁷ copies mL⁻¹ (A = 0) to about 2×10² (A = 60) and peak cytokine from 206 to 5 pg mL⁻¹: vaccination alone removes cytokine-storm risk well before it removes infection. A TIP with c_T = 0.5 d⁻¹ has no effect on extinction below A ≈ 12 µg mL⁻¹ but clears the virus (peak equal to the inoculum) from that titre upward; a TIP with c_T = 0.1 d⁻¹ clears it at every titre. Cytokine never approaches the bound in any treated arm.

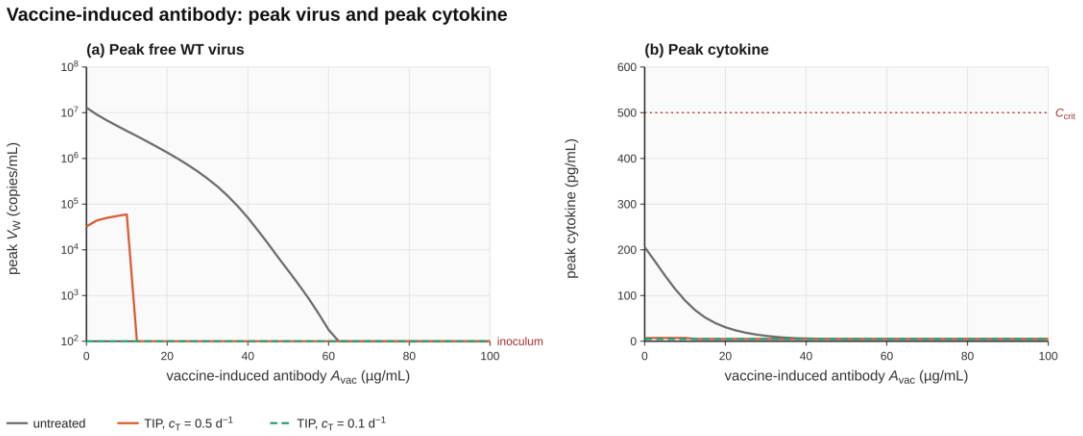


Figure 15. *(a) Peak free WT virus and (b) peak cytokine against vaccine-induced antibody A_{vac} for untreated hosts and hosts given the TIP profile at c_T = 0.5 and 0.1 d^{-1}. A peak equal to the inoculum (100 copies mL^{-1}) means no growth at all.*

11.7 Hypotheses and falsifiers added by the extension

- **P5 (vaccination coupling).** Antibody lowers R_{eff} and so raises the free-TIP clearance that still permits extinction; the effect is monotone in titre. The extension shows c_T* rising from 0.26 to about 5 d^{-1} (ξ_T = 1) or 10 d^{-1} (ξ_T = 0) between A = 0 and 75 µg mL^{-1}.
- **P6 (antigenic distinctness).** Whether antibody helps or hinders the TIP is controlled by ξ_T and by the form of antibody clearance of the TIP (multiplicative or additive). A TIP that is recognised and cleared additively is incompatible with vaccination.
- **F5.** Post-vaccination sera clear an engineered particle at rates of the order of c_W (several per day) at titres where the vaccine also reduces WT infectivity. Then the additive worst case applies and P6 limits the combination.
- **F6.** Measured antibody levels in vaccinated hosts exceed A_{crit} (Equation 25, about 69 µg mL^{-1} with these constants). Then vaccine-alone protection is deterministic in the model and the TIP adds nothing in that population.

12 Stochastic Boundaries of Extinction

All previous results are deterministic. Near extinction the populations are a handful of virions and cells per host, and demographic noise decides whether the infection dies out or regrows. Two analyses address this. The first is exact for the start of infection; the second tests the clearance threshold itself.

12.1 Establishment probability of an infection

Consider a single WT virion in a host with constant target-cell supply, a burst-producing infected cell and no TIP. The virion infects at rate $\beta_W T^*(1 - \varepsilon_{Ab})$ or is cleared at rate c_{eff}; an infected cell releases virions at rate p_W until it dies at rate δ_{eff}. The numbers of cells infected by a virion and of virions released by a cell are geometric, and the extinction probability q of the whole lineage solves a quadratic whose non-trivial root is:

$$q = \min\left\{1,\ \frac{p_W + \delta_{\text{eff}}}{p_W + R_{\text{eff}}\,\delta_{\text{eff}}}\right\}, \qquad P_{\text{est}}(N_0) = 1 - q^{N_0} \tag{26}$$

In words: q is the probability that one virion never starts a self-sustaining infection. The establishment probability for an inoculum of N_0 virions is 1 − q^N0 (assuming independent lineages). For $R_{eff} \le 1$, q = 1.

The result is counter-intuitive. For the reference parameters (p_W = 500, δ_{eff} = 2.05 d^{-1}, R_0 = 8.73), q = 0.970, so a single virion establishes an infection with probability only 3.0%, not 1 − 1/R_0 = 89%: a virion rarely infects a cell before it is cleared, and a cell that is infected produces a burst of about 250 virions. This is the established behaviour of burst-producing infections. Monte Carlo simulation of the virion–infected-cell Markov chain confirms the formula in every case tested (Table 16, Figure 16a): the largest deviation is 0.8 standard errors over eight parameter sets spanning R_{eff} = 1.2–20.

Table 16. Establishment probability from one virion: theory against Monte Carlo

p_W	δ	c_W	R_{eff}	1 − q (Eq 26)	Monte Carlo	Lineages	±SE
500	2	10	8.73	0.0299	0.0289	12 000	0.0015
500	2	10	20	0.0704	0.0723	6 000	0.0033
50	2	10	8.73	0.2292	0.2297	6 000	0.0054
50	2	10	4	0.1034	0.1067	6 000	0.0040
50	2	10	2	0.0370	0.0373	12 000	0.0017
20	2	10	2	0.0833	0.0846	12 000	0.0025
20	2	10	1.5	0.0435	0.0429	12 000	0.0019
20	2	10	1.2	0.0179	0.0179	20 000	0.0009

For exposure biology the consequence is a steep dependence on inoculum size (Figure 16b). A naive host exposed to ten virions has a 26% chance of infection and to a hundred virions 95%, whereas a vaccinated host with R_{eff} = 1.19 (A = 50 µg mL^{-1}) has a 0.8% chance at ten virions, 7% at a hundred and 54% at a thousand. A host with R_{eff} below 1 cannot establish infection at any inoculum. These probabilities apply to the first hours of infection and to post-exposure prophylaxis with a TIP; they are a property of the unmodified branching model and exclude any TIP.

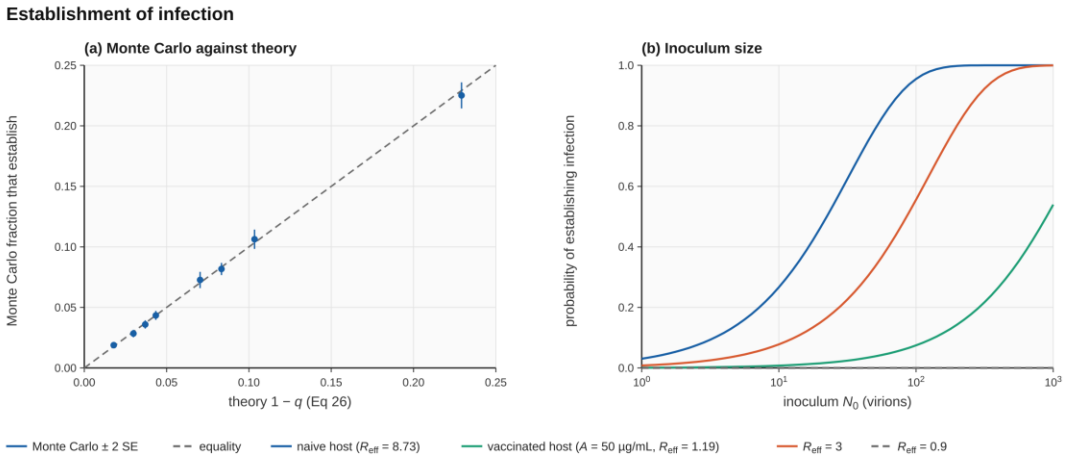


Figure 16. *Establishment of infection. (a) Monte Carlo fraction of lineages that establish (±2 standard errors) against the prediction 1 − q of Eq 26 for the eight parameter sets of Table 16; the dashed line is equality. (b) Probability of establishing infection against inoculum size for a naive host (R_{eff} = 8.73; P = 0.27 for 10 virions), a vaccinated host (A = 50 µg mL^{-1}, R_{eff} = 1.19; P = 0.008), R_{eff} = 3 and R_{eff} = 0.9. Reproduce: make_figures_analysis.py fig16; data in figure_data/.*

12.2 Does the persistence threshold survive demographic noise?

Falsifier F4 asks whether the deterministic extinction used so far, which relies on a floor at one copy per host, survives noise. A hybrid simulation tests this. The deterministic ODE supplies the slowly changing environment (target cells, primed cells, free TIP, cytokine); when the total WT burden first falls below about 300 copies per host (blood volume 5 L), the WT subsystem (E_W, I_W, E_{Co}, I_{Co}, V_W) is switched to exact Gillespie simulation with ten event types and the environment held piecewise constant over steps of 0.01 d. A replicate counts as an extinction when all five counts reach zero, and as an escape when the total exceeds 1500. Two hundred replicates were run for each of eleven values of c_T (the algorithm is listed in Appendix E).

The deterministic threshold turns out to be conservative (Figure 17, Table 17). For $c_T \leq 0.25$ d^{-1} the extinction probability is at least 0.99, essentially identical to the deterministic outcome. Above the deterministic threshold the outcome is not a sharp switch: at 0.30 d^{-1} the extinction probability is still 0.94, at 0.35 it is 0.82, at 0.40 it is 0.53, and it falls to 0.05 at 0.5 d^{-1} and zero at 0.7 d^{-1} and above, where the trough never gets low enough for noise to help (it already holds about 450 copies per host at 0.5 d^{-1} and more than 2000 at 0.7 d^{-1}). The 50% point of the stochastic transition is therefore near 0.4 d^{-1}, about 1.6 times the deterministic c_T*.

Falsifier F4 is therefore not triggered in the direction that would damage the theory: noise does not destroy extinction below c_T*. It does show that the deterministic threshold is a lower bound on the clearance that can be tolerated, and that a half-life of about 1.7 days (c_T = 0.4 d^{-1}) gives roughly even odds in the low-count regime rather than certain failure. The persistence requirement itself is unchanged: both thresholds are an order of magnitude below c_W.

Table 17. Stochastic tail simulation: extinction probability against c_T (200 replicates each)

c_T (d^{-1})	Extinct (of 200)	P(extinct)	Deterministic result	Tail start (d, copies)
0.10	200	1.00	extinct	7.3, 299
0.20	200	1.00	extinct	7.3, 298
0.25	199	1.00	extinct	7.3, 298
0.26	198	0.99	regrows	7.3, 299
0.28	193	0.97	regrows	7.3, 298
0.30	188	0.94	regrows	7.3, 297
0.35	163	0.82	regrows	7.4, 299
0.40	105	0.53	regrows	7.5, 299
0.50	10	0.05	regrows	7.5, 448
0.70	0	0.00	regrows	5.4, 2325
1.00	0	0.00	regrows	3.9, 8000

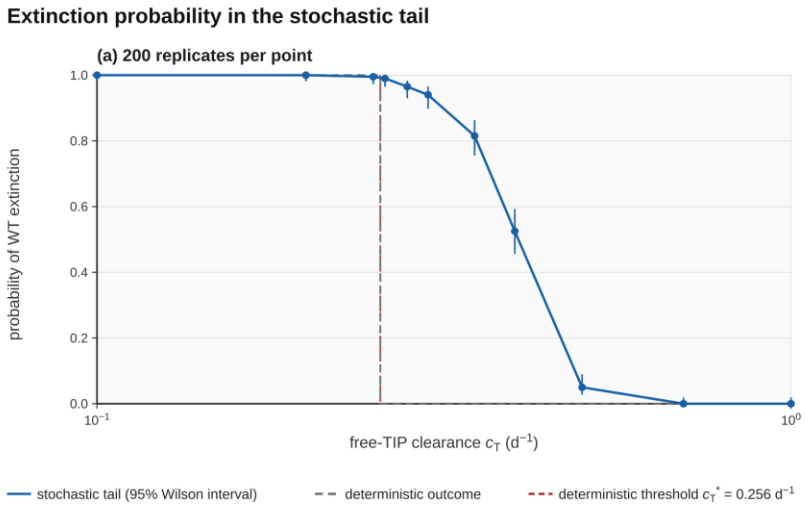


Figure 17. *Probability of WT extinction in the stochastic tail against free-TIP clearance, at 11 values of c_T, with 95% Wilson confidence intervals (200 replicates per point, 300 starting virions), and the deterministic outcome (dashed). The deterministic threshold c_T^* = 0.256 d^{-1} is marked. Reproduce: make_figures_analysis.py fig17; data in figure_data/.*

Limitations of the tail model: the environment is taken from the deterministic solution and does not respond to the stochastic WT subsystem, the 300-copy starting point is a modelling choice, and the 0.01 d environment step is an approximation. The analysis uses Python (Appendix E); a C++ port is planned so that the zero-dependency commitment extends to it.

13 Extended Specification: Pan-Filovirus Scope, Tropism and Compartments

The theory in Sections 4–12 is stated for one virus, *Orthoebolavirus zairense*, in one well-mixed compartment. Two natural questions are whether one TIP concept could cover several filoviruses, and whether tissue structure changes the persistence requirement. This section specifies, in equations, how the model would be extended. **None of it is implemented in the engine yet**; Table 19 records the status. The specification is given so that a later researcher can implement it without having to re-derive the structure.

13.1 Species and naming

Candidate species follow ICTV binomial naming (check the current release): *Orthoebolavirus zairense* (Ebola virus, EBOV), *Orthoebolavirus sudanense* (Sudan virus, SUDV), *Orthoebolavirus bundibugyoense* (Bundibugyo virus, BDBV), *Orthoebolavirus taiense* (Taï Forest virus, TAFV), *Orthoebolavirus restonense* (Reston virus, RESTV) and *Orthomarburgvirus marburgense* (Marburg virus, MARV, and Ravn virus, RAVV). A species index s runs over these, and a TIP concept that covers several of them would have to share the packaging signals and the replication machinery of all.

13.2 Equations

Antibody cross-reactivity. Antibody specificities m neutralise species s with a cross-reactivity matrix χ ($\chi_{s,s} = 1$). The effective antibody against species s and the resulting entry blocking are:

$$A_s^{\mathrm{eff}} = \sum_m \chi_{s,m} A_m, \qquad \varepsilon_{\mathrm{Ab},s} = \varepsilon_{\max} \frac{A_s^{\mathrm{eff}}}{K_A + A_s^{\mathrm{eff}}} \tag{27}$$

In words: A species is protected by antibody raised against a relative only to the extent χ allows; $\varepsilon_{\mathrm{Ab},s}$ replaces the single entry-blocking effect of Equation 17 for species s.

Tropism. Two target-cell classes X (macrophage and monocyte lineage, M, and hepatocyte, H) have their own supply, death and entry rates. The force of infection Λ_X sums over species and the TIP, each with its own antibody effect:

$$\frac{dT_X}{dt} = \lambda_X - d_X T_X - \Lambda_X T_X, \qquad X \in \{M, H\} \tag{28}$$

In words: Target-cell dynamics for class X: supply λ_X, death d_X, and loss by infection at rate Λ_X.

$$\Lambda_X = \sum_s (1 - \varepsilon_{\mathrm{Ab},s}) \beta_{s,X} V_{W,s} + (1 - \varepsilon_{\mathrm{Ab},T}) \beta_{T,X} V_T \tag{29}$$

In words: Force of infection on class X from all WT species s and from the TIP, each reduced by the corresponding antibody effect. This replaces the single factor $(1 - \varepsilon_{\mathrm{Ab}})$ of Equation 1, which assumed one antibody effect for all virus.

$$\frac{dV_{W,s}}{dt} = \sum_X p_{s,X} (I_{W,s,X} + \alpha_s I_{\mathrm{Co},s,X}) - c_{W,s}(1 + \kappa \varepsilon_{\mathrm{Clr},s}) V_{W,s} - \psi_V \left(V_{W,s} - V_{W,s}^{\mathrm{org}} \right) \tag{30}$$

In words: Free virus of species s is produced by infected cells of every class; co-infected cells produce it at the hijacked fraction α_s. Clearance carries the species-specific antibody effect, and the last term exchanges virus with an organ pool (superscript org).

Compartment exchange. Exchange of any quantity X between a blood compartment b of volume $\mathcal{V}_b$ and an organ compartment o of volume $\mathcal{V}_o$ must conserve mass, so the organ equation carries the volume ratio:

$$\frac{dX_b}{dt} = \cdots - \psi_X (X_b - X_o), \qquad \frac{dX_o}{dt} = \cdots + \psi_X \frac{\mathcal{V}_b}{\mathcal{V}_o} (X_b - X_o) \tag{31}$$

In words: The dots stand for local source and loss terms. Without the factor $\mathcal{V}_b/\mathcal{V}_o$, exchange would create or destroy material whenever the two volumes differ.

Reproduction number with tropism. For species s with infection-free target-cell levels T_X^*, the reproduction number sums over cell classes:

$$R_{0,s} = \frac{1}{c_{W,s}} \sum_X \frac{p_{s,X}\,\beta_{s,X}\,T_X^*}{\delta_{s,X}} \tag{32}$$

In words: Each cell class X contributes the product of its infection rate, its production per cell and its lifetime $1/\delta_{s,X}$, scaled by the species clearance. A non-trivial next-generation matrix is needed when several species co-circulate.

Antigen-specific memory. Each specificity has its own B-cell pool, driven by the matching free antigen only (every term has the same units):

$$\frac{dB_m}{dt} = \pi_B\,B_m\,\frac{V_{W,m}}{K_B + V_{W,m}}\left(1 - \frac{B_m}{B_{\max}}\right) - \mu_{\text{decay}}(B_m - B_{\text{naive}}) \tag{33}$$

In words: This is Equation 19 applied to specificity m, with $V_{W,m}$ the free virus of the matching species. Infected cells do not appear here because they have different units; they drive the CTL pool only.

13.3 Formulation checks

Each equation of the extensions was checked for dimensional consistency, conservation, scale and the invariants that the simulator tests. Table 18 lists the checks that changed the form of an equation. They are recorded because each is an easy mistake that would have produced numbers without meaning.

Table 18. Consistency checks and the form adopted

Quantity	Potential problem	Form adopted
Antigen drive of B cells	Free virus (copies mL^{-1}) and infected cells (cells mL^{-1}) cannot share one saturation constant	B driven by free virus only (Eq 19, 33); Z driven by infected cells only (Eq 23)
Decay of B	Separate loss and relaxation terms double-count turnover and shift the equilibrium	One relaxation term toward B_{naive}; the equilibrium is B_{naive} without antigen
Antibody effect scale	A mass-action coefficient of order 10^{-4} per μg mL^{-1} per day gives 5×10^{-3} d^{-1} at 50 μg mL^{-1}, against c_W = 10 d^{-1}: no effect	Saturating a(A) feeding ε_{Ab} and ε_{Clr} of the core (Eq 17, 18); R_{eff} = 1.19 at A = 50
Antibody production scale	p_A = 10 μg $cell^{-1}$ d^{-1} with B = 100 gives A* = 3×10^4 μg mL^{-1}, not 50	$p_A = d_A\,A_{vac} / B_{mem}$ (Eq 24), so the initial state is an equilibrium
Organ exchange	Exchange without a volume ratio does not conserve mass	Eq 31 with $\mathcal{V}_b/\mathcal{V}_o$
TIP and antibody	A TIP sharing the envelope is neutralised too; omitting this overstates synergy	Explicit ξ_T and a worst-case additive form (Eq 23)
Force of infection	One ε for all species	Species- and TIP-specific ε_{Ab} (Eq 28, 29)
Establishment	Taking 1 − 1/R as the probability that an infection starts	Eq 26; Monte Carlo confirms (Table 16)
Organ virus pool	One shared organ pool for all species and the TIP	One organ pool per virus species and for the TIP

13.4 Implementation status

Table 19. What exists and what is specified

Component	Status	Evidence or next step
Ten-state core, RK4, stability guard, screening	**Implemented** (code.cpp)	Sections 6–7; Appendix C
Adaptive immunity and vaccination (B, A, Z)	**Implemented** (rpt_adapt.cpp)	Section 11; Appendix D; cross-validated against the core
Establishment probability (branching process)	**Derived and validated** (Python)	Section 12.1; C++ port planned
Tail-stochastic extinction check	**Implemented** (Python)	Section 12.2; C++ port planned
Cross-reactive antibody matrix χ; several species	Specified	Eq 27, 32; needs a species-resolved state layout
Tropism (macrophage and hepatocyte classes)	Specified	Eq 28–29; parameters to be sourced
Blood and organ compartments	Specified	Eq 30–31; volumes and exchange rates to be sourced
Calibration to published kinetics and titres	Not done	Phase 1; the adaptive constants of Table 15 are placeholders

A single TIP covering several species is a hypothesis, not a result. The persistence requirement c_T^* was derived for *Orthoebolavirus zairense* only; whether it holds for other species depends on their R_0 and on whether a common TIP can be designed. Section 10.5 shows that c_T^* is controlled mainly by R_0 and by the infected-cell lifetime, so species with larger R_0 would be expected to need an even longer-lived TIP.

14 Design Requirements, Measurement Targets and Roadmap

The purpose of the model is to say what must be measured and how well. Table 20 turns the computational results into measurement targets for experimental groups. It is the experimental shopping list of this work: each row names a quantity, the value the model needs and the postulate or falsifier it decides (Table 3a).

Table 20. Measurement targets derived from the model (all conditional on illustrative parameters)

Quantity	Target from the model	Why it matters
Free-TIP in vivo half-life	At least 1.5 to 2.7 days ($c_T \leq 0.46$ to 0.26 d^{-1}) for a naive host; at least 0.6 days ($c_T \leq 1.2$ d^{-1}) if $A_{vac} \geq 20$ to 28 μg mL^{-1}	Decides Postulate P3; the strictest requirement of the theory
Hijacking factor α	At most about 0.07 (suppression to 7% or less of normal WT output)	Above about 0.08 no extinction is possible in the reference model (Figure 10a)
R_0 of the infection	Calibrated to published kinetics	c_T^* changes by ±33% to 48% for ±25% in R_0 (Figure 11)
Antibody recognition of the TIP (ξ_T) and its clearance form	Measure with post-vaccination sera: ξ_T near 0 preferred; additive clearance disqualifying	Decides Postulates P5 and P6; synergy or antagonism with vaccination (Figure 14)
Antibody constants K_A and ε_{max}	Dose–response of neutralisation and clearance in sera	Sets A_{crit} and every vaccine-related threshold
Primed-cell protection	Is a TIP-only cell protected from later WT entry?	Decides Postulate P2 and falsifier F2
Innate stimulation by TIP (ρ_T)	Cytokine rise per unit of TIP	Must stay below about 10^{-4} before the cytokine bound is breached (Figure 12)

14.1 Open questions

1. What is the effective in vivo half-life of an engineered filovirus-derived TIP-type particle? (Decides P3.)
2. Are cells entered by a TIP, but not yet infected by WT virus, protected by interferon or other innate mechanisms? (Decides P2.)
3. Do antibodies raised by vaccination recognise and clear an engineered TIP, and is that clearance additive or multiplicative? (Decides P6.)
4. What produces the resonance of the persistence threshold near a dose of 10^6 copies mL^{-1} (Figure 10d)?
5. Does the persistence requirement hold for other filovirus species and in a two-tropism, two-compartment host?
6. How should the extinction probability in the low-count regime be combined with the clearance requirement into a single design criterion?

14.2 Roadmap

Table 21. Computational milestones

Step	Task	Exit criterion
M1	Calibrate the core and adaptive constants to published EBOV kinetics and vaccine titres (Phase 1)	Peaks and timings within the stated tolerance; provenance register complete
M2	Port the tail-stochastic simulation and establishment calculation to C++	Identical results to Appendix E; no dependencies
M3	Add tropism and organ compartments (Eq 28–31)	Conservation checks pass; c_T^* recomputed with sanctuary compartments
M4	Add the cross-reactivity matrix and several species (Eq 27, 32)	Species-resolved thresholds and a common-TIP feasibility map

Step	Task	Exit criterion
M5	Provide experimental groups with a ranked, uncertainty-bounded shortlist (Table 20)	Report plus open-source engine, parameter files and scripts

Aims 1 to 3 of Table 14a map onto these milestones: Aim 1 is M1, Aim 2 is M2 and M3, and Aim 3 is M5. M4 (several species) lies beyond the 24-month scope.

14.3 Pre-registered falsification and stop rule

Before any calibrated run, the items of Table 21a are fixed, tagged in the public repository and registered on OSF Registries (Open Science Framework, osf.io) using the OSF Preregistration template, and the time-stamped registration is cited in every report. The protocol is framed as a decision tool, not a biological claim: it reduces the number of candidate constructs that must enter BSL-4 laboratories by ranking them in silico, and it states in advance what result would make the ranking worthless. Tolerances marked “proposed” are the author’s starting proposal and are to be confirmed before fitting begins.

Table 21a. Pre-registration lock list

Item locked	Value or rule	Status
Calibration data	Published EBOV within-host kinetics in non-human primates and patients (Madelain et al., 2018; Martyushev et al., 2016; Nguyen et al., 2015) and published vaccine antibody titres; the dataset list is fixed at the start	Fixed at start of Aim 1
Tolerance	Simulated peak viral load within 0.5 log10 and time of peak within 2 days of each reference curve; antibody peaks within a factor of 2	Proposed; confirm before fitting
Stop rule	If the calibrated model cannot meet the tolerance within the pre-declared parameter freedom, cT*, the shortlist and the vaccine thresholds are withdrawn and only the structural results are reported. A failure to calibrate is a result	Fixed
Falsifiers	F1 to F6 with the numeric rules of Table 3a	Fixed (F2, F4, F5 proposed)
Extinction definition	Vext = 2×10^{-4} copies mL^{-1} with the extinction floor on; stochastic tail with 200 replicates per cT	Fixed (reference campaign)
Code and data lock	Repository tag of engine, scripts and parameter files; SHA-256 of the screening CSV; seed 12345	Fixed at tag
Analysis plan	Bisection for cT*; ±25% local and Latin hypercube global (PRCC) sensitivity; an uncertainty-bounded ranking of profiles	Fixed

Two statements hold whichever way the calibration goes. A calibration that passes turns the illustrative numbers of this document into the first calibrated ones, and the shortlist becomes a defensible input to experimental design. A calibration that fails ends the quantitative claims at low cost, which is the purpose of stating the stop rule in advance.

15 Synthesis

The theory of persistence-limited interference started as a statement about one number: for the TIP to clear *Orthoebolavirus zairense*, the free particle must survive for days. This document has tested that statement in several ways, and it has held. The numerical core converges at fourth order and reproduces the analytic reproduction-number threshold to seven digits (verify_threshold.py). The persistence threshold is robust to ±25% changes in every biological parameter, controlled chiefly by R_0 and the infected-cell lifetime, and not rescuable by dose, production advantage or innate stimulation. Hijacking strength sets a second, sharp limit. Adaptive immunity relaxes the requirement substantially (from 0.26 to above 1 d^{-1} at moderate titres) provided the TIP is antigenically distinct from the vaccine antigen, and turns it into a failure if antibody clears the TIP additively. Demographic noise leaves extinction intact below the threshold and shows the deterministic threshold to be a conservative bound. The specification of Section 13 indicates how the same structure extends across species and tissues.

What the document does not show is equally clear. No parameter is calibrated; no TIP data exist in vivo; the TIP-versus-antibody interaction is an assumption in two alternative forms; the multi-species and multi-compartment structure is a specification, not a result. The theory is therefore a set of sharp, testable, quantitative requirements that experiments can confirm or refute, and the engine, the parameter files and the scripts are provided so that the tests can be repeated and extended. If the requirement survives measurement, it identifies which engineering target matters most on the way to a therapy or a combination

with vaccination; if it fails, the failure will have been located precisely, and it will have saved experimental effort that would otherwise have gone to the wrong particle.

References

Calain, P., Monroe, M. C. and Nichol, S. T. (1999) 'Ebola virus defective interfering particles and persistent infection', *Virology*, 262(1), pp. 114–128.

Chaturvedi, S., Vasen, G., Pablo, M., Chen, X., Beutler, N., Kumar, A. *et al.* (2021) 'Identification of a therapeutic interfering particle: a single-dose SARS-CoV-2 antiviral intervention with a high barrier to resistance', *Cell*, 184(25), pp. 6022–6036.

Fajgenbaum, D. C. and June, C. H. (2020) 'Cytokine storm', *New England Journal of Medicine*, 383(23), pp. 2255–2273.

Hairer, E., Nørsett, S. P. and Wanner, G. (1993) *Solving Ordinary Differential Equations I: Nonstiff Problems*. 2nd edn. Berlin: Springer.

Heffernan, J. M., Smith, R. J. and Wahl, L. M. (2005) 'Perspectives on the basic reproductive ratio', *Journal of the Royal Society Interface*, 2(4), pp. 281–293.

International Committee on Taxonomy of Viruses (ICTV) (current release) *Virus Taxonomy*. Available at: https://ictv.global (check the current release for any name changes).

Madelain, V., Baize, S., Jacquot, F. *et al.* (2018) 'Ebola viral dynamics in nonhuman primates provides insights into virus immuno-pathogenesis and antiviral strategies', *Nature Communications*, 9, 4013.

Marino, S., Hogue, I. B., Ray, C. J. and Kirschner, D. E. (2008) 'A methodology for performing global uncertainty and sensitivity analysis in systems biology', *Journal of Theoretical Biology*, 254(1), pp. 178–196.

Martyushev, A., Nakaoka, S., Sato, K., Noda, T. and Iwami, S. (2016) 'Modelling Ebola virus dynamics: implications for therapy', *Antiviral Research*, 135, pp. 62–73.

McKay, M. D., Beckman, R. J. and Conover, W. J. (1979) 'A comparison of three methods for selecting values of input variables in the analysis of output from a computer code', *Technometrics*, 21(2), pp. 239–245.

Metzger, V. T., Lloyd-Smith, J. O. and Weinberger, L. S. (2011) 'Autonomous targeting of infectious superspreaders using engineered transmissible therapies', *PLoS Computational Biology*, 7(3), e1002015.

Mulangu, S., Dodd, L. E., Davey, R. T. *et al.* (2019) 'A randomized, controlled trial of Ebola virus disease therapeutics', *New England Journal of Medicine*, 381(24), pp. 2293–2303.

Nelder, J. A. and Mead, R. (1965) 'A simplex method for function minimization', *The Computer Journal*, 7(4), pp. 308–313.

Nguyen, V. K., Binder, S. C., Boianelli, A., Meyer-Hermann, M. and Hernandez-Vargas, E. A. (2015) 'Ebola virus infection modeling and identifiability problems', *Frontiers in Microbiology*, 6, 257.

Notton, T., Sardanyés, J., Weinberger, A. D. and Weinberger, L. S. (2014) 'The case for transmissible antivirals to control population-wide infectious disease', *Trends in Biotechnology*, 32(11), pp. 542–551.

Nowak, M. A. and May, R. M. (2000) *Virus Dynamics: Mathematical Principles of Immunology and Virology*. Oxford: Oxford University Press.

Perelson, A. S. (2002) 'Modelling viral and immune system dynamics', *Nature Reviews Immunology*, 2(1), pp. 28–36.

Republic of Kenya (2019) *The Data Protection Act, No. 24 of 2019*. Nairobi: Government Printer.

Rouzine, I. M. and Weinberger, L. S. (2013) 'Design requirements for interfering particles to maintain coadaptive stability with HIV-1', *Journal of Virology*, 87(4), pp. 2081–2093.

Tanner, E. J., Kirkegaard, K. A. and Weinberger, L. S. (2016) 'Exploiting genetic interference for antiviral therapy', *PLoS Genetics*, 12(5), e1005986.

Tracey, K. J. (2002) 'The inflammatory reflex', *Nature*, 420(6917), pp. 853–859.

van den Driessche, P. and Watmough, J. (2002) 'Reproduction numbers and sub-threshold endemic equilibria for compartmental models of disease transmission', *Mathematical Biosciences*, 180, pp. 29–48.

World Health Organization (2015) *World Health Organization Best Practices for the Naming of New Human Infectious Diseases*. Geneva: WHO (WHO/HSE/FOS/15.1).

World Health Organization (2024) *Ebola disease: fact sheet*. Geneva: WHO. Available at: https://www.who.int (Accessed: November 2026).

Appendix A Glossary

Term	Meaning
Hijacking factor α	Fraction of normal WT output that survives in a co-infected cell (0.05 = strong suppression).
PRCC	Partial rank correlation coefficient: sensitivity of an output to an input with the others held fixed.
Antibody cross-recognition ξ_T	Degree to which vaccine-induced antibody recognises and clears the TIP (0 invisible, 1 recognised like WT).
Establishment probability	Probability that an inoculum of N_0 virions starts a self-sustaining infection, 1 − q^N0 (Eq 26).
Persistence threshold c_T*	Largest free-TIP clearance rate that still allows WT extinction (Eq 15).
Primed cell	A cell entered by a TIP but not (yet) by WT virus; cannot make TIP alone and remains open to WT.
R_0, R_{eff}	Basic and effective reproduction numbers; below 1 an infection cannot sustain itself.
Runge–Kutta (RK4)	A classical method for stepping differential equations forward; error shrinks as the 4th power of the step.
TIP	Therapeutic Interfering Particle: a replication-defective particle that multiplies only inside WT-infected cells and suppresses WT output.
Vagal gain m(H)	The factor by which the autonomic (vagal) reflex dampens cytokine production, depending on pulse rate.

Appendix B Parameter and Screening Files, and the C-API Test

These plain-text files are the exact inputs of the reference campaign. They are listed in full so that every result in Section 7 can be regenerated.

input.txt (single-run and baseline parameters)

```
# input.txt - EBOV-TIP engine parameters (name = value). '#' starts a comment.
# All values are ILLUSTRATIVE placeholders until the Phase-1 provenance register is complete.
# Units: cells/mL, virions/mL, time in days.

# --- cellular scale ---
lambda   = 5e4
d_T      = 0.07
beta_W   = 5e-7
beta_T   = 5e-7
k_W      = 6
k_Co     = 6
delta_W  = 2
delta_Co = 2
p_W      = 500
p_T      = 1500
alpha    = 0.2
c_W      = 10
c_T      = 10
delta_N  = 0.5
K_F      = 50
eps_Ab   = 0
eps_Clr  = 0
kappa    = 1

# --- cytokine and autonomic ---
C_rest   = 5
d_C      = 5
gamma    = 3
K_C      = 50
rho_I    = 0.005
rho_V    = 1e-6
sigma_v  = 0.5
H_0      = 70
H_max    = 130
Delta_H  = 50
K_H      = 100
n_hill   = 2
tau_H    = 0.02

# --- objective and safety bounds ---
w1 = 1
w2 = 1
w3 = 1
w4 = 1e-6
C_crit = 500
H_crit = 130
V_ext  = 2e-4
ext_floor = 1      # 1 = WT set to zero once < 1 virion and < 1 infected cell per host (stochastic-extinction proxy); 0 = pure ODE

# --- run control (set tip_dose = 0 for the untreated baseline) ---
V_W0     = 100
tip_dose = 1e7
tip_time = 2
t_end    = 30
h        = 0.005
dt_out   = 0.25

# --- model extensions (v2) ---
primed = 1    # 1 = TIP-only "primed" cells are tracked and can be superinfected by WT; 0 = old abortive-entry behaviour
rho_T  = 0    # cytokine synthesis stimulated by free TIP (pg/mL/d per copy/mL); 0 = no TIP innate stimulation
```

screen.txt (TIP-profile search space and control)

```
# screen.txt - TIP-profile search space and search control
# Format for a dimension:  name = min max [log]      (any name from input.txt can be swept)
beta_T   = 2.5e-7 2e-6 log
alpha    = 0.05 0.4
p_T      = 500 3000
c_T      = 0.1 15 log
tip_dose = 1e5 1e9 log
tip_time = 0 5

# search control
samples    = 2000     # Latin-hypercube profiles
seed       = 12345    # fixed seed: identical results every run and for any thread count
refine_top = 3        # Nelder-Mead refinement of the best N sampled profiles
nm_iters   = 80
top_k      = 10       # rows in the ranked table
threads    = 0        # 0 = all hardware threads
weight_factor = 2     # weight-sensitivity scaling (x and 1/x)
w_ext      = 5        # smooth penalty on final V_W above V_ext (non-extinction)
prefix     = screen   # output files: screen_report.txt, screen_samples.csv, screen_pareto.svg, screen_best.svg
```

test_api.cpp (proves the C API works as Kotlin/JNI will call it)

```
// test_api.cpp - proves the C API works exactly as Kotlin/JNI will call it (text in, text out, no files).
// Build: g++ -std=c++20 -O2 -pthread -DEBOV_NO_MAIN test_api.cpp code.cpp -o test_api
extern "C" { const char* ebov_version(); char* ebov_run(const char*, int); char* ebov_screen(const char*, const char*, const char*); void ebov_free(char*); }
#include <cstdio>
#include <cstring>
int main() {
  std::printf("%s\n", ebov_version());
  const char* params = "tip_dose = 1e9\ntip_time = 1\nc_T = 0.6\nalpha = 0.05\nbeta_T = 2e-6\nV_ext = 2e-4\next_floor = 1\n";
  char* s = ebov_run(params, 0);   std::printf("--- summary ---\n%s", s); ebov_free(s);
  char* c = ebov_run(params, 1);   std::printf("--- csv: %zu bytes, first line: %.40s\n", std::strlen(c), c); ebov_free(c);
  char* v = ebov_run(params, 2);   std::printf("--- svg: %zu bytes, starts: %.30s\n", std::strlen(v), v); ebov_free(v);
  char* bad = ebov_run("alpha = 7\nnot_a_param = 3\n", 0); std::printf("--- bad input ---\n%s", bad); ebov_free(bad);
```

```
 char* r = ebov_screen("ext_floor = 1\n", "samples = 100\nc_T = 0.5 15 log\ntip_dose = 1e5 1e9 log\ntip_time = 0 5\nbeta_T = 2.5e-7 2e-6 log\n", "/tmp/api_screen");
 std::printf("--- screen report (first 3 lines) ---\n"); int n = 0; for (char* p = r; *p && n < 3; ++p) { std::putchar(*p); if (*p == '\n') ++n; } ebov_free(r);
 return 0;
}
```

Appendix C Annotated Source Code: code.cpp

The complete engine follows (about one thousand lines). The comments are explanatory only; **with the comments removed, the code is unchanged** from the unannotated program, and all outputs are byte-identical. Equation numbers refer to Section 5. Read the header block first: it contains the build commands, the reading order and a list of what the code does not yet do.

```
// ============================================================================
// code.cpp  -  EBOV-TIP within-host engine and TIP-profile screening  (C++20, std library only)
// Reference implementation of the Rein Persistence Theory of Persistence-Limited
// Interference in Orthoebolavirus zairense (Ebola virus, EBOV).      Author: Reinford Nyachio
//
// WHAT THIS FILE IS
//   A deterministic ODE simulator of one infected host: wild-type (WT) Ebola virus plus an
//   optional dose of Therapeutic Interfering Particles (TIPs), a cytokine surrogate and a pulse
//   rate. On top of the simulator sits a screening module that tries thousands of candidate
//   TIP "profiles" and ranks them by efficacy and safety. Equations 1-14 of the proposal are
//   implemented literally; each line of rhs() carries its equation number.
//
// HOW TO BUILD AND RUN (any C++20 compiler; no third-party libraries)
//   g++ -std=c++20 -O2 -pthread -Wall -Wextra code.cpp -o engine
//   ./engine [input.txt] [output.txt] [errors.txt]           one trajectory (+ output.svg)
//   ./engine --screen [screen.txt] [input.txt] [errors.txt]   TIP-profile screening
//   ./engine --selftest [input.txt]                           RK4 convergence-order check
//   g++ -std=c++20 -O2 -pthread -DEBOV_NO_MAIN test_api.cpp code.cpp -o test_api   (C-API test)
//
// STATE VECTOR: 10 dynamic states (T, EW, IW, VW, EC, IC, VT, C, H, P) followed by 4 running
//   integrals (JV, JC, JH, U) that accumulate the terms of the objective J (Eq 14), so the
//   objective weights can be re-applied after a run without re-simulating.
//
// UNITS: cells/mL, virions (copies)/mL, time in DAYS, cytokine in pg/mL, pulse in beats/min.
//
// READING ORDER FOR A NEWCOMER
//   1 Params, rhs, rk4        the mathematics      2 run()        one simulation
//   3 evaluate, screen()      the search           4 C API        how Kotlin/JNI calls it
//
// STATUS NOTES (what the proposal plans but this file does NOT yet contain)
//   * No 64-byte alignment / structure-of-arrays tiling: State is a plain std::array<double,14>.
//   * No JNI glue: only the C API at the bottom; the Kotlin/JNI layer is a Phase-3 deliverable.
//   * R_eff here is the entry+clearance value only; the next-generation-matrix R_eff with a TIP
//     dose (proposal Section 4.4) and the Gillespie stochastic extinction check are not coded.
//   * Workers use std::thread, not std::jthread (works on every NDK toolchain).
//   * Every default parameter is an ILLUSTRATIVE placeholder, not a calibrated measurement.
//   * Parameter structs are read/written by byte offset in evaluate()/load_screen_stream():
//     Params must stay a plain struct whose swept members are doubles (see notes there).
// ============================================================================
// Only the standard library is used (zero-dependency commitment): a future reader needs a
// compiler and nothing else. <thread>/<atomic> provide the parallel screening workers.
#include <algorithm>
#include <array>
#include <atomic>
#include <chrono>
#include <cmath>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <fstream>
#include <limits>
#include <map>
#include <numeric>
#include <sstream>
#include <string>
#include <thread>
#include <vector>

// Index names for the state vector. Keeping them as an enum means every equation below reads
// like the paper: y[iVW] is V_W, y[iC] is the cytokine C, and so on. N is the vector length.
//   T  uninfected target cells        EW/IW  eclipse/productive WT-only infected cells
//   EC/IC  eclipse/productive co-infected cells (WT + TIP)      P  TIP-only "primed" cells
//   VW/VT  free WT / free TIP virions    C  cytokine (IL-6-like)    H  pulse rate
// 0..9 dynamic states (9 = P, TIP-only primed cells), 10..13 running integrals (JV, JC, JH, U) so objective weights can be applied afterwards
enum Idx { iT, iEW, iIW, iVW, iEC, iIC, iVT, iC, iH, iP, iJV, iJC, iJH, iU, N };
// kDyn = number of true dynamic states (the first ten); the last four entries are bookkeeping integrals.
constexpr int kDyn = 10;
// One point in time. A plain array: copying it is cheap and contains no hidden state.
using State = std::array<double, N>;

// Every number the model needs, with its default (the illustrative reference set; input.txt
// overrides any of them). Naming: b = beta (infection rate), d = delta/death, k = eclipse
// exit rate, p = production, c = clearance. W = wild type, T = TIP, Co = co-infected.
// Line 1: cell kinetics.  Line 2: death/production/hijacking/clearance.  Line 3: cytokine-
// mediated killing and the antibody-like entry (eps_ab) and clearance (eps_clr) effects.
// Line 4-5: cytokine and autonomic (vagal/pulse) subsystem.  Line 6: objective weights and
// safety bounds.  Line 7: run control (initial inoculum, TIP dose/time, horizon, step).
// The last line holds values DERIVED from the others (see derive()); never set them by hand.
struct Params {
  double lambda = 5e4, dT = 0.07, bW = 5e-7, bT = 5e-7, kW = 6, kCo = 6;
  double dW = 2, dCo = 2, pW = 500, pT = 1500, alpha = 0.2, cW = 10, cT = 10;
  double dN = 0.5, KF = 50, eps_ab = 0, eps_clr = 0, kappa = 1;
  double c_rest = 5, dC = 5, gamma = 3, KC = 50, rhoI = 0.005, rhoV = 1e-6;
  double sigma_v = 0.5, H0 = 70, Hmax = 130, dH = 50, KH = 100, n_hill = 2, tau_H = 0.02;
  double w1 = 1, w2 = 1, w3 = 1, w4 = 1e-6, C_crit = 500, H_crit = 130, v_ext = 2e-4;
  double primed = 1, rhoT = 0, ext_floor = 0, VW0 = 100, tip_dose = 0, tip_time = 0, t_end = 30, h = 0.005, dt_out = 0.25;
  double Ts = 0, Cs = 0, Hs = 0, s0 = 0;  // derived
};

// Hill function x^n/(K^n+x^n): a smooth saturating switch from 0 to 1 centred on K.
// Integer n is used so the power is exact multiplication (fast, deterministic, no pow()).
static double hill(double x, double K, int n) {
  double a = 1, b = 1;
```

```
  for (int i = 0; i < n; ++i) { a *= x; b *= K; }
  return a / (b + a);
}
// Vagal (cholinergic anti-inflammatory) gain m(H) of Eq 9: equal to 1-sigma_v at resting pulse
// H0 (maximum suppression of cytokine synthesis) and fading to 1 as H rises to Hmax.
// Bounded in [1-sigma_v, 1] by construction, which keeps the feedback loop well behaved.
static double vagal(const Params& p, double H) {
  return 1.0 - p.sigma_v * std::clamp((p.Hmax - H) / (p.Hmax - p.H0), 0.0, 1.0);
}

// THE MODEL. Right-hand side dy/dt = f(y) of Equations 1-10 (+14). PURE: no globals, no I/O,
// no randomness, so the same input always gives the same output bit for bit.
//   cyt  = cytokine-mediated extra death rate of infected cells (saturating in C)
//   inf  = rate of new WT infection of uninfected cells          (entry blocked by 1-eps_ab)
//   tip  = per-cell rate at which free TIP enters a cell         (beta_T * V_T)
//   supW = per-cell rate at which free WT enters a cell (used for superinfection of P cells)
static void rhs(const Params& p, const State& y, State& d) {
  const double ent = 1.0 - p.eps_ab;
  const double cyt = p.dN * y[iC] / (p.KF + y[iC]);
  const double inf = ent * p.bW * y[iT] * y[iVW];
  const double tip = ent * p.bT * y[iVT];
  const double supW = ent * p.bW * y[iVW];                                  // WT entry into a primed cell
  // --- cells (Eq 1-5, 4b). TIP entering an uninfected cell makes a PRIMED cell P (it cannot
  // replicate the TIP alone); WT can later enter P and turn it into a co-infected eclipse cell.
  // That superinfection path is the heart of the primed-cell postulate; primed=0 removes it.
  d[iT]  = p.lambda - p.dT * y[iT] - inf - tip * y[iT];                     // Eq 1
  d[iEW] = inf - p.kW * y[iEW] - tip * y[iEW];                              // Eq 2
  d[iIW] = p.kW * y[iEW] - (p.dW + cyt) * y[iIW] - tip * y[iIW];            // Eq 3
  d[iEC] = tip * y[iEW] + p.primed * supW * y[iP] - p.kCo * y[iEC];         // Eq 4 (+ superinfection of primed cells)
  d[iIC] = p.kCo * y[iEC] + tip * y[iIW] - (p.dCo + cyt) * y[iIC];          // Eq 5
  d[iP]  = p.primed * (tip * y[iT] - supW * y[iP]) - p.dT * y[iP];          // Eq 4b primed (TIP-only) cells
  // --- virus (Eq 6-7). Inside co-infected cells WT output is cut to alpha*pW (hijacking) while
  // TIP output is pT (> pW). Clearance of WT is boosted by antibody-like eps_clr; free TIP is
  // cleared at cT. The small ratio cT/cW is what the Persistence Theory is about.
  d[iVW] = p.pW * y[iIW] + p.alpha * p.pW * y[iIC]
         - p.cW * (1.0 + p.kappa * p.eps_clr) * y[iVW];                     // Eq 6
  d[iVT] = p.pT * y[iIC] - p.cT * y[iVT];                                   // Eq 7
  // --- cytokine (Eq 8-9) and pulse (Eq 10). m(H) multiplies ALL synthesis terms: basal s0,
  // infected-cell term rhoI, virus term rhoV, optional TIP-innate term rhoT (default 0) and the
  // second-order Hill self-amplification gamma*C^2/(KC^2+C^2); first-order clearance dC.
  d[iC]  = vagal(p, y[iH]) * (p.s0 + p.rhoI * (y[iIW] + y[iIC]) + p.rhoV * y[iVW] + p.rhoT * y[iVT]
         + p.gamma * hill(y[iC], p.KC, 2)) - p.dC * y[iC];                  // Eq 8-9
  d[iH]  = (p.H0 + p.dH * hill(y[iC], p.KH, (int)p.n_hill) - y[iH]) / p.tau_H;  // Eq 10
  // --- running integrals for the objective J (Eq 14): deviations are scaled by the safety
  // bounds and taken relative to the infection-free equilibrium (Cs, Hs); U = TIP exposure.
  const double ec = (y[iC] - p.Cs) / p.C_crit, eh = (y[iH] - p.Hs) / p.H_crit;
  d[iJV] = std::log10(1.0 + std::max(y[iVW], 0.0));                         // Eq 14 terms
  d[iJC] = ec * ec;
  d[iJH] = eh * eh;
  d[iU]  = y[iVT];
}

// Classical 4th-order Runge-Kutta, one fixed step h. Fixed (not adaptive) on purpose: with the
// same step sequence, results are reproducible bit for bit on any machine/thread count. The
// step is validated by the step-halving self-test (order should be ~4) and by jac_bound().
static void rk4(const Params& p, State& y, double h) {
  State k1, k2, k3, k4, s;
  rhs(p, y, k1);
  for (int i = 0; i < N; ++i) s[i] = y[i] + 0.5 * h * k1[i];
  rhs(p, s, k2);
  for (int i = 0; i < N; ++i) s[i] = y[i] + 0.5 * h * k2[i];
  rhs(p, s, k3);
  for (int i = 0; i < N; ++i) s[i] = y[i] + h * k3[i];
  rhs(p, s, k4);
  for (int i = 0; i < N; ++i) y[i] += (h / 6.0) * (k1[i] + 2 * k2[i] + 2 * k3[i] + k4[i]);
}

// STABILITY GUARD. RK4 is stable only if h*|lambda_max| < ~2.78 (real axis). Infection terms
// such as bW*VW can reach tens-hundreds per day at peak viraemia, so before each integration
// segment we bound the Jacobian spectral radius by its infinity norm (rows of |df_i/dy_j|,
// by forward differences) and advance() shrinks h if h*bound would exceed 2.5.
// infinity-norm of a finite-difference Jacobian of the dynamic block: bound on the spectral radius
static double jac_bound(const Params& p, const State& y) {
  State f0, f1, z = y;
  rhs(p, y, f0);
  std::array<double, kDyn> row{};
  for (int j = 0; j < kDyn; ++j) {
    const double e = 1e-6 * std::max(1.0, std::fabs(y[j]));
    z[j] = y[j] + e;
    rhs(p, z, f1);
    z[j] = y[j];
    for (int i = 0; i < kDyn; ++i) row[i] += std::fabs((f1[i] - f0[i]) / e);
  }
  return *std::max_element(row.begin(), row.end());
}

// ---- Parameter input. Plain-text "name = value" lines; '#' starts a comment. Unknown names,
// bad numbers and out-of-range values are REPORTED, never silently ignored.
// ------------------------------------------------------------------ parameters
// Table row linking a text-file name (e.g. "beta_T") to the C++ member it sets.
struct Entry { const char* name; double* ptr; };
// The single source of truth for which parameters can be set from text and swept in screening.
// To add a parameter: add the member to Params, one row here, and (if needed) a check in validate().
static std::vector<Entry> registry(Params& p) {
  return {{"lambda", &p.lambda}, {"d_T", &p.dT}, {"beta_W", &p.bW}, {"beta_T", &p.bT},
    {"k_W", &p.kW}, {"k_Co", &p.kCo}, {"delta_W", &p.dW}, {"delta_Co", &p.dCo},
    {"p_W", &p.pW}, {"p_T", &p.pT}, {"alpha", &p.alpha}, {"c_W", &p.cW}, {"c_T", &p.cT},
    {"delta_N", &p.dN}, {"K_F", &p.KF}, {"eps_Ab", &p.eps_ab}, {"eps_Clr", &p.eps_clr},
    {"kappa", &p.kappa}, {"C_rest", &p.c_rest}, {"d_C", &p.dC}, {"gamma", &p.gamma},
    {"K_C", &p.KC}, {"rho_I", &p.rhoI}, {"rho_V", &p.rhoV}, {"rho_T", &p.rhoT}, {"primed", &p.primed}, {"sigma_v", &p.sigma_v},
    {"H_0", &p.H0}, {"H_max", &p.Hmax}, {"Delta_H", &p.dH}, {"K_H", &p.KH},
    {"n_hill", &p.n_hill}, {"tau_H", &p.tau_H}, {"w1", &p.w1}, {"w2", &p.w2},
    {"w3", &p.w3}, {"w4", &p.w4}, {"C_crit", &p.C_crit}, {"H_crit", &p.H_crit},
    {"V_ext", &p.v_ext}, {"ext_floor", &p.ext_floor}, {"V_W0", &p.VW0}, {"tip_dose", &p.tip_dose},
    {"tip_time", &p.tip_time}, {"t_end", &p.t_end}, {"h", &p.h}, {"dt_out", &p.dt_out}};
}

static std::string trim(std::string s) {
  s.erase(0, s.find_first_not_of(" \t\r")); s.erase(s.find_last_not_of(" \t\r") + 1); return s;
}
```

```
static std::string errs_line(int ln, const std::string& m) { return "ERROR: line " + std::to_string(ln) + ": " + m; }

// Parse a stream of "name = value" lines into Params; every problem is appended to err with its
// line number. Used for both files (CLI) and in-memory text (Android/JNI), so behaviour is identical.
static void load_stream(std::istream& in, Params& p, std::vector<std::string>& err) {
  auto reg = registry(p);
  std::string line; int ln = 0;
  while (std::getline(in, line)) {
    ++ln;
    if (auto c = line.find('#'); c != std::string::npos) line.erase(c);
    auto eq = line.find('=');
    if (eq == std::string::npos) {
      if (!trim(line).empty()) err.push_back(errs_line(ln, "expected 'name = value'"));
      continue;
    }
    const std::string key = trim(line.substr(0, eq)), val = trim(line.substr(eq + 1));
    char* end = nullptr;
    const double v = std::strtod(val.c_str(), &end);
    if (val.empty() || *end != '\0' || !std::isfinite(v)) { err.push_back(errs_line(ln, "bad number for '" + key + "'")); continue; }
    bool found = false;
    for (auto& e : reg) if (key == e.name) { *e.ptr = v; found = true; }
    if (!found) err.push_back(errs_line(ln, "unknown parameter '" + key + "'"));
  }
}

// File wrapper around load_stream (the CLI uses it; the C API does not, hence maybe_unused).
[[maybe_unused]] static void load(const std::string& path, Params& p, std::vector<std::string>& err) {
  std::ifstream in(path);
  if (!in) { err.push_back("ERROR: cannot open " + path); return; }
  load_stream(in, p, err);
}

// Derived quantities. Infection-free equilibrium: T* = lambda/dT; C* = C_rest (chosen by the user);
// H* from Eq 10 at C*; and the basal cytokine synthesis s0 that makes C* an exact equilibrium of
// Eq 8 (solve dC/dt = 0 with no infection). So "C_rest" is an input and s0 follows from it.
static void derive(Params& p) {
  p.Ts = p.lambda / p.dT;
  p.Cs = p.c_rest;
  p.Hs = p.H0 + p.dH * hill(p.Cs, p.KH, (int)p.n_hill);
  p.s0 = p.dC * p.Cs / vagal(p, p.Hs) - p.gamma * hill(p.Cs, p.KC, 2);
}

// Reject physically meaningless input (errors) and flag risky-but-legal input (warnings).
// Warning example: if Eq 11 fails, the cytokine subsystem may be bistable (the "storm class").
// Order matters: derive() runs only if all basic checks pass, because it divides by parameters.
static void validate(Params& p, std::vector<std::string>& err) {
  auto pos = [&](const char* n, double v) { if (!(v > 0)) err.push_back(std::string("ERROR: ") + n + " must be > 0"); };
  pos("lambda", p.lambda); pos("d_T", p.dT); pos("beta_W", p.bW); pos("k_W", p.kW);
  pos("k_Co", p.kCo); pos("delta_W", p.dW); pos("delta_Co", p.dCo); pos("p_W", p.pW);
  pos("p_T", p.pT); pos("c_W", p.cW); pos("c_T", p.cT); pos("K_F", p.KF); pos("K_C", p.KC);
  pos("K_H", p.KH); pos("d_C", p.dC); pos("tau_H", p.tau_H); pos("C_rest", p.c_rest);
  pos("C_crit", p.C_crit); pos("H_crit", p.H_crit); pos("t_end", p.t_end); pos("h", p.h);
  pos("dt_out", p.dt_out);
  if (p.bT < 0 || p.dN < 0 || p.gamma < 0 || p.rhoI < 0 || p.rhoV < 0 || p.VW0 < 0 || p.tip_dose < 0)
    err.push_back("ERROR: beta_T, delta_N, gamma, rho_I, rho_V, V_W0, tip_dose must be >= 0");
  if (p.alpha < 0 || p.alpha > 1) err.push_back("ERROR: alpha must be in [0,1]");
  if (p.sigma_v < 0 || p.sigma_v >= 1) err.push_back("ERROR: sigma_v must be in [0,1)");
  if (p.eps_ab < 0 || p.eps_ab >= 1) err.push_back("ERROR: eps_Ab must be in [0,1)");
  if (p.eps_clr < 0 || p.eps_clr > 1) err.push_back("ERROR: eps_Clr must be in [0,1]");
  if (p.n_hill < 1 || p.n_hill > 4 || p.n_hill != std::floor(p.n_hill)) err.push_back("ERROR: n_hill must be an integer 1..4");
  if (p.ext_floor != 0 && p.ext_floor != 1) err.push_back("ERROR: ext_floor must be 0 or 1");
  if (p.Hmax <= p.H0) err.push_back("ERROR: H_max must exceed H_0");
  if (p.tip_time < 0 || p.tip_time > p.t_end) err.push_back("ERROR: tip_time must lie in [0, t_end]");
  if (p.dt_out < p.h) err.push_back("ERROR: dt_out must be >= h");
  if (!err.empty()) return;
  derive(p);
  if (p.s0 < 0) err.push_back("ERROR: C_rest too low for these parameters (basal synthesis s_0 would be negative)");
  if (3.0 * std::sqrt(3.0) / 8.0 * p.gamma / p.KC >= p.dC)
    err.push_back("WARNING: Eq 11 violated - cytokine subsystem may be bistable (storm-class regime)");
}

// Basic reproduction number of WT alone, Eq 12: R0 = bW*pW*T*/(cW*delta_eff), delta_eff at C*.
static double R0(const Params& p) {
  return p.bW * p.pW * p.Ts / (p.cW * (p.dW + p.dN * p.Cs / (p.KF + p.Cs)));  // Eq 12
}

// ---- One simulation: integrate from the infection-free state plus a WT inoculum, apply the TIP dose, record metrics.
// ---------------------------------------------------------------------- one run
// A recorded trajectory: output times t[k] and the full state y[k] at those times.
struct Traj { std::vector<double> t; std::vector<State> y; };
// Summary metrics of one run: peak WT load and its time, peak cytokine/pulse, extinction flag and
// time, final WT, the four objective integrals, and ok=false if an invariant (finite, non-negative) broke.
struct Result {
  double peakV = 0, tPeakV = 0, peakC = 0, peakH = 0, tExt = -1, Vfinal = 0;
  std::array<double, 4> I{};  // JV, JC, JH, U
  bool extinct = false, ok = true;
  int reduced = 0;
};
// Objective J of Eq 14 = w1*JV + w2*JC + w3*JH + w4*U (smaller is better).
static double Jw(const Params& p, const Result& r) { return p.w1 * r.I[0] + p.w2 * r.I[1] + p.w3 * r.I[2] + p.w4 * r.I[3]; }

// Simulate one host. Steps: (1) infection-free start with V_W0 inoculum; (2) at t = tip_time add
// tip_dose to the free TIP pool; (3) march with RK4 between output times, splitting a segment so
// the dose lands exactly on tip_time; (4) after every step check invariants, apply the optional
// extinction floor, and update peaks. EXTINCTION means V_W < V_ext and stays below until t_end.
static Result run(const Params& p, Traj* tr, std::vector<std::string>& err) {
  Result r;
  State y{};
  y[iT] = p.Ts; y[iVW] = p.VW0; y[iC] = p.Cs; y[iH] = p.Hs;
  double t = 0, lastAbove = -1;
  bool dosed = false;
  // One-shot TIP dose: an instantaneous jump of V_T (e.g. an injection), applied once at tip_time.
  auto dose_if_due = [&] { if (!dosed && t >= p.tip_time - 1e-12) { y[iVT] += p.tip_dose; dosed = true; } };
  // Update running maxima and remember the last moment V_W was still at/above V_ext (for t_ext).
  auto track = [&] {
    if (y[iVW] > r.peakV) { r.peakV = y[iVW]; r.tPeakV = t; }
    r.peakC = std::max(r.peakC, y[iC]); r.peakH = std::max(r.peakH, y[iH]);
    if (y[iVW] >= p.v_ext) lastAbove = t;
  };
  // Integrate to time "to". The step is the smaller of h and the stability limit 2.5/rho, then
```

```
  // divided evenly so the segment ends exactly at "to". Invariants are checked every step.
  auto advance = [&](double to) {
    const double rho = jac_bound(p, y);
    const double hmax = std::min(p.h, 2.5 / std::max(rho, 1e-12));  // pre-flight stability bound
    const int n = std::max(1, (int)std::ceil((to - t) / hmax - 1e-9));
    if (hmax < p.h) ++r.reduced;
    const double hs = (to - t) / n;
    for (int s = 0; s < n; ++s) {
      rk4(p, y, hs);
      t += hs;
      for (int i = 0; i < N; ++i)
        if (!std::isfinite(y[i]) || y[i] < -1e-6 * std::max(1.0, std::fabs(p.Ts))) {
          err.push_back("ERROR: invariant broken (non-finite or negative state) at t = " + std::to_string(t));
          r.ok = false; return;
        }
      // Extinction floor: a continuous ODE never reaches exactly zero, so when fewer than one virion
      // and fewer than one infected cell per host remain, WT is set to zero (proxy for stochastic extinction).
      if (p.ext_floor > 0.5 && y[iVW] < p.v_ext &&
          y[iEW] + y[iIW] + y[iEC] + y[iIC] < p.v_ext) {  // WT-free: less than one virion/cell per host
        y[iVW] = y[iEW] = y[iIW] = y[iEC] = y[iIC] = 0;
      }
      track();
    }
    t = to;
  };
  // Save the current state when a trajectory is requested (screening passes nullptr to save memory).
  auto emit = [&] { if (tr) { tr->t.push_back(t); tr->y.push_back(y); } };
  dose_if_due(); track(); emit();
  const int nOut = (int)std::ceil(p.t_end / p.dt_out - 1e-9);
  for (int k = 1; k <= nOut && r.ok; ++k) {
    const double target = std::min(k * p.dt_out, p.t_end);
    if (!dosed && p.tip_time > t && p.tip_time < target) { advance(p.tip_time); dose_if_due(); }
    if (!r.ok) break;
    advance(target);
    dose_if_due();
    emit();
  }
  for (int i = 0; i < 4; ++i) r.I[i] = y[iJV + i];
  r.Vfinal = y[iVW];
  r.extinct = r.ok && y[iVW] < p.v_ext;
  if (r.extinct) r.tExt = std::max(lastAbove, 0.0);
  return r;
}

// ---- Plots written as plain SVG text (vector, no libraries, viewable in any browser). The reader
// of the future needs nothing but a text editor to regenerate or inspect the figures.
// ---------------------------------------------------------------------- SVG plots (no dependencies)
// One curve on a plot: x/y points, colour, dashed flag and a legend name.
struct Series { std::vector<double> x, y; std::string color; bool dashed = false; std::string name; };
static std::string fmt(double v) { char b[32]; std::snprintf(b, sizeof b, "%.3g", v); return b; }
static const double kNaN = std::numeric_limits<double>::quiet_NaN();

// Draw one framed panel (axes, ticks, labels, optional horizontal safety line, curves) at (X,Y) of size WxH.
// logy plots log10 of the data. Returns SVG fragments; no state is kept.
static std::string panel(double X, double Y, double W, double H, const std::string& title, const std::string& ylab,
                         const std::vector<Series>& ss, bool logy, double hline, const std::string& hlab, double xlo_ = kNaN) {
  const double L = 76, R = 14, T = 26, B = 34;
  double x0 = 1e300, x1 = -1e300, y0 = 1e300, y1 = -1e300;
  auto ty = [&](double v) { return logy ? std::log10(std::max(v, 1e-4)) : v; };
  for (auto& s : ss) for (size_t i = 0; i < s.x.size(); ++i) {
    x0 = std::min(x0, s.x[i]); x1 = std::max(x1, s.x[i]); y0 = std::min(y0, ty(s.y[i])); y1 = std::max(y1, ty(s.y[i])); }
  if (!std::isnan(hline)) { y0 = std::min(y0, ty(hline)); y1 = std::max(y1, ty(hline)); }
  if (!std::isnan(xlo_)) x0 = xlo_;
  if (logy) { y0 = std::floor(y0); y1 = std::ceil(y1); if (y1 - y0 < 1) y1 = y0 + 1; }
  else { y0 = std::min(0.0, y0); y1 = y1 * 1.06 + 1e-12; }
  if (x1 <= x0) x1 = x0 + 1;
  const double pw = W - L - R, ph = H - T - B;
  auto px = [&](double v) { return X + L + (v - x0) / (x1 - x0) * pw; };
  auto py = [&](double v) { return Y + T + (1 - (ty(v) - y0) / (y1 - y0)) * ph; };
  auto pyr = [&](double raw) { return Y + T + (1 - (raw - y0) / (y1 - y0)) * ph; };
  std::ostringstream o;
  o << "<text x='" << X + L << "' y='" << Y + 16 << "' font-size='13' font-weight='bold'>" << title << "</text>\n";
  o << "<rect x='" << X + L << "' y='" << Y + T << "' width='" << pw << "' height='" << ph << "' fill='none' stroke='#888'/>\n";
  for (int k = 0; k <= 4; ++k) {
    const double fx = x0 + (x1 - x0) * k / 4.0, fy = y0 + (y1 - y0) * k / 4.0;
    o << "<line x1='" << px(fx) << "' y1='" << Y + T + ph << "' x2='" << px(fx) << "' y2='" << Y + T + ph + 4 << "' stroke='#888'/>"
      << "<text x='" << px(fx) << "' y='" << Y + T + ph + 17 << "' font-size='11' text-anchor='middle'>" << fmt(fx) << "</text>\n";
    o << "<line x1='" << X + L - 4 << "' y1='" << pyr(fy) << "' x2='" << X + L << "' y2='" << pyr(fy) << "' stroke='#888'/>"
      << "<text x='" << X + L - 7 << "' y='" << pyr(fy) + 4 << "' font-size='11' text-anchor='end'>"
      << (logy ? "1e" + fmt(fy) : fmt(fy)) << "</text>\n";
  }
  o << "<text x='" << X + L + pw / 2 << "' y='" << Y + H - 4 << "' font-size='11' text-anchor='middle'>time (days)</text>\n";
  o << "<text x='" << X + 12 << "' y='" << Y + T + ph / 2 << "' font-size='11' text-anchor='middle' transform='rotate(-90 " << X + 12 << " " << Y + T + ph / 2 << ")'>" << ylab << "</text>\n";
  if (!std::isnan(hline)) {
    o << "<line x1='" << X + L << "' y1='" << py(hline) << "' x2='" << X + L + pw << "' y2='" << py(hline)
      << "' stroke='#A32D2D' stroke-dasharray='2 3'/><text x='" << X + L + pw - 3 << "' y='" << py(hline) - 4
      << "' font-size='11' text-anchor='end' fill='#A32D2D'>" << hlab << "</text>\n";
  }
  for (auto& s : ss) {
    o << "<polyline fill='none' stroke='" << s.color << "' stroke-width='1.8'" << (s.dashed ? " stroke-dasharray='6 4'" : "") << " points='";
    for (size_t i = 0; i < s.x.size(); ++i) o << px(s.x[i]) << "," << py(s.y[i]) << " ";
    o << "'/>\n";
  }
  return o.str();
}

// Draw a one-row legend of coloured line samples.
static void legend(std::ostringstream& o, double x, double y, const std::vector<std::pair<std::string, std::pair<std::string, bool>>>& items) {
  for (auto& it : items) {
    o << "<line x1='" << x << "' y1='" << y << "' x2='" << x + 26 << "' y2='" << y << "' stroke='" << it.second.first
      << "' stroke-width='2'" << (it.second.second ? " stroke-dasharray='6 4'" : "") << "/><text x='" << x + 32 << "' y='" << y + 4
      << "' font-size='12'>" << it.first << "</text>\n";
    x += 40 + 7.2 * it.first.size();
  }
}

// Extract column idx (optionally idx+idx2, e.g. total of two compartments) of a trajectory as a Series.
static Series col(const Traj& t, int idx, const std::string& c, bool dashed, const std::string& name, int idx2 = -1) {
  Series s; s.color = c; s.dashed = dashed; s.name = name; s.x = t.t;
  for (auto& v : t.y) s.y.push_back(v[idx] + (idx2 >= 0 ? v[idx2] : 0.0));
```

```
  return s;
}

// 2x2 dashboard: free virus (log), infected cells, cytokine vs C_crit, pulse vs H_crit.
// "b" is the optional untreated trajectory drawn dashed for comparison. Returned as a string
// so the Android app can display it without touching the file system.
static std::string traj_svg_str(const std::string& title, const Params& p, const Traj& a, const Traj* b) {
  std::ostringstream o;
  const double W = 440, H = 250;
  o << "<svg xmlns='http://www.w3.org/2000/svg' width='" << 2 * W + 20 << "' height='" << 2 * H + 70
    << "' viewBox='0 0 " << 2 * W + 20 << " " << 2 * H + 70 << "' font-family='Arial,Helvetica,sans-serif'>\n<rect width='100%' height='100%' fill='white'/>\n";
  o << "<text x='10' y='20' font-size='15' font-weight='bold'>" << title << "</text>\n";
  const std::string blue = "#185FA5", coral = "#D85A30", grey = "#888780";
  { std::vector<Series> s{col(a, iVW, blue, false, "V_W")};
    s.push_back(col(a, iVT, coral, false, "V_T"));
    if (b) s.push_back(col(*b, iVW, grey, true, "V_W untreated"));
    o << panel(10, 30, W, H, "Free virus (copies/mL, log)", "log10 copies/mL", s, true, p.v_ext, "V_ext"); }
  { std::vector<Series> s{col(a, iIW, blue, false, "I_W"), col(a, iIC, coral, false, "I_Co")};
    if (b) s.push_back(col(*b, iIW, grey, true, "I_W untreated"));
    o << panel(20 + W, 30, W, H, "Productively infected cells", "cells/mL", s, false, kNaN, ""); }
  { std::vector<Series> s{col(a, iC, blue, false, "C")};
    if (b) s.push_back(col(*b, iC, grey, true, "C untreated"));
    o << panel(10, 30 + H, W, H, "Cytokine (IL-6-like)", "pg/mL", s, false, p.C_crit, "C_crit"); }
  { std::vector<Series> s{col(a, iH, blue, false, "H")};
    if (b) s.push_back(col(*b, iH, grey, true, "H untreated"));
    o << panel(20 + W, 30 + H, W, H, "Pulse rate", "beats/min", s, false, p.H_crit, "H_crit"); }
  legend(o, 14, 2 * H + 52, {{"with TIP", {blue, false}}, {"TIP virions V_T / I_Co", {coral, false}}, {"untreated", {grey, true}}});
  o << "</svg>\n";
  return o.str();
}
// Same figure written to a file (CLI use).
static void write_traj_svg(const std::string& path, const std::string& title, const Params& p, const Traj& a, const Traj* b) {
  std::ofstream(path) << traj_svg_str(title, p, a, b);
}

// ---- Random numbers and threading helpers (self-contained so results do not depend on the platform library).
// ---------------------------------------------------------------------- RNG and helpers
// xoshiro256** generator seeded by SplitMix64: tiny, fast, high quality, and fully specified,
// so the Latin-hypercube design is identical on every compiler and machine for a given seed.
struct Rng {
  uint64_t s[4];
  explicit Rng(uint64_t seed) {
    for (auto& x : s) { seed += 0x9e3779b97f4a7c15ULL; uint64_t z = seed;
      z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9ULL; z = (z ^ (z >> 27)) * 0x94d049bb133111ebULL; x = z ^ (z >> 31); }
  }
  uint64_t next() {
    auto rotl = [](uint64_t x, int k) { return (x << k) | (x >> (64 - k)); };
    const uint64_t r = rotl(s[1] * 5, 7) * 9, t = s[1] << 17;
    s[2] ^= s[0]; s[3] ^= s[1]; s[1] ^= s[2]; s[0] ^= s[3]; s[2] ^= t; s[3] = rotl(s[3], 45);
    return r;
  }
  double u01() { return (next() >> 11) * (1.0 / 9007199254740992.0); }
};

// Run fn(i) for i in [0,n) on a pool of threads that grab the next index from an atomic counter.
// Each i writes only its own result slot, so the answer cannot depend on thread count or timing
// (checked: 1-thread and 4-thread screens give byte-identical CSV output).
template <class F> static void parallel_for(size_t n, int threads, F fn) {
  std::atomic<size_t> next{0};
  std::vector<std::thread> pool;  // std::thread (not jthread): works on every NDK toolchain
  for (int t = 0; t < threads; ++t)
    pool.emplace_back([&] { for (;;) { const size_t i = next.fetch_add(1); if (i >= n) break; fn(i); } });
  for (auto& th : pool) th.join();
}

// ---- TIP-profile screening: sample the design space, evaluate every profile with run(), rank,
// refine, find the Pareto front, and measure which parameters matter.
// ---------------------------------------------------------------------- screening
// One search dimension: parameter name, range [lo,hi], log-scale flag, and its byte offset inside Params.
struct Dim { std::string name; double lo, hi; bool lg; size_t off; };
// Screening settings read from screen.txt: dimensions plus sample count, seed, refinement effort, weights, output prefix.
struct ScreenCfg {
  std::vector<Dim> dims;
  int samples = 2000, refine_top = 3, nm_iters = 80, threads = 0, top_k = 10;
  double seed = 12345, wfactor = 2, w_ext = 5;
  std::string prefix = "screen";
};

// Parse screen.txt. Control keys (seed, samples, ...) set ScreenCfg; any registry name followed by
// "min max [log]" becomes a search dimension. OFFSET TRICK: the dimension stores the member's byte
// offset in Params so evaluate() can write a sampled value into a copy of Params without a
// big switch. This is valid only because every swept member is a double inside one standard-layout struct.
static void load_screen_stream(std::istream& in, ScreenCfg& c, Params& base, std::vector<std::string>& err) {
  auto reg = registry(base);
  std::string line; int ln = 0;
  while (std::getline(in, line)) {
    ++ln;
    if (auto k = line.find('#'); k != std::string::npos) line.erase(k);
    auto eq = line.find('=');
    if (eq == std::string::npos) { if (!trim(line).empty()) err.push_back(errs_line(ln, "expected 'name = value'")); continue; }
    const std::string key = trim(line.substr(0, eq));
    std::istringstream rest(line.substr(eq + 1));
    std::vector<std::string> tok; std::string w;
    while (rest >> w) tok.push_back(w);
    auto num = [&](const std::string& s, double& v) { char* e = nullptr; v = std::strtod(s.c_str(), &e); return !s.empty() && *e == '\0' && std::isfinite(v); };
    double v = 0;
    if (key == "prefix") { if (tok.size() == 1) c.prefix = tok[0]; else err.push_back(errs_line(ln, "prefix needs one word")); continue; }
    const std::map<std::string, double*> ctl = {{"seed", &c.seed}, {"weight_factor", &c.wfactor}, {"w_ext", &c.w_ext}};
    const std::map<std::string, int*> ictl = {{"samples", &c.samples}, {"refine_top", &c.refine_top}, {"nm_iters", &c.nm_iters},
                                              {"threads", &c.threads}, {"top_k", &c.top_k}};
    if (ctl.count(key)) { if (tok.size() == 1 && num(tok[0], v)) *ctl.at(key) = v; else err.push_back(errs_line(ln, "bad value for " + key)); continue; }
    if (ictl.count(key)) { if (tok.size() == 1 && num(tok[0], v)) *ictl.at(key) = (int)v; else err.push_back(errs_line(ln, "bad value for " + key)); continue; }
    const Entry* hit = nullptr;
    for (auto& e : reg) if (key == e.name) hit = &e;
    if (!hit) { err.push_back(errs_line(ln, "unknown parameter '" + key + "'")); continue; }
    double lo, hi;
    if (tok.size() < 2 || !num(tok[0], lo) || !num(tok[1], hi) || lo >= hi) { err.push_back(errs_line(ln, "need 'min max [log]' with min < max for " + key)); continue; }
    const bool lg = tok.size() > 2 && tok[2] == "log";
    if (lg && lo <= 0) { err.push_back(errs_line(ln, "log range needs min > 0 for " + key)); continue; }
    c.dims.push_back({key, lo, hi, lg, (size_t)((char*)hit->ptr - (char*)&base)});
```

```
  }
  if (c.dims.empty()) err.push_back("ERROR: screen file defines no search dimensions");
  if (c.samples < 10) err.push_back("ERROR: samples must be >= 10");
}

// File wrapper for load_screen_stream.
[[maybe_unused]] static void load_screen(const std::string& path, ScreenCfg& c, Params& base, std::vector<std::string>& err) {
  std::ifstream in(path);
  if (!in) { err.push_back("ERROR: cannot open " + path); return; }
  load_screen_stream(in, c, base, err);
}

// Result of evaluating one candidate profile: its parameter values x, the run Result, validity,
// feasibility (safety bounds respected), penalty, scalar objective F, and a class:
// 0 = feasible AND WT extinction (best), 1 = feasible, 2 = violates a safety bound.
struct Eval {
  std::vector<double> x;
  Result r;
  bool valid = false, feasible = false;
  double pen = 0, F = 0;
  int cls = 2;  // 0 = feasible + extinction, 1 = feasible, 2 = infeasible
};

// Map a unit-interval coordinate u in [0,1] to the real parameter range (linear or logarithmic).
static double dimval(const Dim& d, double u) {
  u = std::clamp(u, 0.0, 1.0);
  return d.lg ? std::exp(std::log(d.lo) + u * (std::log(d.hi) - std::log(d.lo))) : d.lo + u * (d.hi - d.lo);
}

// Evaluate one profile. F = J (Eq 14) + penalty, where the penalty is 1000x the relative excess of
// peak cytokine / pulse over their bounds plus w_ext*log10(final V_W / V_ext) so that "closer to
// extinction" is rewarded smoothly. Screening uses a coarser OUTPUT grid (dt_out >= 1 d) for
// speed; peaks are still tracked on every integration step, so results are not degraded.
static Eval evaluate(const Params& base, const ScreenCfg& c, const std::vector<double>& u) {
  Eval e;
  Params q = base;
  for (size_t j = 0; j < c.dims.size(); ++j) {
    const double v = dimval(c.dims[j], u[j]);
    *(double*)((char*)&q + c.dims[j].off) = v;
    e.x.push_back(v);
  }
  q.dt_out = std::max(q.dt_out, 1.0);  // coarser output grid in screening (peaks are tracked every step)
  std::vector<std::string> tmp;
  validate(q, tmp);
  for (auto& s : tmp) if (s.rfind("ERROR", 0) == 0) { e.F = 1e12; e.pen = 1e12; return e; }
  e.r = run(q, nullptr, tmp);
  if (!e.r.ok) { e.F = 1e12; e.pen = 1e12; return e; }
  e.valid = true;
  e.feasible = e.r.peakC <= q.C_crit && e.r.peakH <= q.H_crit;
  e.pen = 1e3 * std::max(0.0, e.r.peakC / q.C_crit - 1) + 1e3 * std::max(0.0, e.r.peakH / q.H_crit - 1)
        + c.w_ext * std::log10(std::max(e.r.Vfinal, q.v_ext) / q.v_ext);
  e.F = Jw(q, e.r) + e.pen;
  e.cls = !e.feasible ? 2 : (e.r.extinct ? 0 : 1);
  return e;
}

// Nelder-Mead downhill simplex (Nelder & Mead, 1965) in the unit cube, used to polish the best
// sampled profiles. Derivative-free because the objective has kinks (extinction floor, bounds).
// Standard moves: reflect (-1), expand (-2), contract (+-0.5), shrink (0.5). Coordinates are
// clamped to [0,1], and the loop stops early when the simplex has collapsed (relative F spread 1e-7).
// Nelder-Mead in the unit cube
static Eval nelder_mead(const Params& base, const ScreenCfg& c, std::vector<double> u0, int& evals) {
  const size_t d = u0.size();
  auto f = [&](std::vector<double>& u) { for (auto& v : u) v = std::clamp(v, 0.0, 1.0); ++evals; return evaluate(base, c, u); };
  std::vector<std::vector<double>> V(d + 1, u0);
  std::vector<Eval> E(d + 1);
  for (size_t i = 0; i < d; ++i) { V[i + 1][i] += (V[i + 1][i] > 0.9 ? -0.1 : 0.1); }
  for (size_t i = 0; i <= d; ++i) E[i] = f(V[i]);
  for (int it = 0; it < c.nm_iters; ++it) {
    std::vector<size_t> o(d + 1); std::iota(o.begin(), o.end(), 0);
    std::sort(o.begin(), o.end(), [&](size_t a, size_t b) { return E[a].F < E[b].F; });
    auto V2 = V; auto E2 = E;
    for (size_t i = 0; i <= d; ++i) { V[i] = V2[o[i]]; E[i] = E2[o[i]]; }
    if (std::fabs(E[d].F - E[0].F) < 1e-7 * (std::fabs(E[0].F) + 1e-9)) break;
    std::vector<double> cen(d, 0.0);
    for (size_t i = 0; i < d; ++i) for (size_t j = 0; j < d; ++j) cen[j] += V[i][j] / d;
    auto mix = [&](double t) { std::vector<double> v(d); for (size_t j = 0; j < d; ++j) v[j] = cen[j] + t * (V[d][j] - cen[j]); return v; };
    auto xr = mix(-1.0); Eval er = f(xr);
    if (er.F < E[0].F) { auto xe = mix(-2.0); Eval ee = f(xe); if (ee.F < er.F) { V[d] = xe; E[d] = ee; } else { V[d] = xr; E[d] = er; } }
    else if (er.F < E[d - 1].F) { V[d] = xr; E[d] = er; }
    else {
      auto xc = er.F < E[d].F ? mix(-0.5) : mix(0.5); Eval ec = f(xc);
      if (ec.F < std::min(er.F, E[d].F)) { V[d] = xc; E[d] = ec; }
      else for (size_t i = 1; i <= d; ++i) { for (size_t j = 0; j < d; ++j) V[i][j] = V[0][j] + 0.5 * (V[i][j] - V[0][j]); E[i] = f(V[i]); }
    }
  }
  size_t b = 0;
  for (size_t i = 1; i <= d; ++i) if (E[i].F < E[b].F) b = i;
  return E[b];
}

// Average ranks (ties share the mean rank): the rank transform that turns Pearson into Spearman/PRCC.
static std::vector<double> ranks(const std::vector<double>& v) {
  std::vector<size_t> o(v.size()); std::iota(o.begin(), o.end(), 0);
  std::sort(o.begin(), o.end(), [&](size_t a, size_t b) { return v[a] < v[b]; });
  std::vector<double> r(v.size());
  for (size_t i = 0; i < o.size();) {
    size_t j = i; while (j + 1 < o.size() && v[o[j + 1]] == v[o[i]]) ++j;
    for (size_t k = i; k <= j; ++k) r[o[k]] = 0.5 * (i + j) + 1;
    i = j + 1;
  }
  return r;
}
// Gauss-Jordan matrix inverse with partial pivoting; returns false if (near-)singular. Small matrices only.
static bool invert(std::vector<std::vector<double>>& A) {
  const size_t n = A.size();
  std::vector<std::vector<double>> I(n, std::vector<double>(n, 0.0));
  for (size_t i = 0; i < n; ++i) I[i][i] = 1;
  for (size_t c = 0; c < n; ++c) {
    size_t p = c; for (size_t r = c + 1; r < n; ++r) if (std::fabs(A[r][c]) > std::fabs(A[p][c])) p = r;
```

```
    if (std::fabs(A[p][c]) < 1e-12) return false;
    std::swap(A[p], A[c]); std::swap(I[p], I[c]);
    const double d = A[c][c];
    for (size_t j = 0; j < n; ++j) { A[c][j] /= d; I[c][j] /= d; }
    for (size_t r = 0; r < n; ++r) if (r != c) { const double f = A[r][c];
      for (size_t j = 0; j < n; ++j) { A[r][j] -= f * A[c][j]; I[r][j] -= f * I[c][j]; } }
  }
  A = I; return true;
}
// PRCC (Marino et al., 2008): rank-transform inputs and output, build the correlation matrix, invert
// it, and read partial correlations from the inverse. |PRCC| near 1 means the input strongly drives
// the output with the other inputs held fixed; sign gives direction.
// partial rank correlation of every input with output y (Marino et al. 2008)
static std::vector<double> prcc(const std::vector<std::vector<double>>& X, const std::vector<double>& y) {
  const size_t d = X.size(), n = y.size();
  std::vector<std::vector<double>> R;
  for (auto& col : X) R.push_back(ranks(col));
  R.push_back(ranks(y));
  std::vector<std::vector<double>> C(d + 1, std::vector<double>(d + 1, 0.0));
  std::vector<double> mean(d + 1, 0.0), sd(d + 1, 0.0);
  for (size_t i = 0; i <= d; ++i) { for (double v : R[i]) mean[i] += v / n; for (double v : R[i]) sd[i] += (v - mean[i]) * (v - mean[i]); sd[i] = std::sqrt(sd[i]); }
  for (size_t i = 0; i <= d; ++i) for (size_t j = 0; j <= d; ++j) {
    double s = 0; for (size_t k = 0; k < n; ++k) s += (R[i][k] - mean[i]) * (R[j][k] - mean[j]);
    C[i][j] = (sd[i] > 0 && sd[j] > 0) ? s / (sd[i] * sd[j]) : (i == j ? 1.0 : 0.0);
  }
  std::vector<double> out(d, kNaN);
  if (!invert(C)) return out;
  for (size_t j = 0; j < d; ++j) out[j] = -C[j][d] / std::sqrt(C[j][j] * C[d][d]);
  return out;
}

// Format a double with 4 significant digits for reports.
static std::string fv(double v) { char b[32]; std::snprintf(b, sizeof b, "%.4g", v); return b; }

// The whole screening workflow. Deterministic for a given seed (and thread count does not matter).
// Writes <prefix>_report.txt, _samples.csv, _pareto.svg and _best.svg. Returns 0 on success.
static int screen(Params base, const ScreenCfg& c, std::vector<std::string>& err) {
  const auto t0 = std::chrono::steady_clock::now();
  const int nt = c.threads > 0 ? c.threads : std::max(1u, std::thread::hardware_concurrency());
  const size_t n = (size_t)c.samples, d = c.dims.size();
  // 1. LATIN HYPERCUBE SAMPLING (McKay et al., 1979): each dimension is cut into n equal slices and
  // every slice is used exactly once (random permutation + jitter), covering the space evenly.
  // Latin hypercube
  Rng rng((uint64_t)c.seed);
  std::vector<std::vector<double>> U(n, std::vector<double>(d));
  for (size_t j = 0; j < d; ++j) {
    std::vector<size_t> perm(n); std::iota(perm.begin(), perm.end(), 0);
    for (size_t i = n - 1; i > 0; --i) std::swap(perm[i], perm[rng.next() % (i + 1)]);
    for (size_t i = 0; i < n; ++i) U[i][j] = (perm[i] + rng.u01()) / n;
  }
  std::vector<Eval> E(n);
  parallel_for(n, nt, [&](size_t i) { E[i] = evaluate(base, c, U[i]); });

  // 2. Untreated reference run (tip_dose = 0) used for comparison in the report and figures.
  // untreated reference
  Params ub = base; ub.tip_dose = 0; std::vector<std::string> tmp;
  Traj trU; Result ru = run(ub, &trU, tmp);

  // 3. Ranking: lower class first (A feasible+extinct, B feasible, C unsafe), then lower F.
  // Ties break on sample index so the order is total and reproducible.
  // ranking (class, then scalar F)
  std::vector<size_t> ord; for (size_t i = 0; i < n; ++i) if (E[i].valid) ord.push_back(i);
  auto better = [&](size_t a, size_t b) { if (E[a].cls != E[b].cls) return E[a].cls < E[b].cls; return E[a].F < E[b].F; };
  std::sort(ord.begin(), ord.end(), [&](size_t a, size_t b) { return better(a, b) || (!better(b, a) && a < b); });
  size_t nValid = ord.size(), nFeas = 0, nExt = 0;
  for (size_t i : ord) { nFeas += E[i].feasible; nExt += E[i].cls == 0; }
  if (nValid == 0) { err.push_back("ERROR: no valid samples - check the search bounds"); return 1; }

  // 4. Pareto front over (peak V_W, peak C, TIP exposure U) among feasible samples: keep a profile
  // only if no kept profile is at least as good in all three and strictly better in one.
  // Pareto front over (peak V_W, peak C, exposure U) among feasible samples
  std::vector<size_t> feas; for (size_t i : ord) if (E[i].feasible) feas.push_back(i);
  std::sort(feas.begin(), feas.end(), [&](size_t a, size_t b) { return E[a].r.peakV < E[b].r.peakV || (E[a].r.peakV == E[b].r.peakV && a < b); });
  std::vector<size_t> front;
  for (size_t i : feas) {
    bool dom = false;
    for (size_t f : front) if (E[f].r.peakC <= E[i].r.peakC && E[f].r.I[3] <= E[i].r.I[3] &&
        (E[f].r.peakC < E[i].r.peakC || E[f].r.I[3] < E[i].r.I[3] || E[f].r.peakV < E[i].r.peakV)) { dom = true; break; }
    if (!dom) front.push_back(i);
  }
  std::vector<char> onFront(n, 0); for (size_t f : front) onFront[f] = 1;

  // 5. Polish the best refine_top sampled profiles with Nelder-Mead.
  // Nelder-Mead refinement of the best few
  struct Ref { size_t from; Eval e; int evals; };
  std::vector<Ref> refined;
  for (int k = 0; k < c.refine_top && k < (int)ord.size(); ++k) {
    int ev = 0; Eval best = nelder_mead(base, c, U[ord[k]], ev);
    refined.push_back({ord[k], best, ev});
  }
  std::sort(refined.begin(), refined.end(), [](const Ref& a, const Ref& b) { return a.e.F < b.e.F; });

  // 6. Weight sensitivity: scale each objective weight by x and 1/x (weight_factor) and check whether
  // the baseline winner stays first; reports how robust the ranking is to subjective weights.
  // weight sensitivity: re-rank under scaled objective weights
  const double w0[4] = {base.w1, base.w2, base.w3, base.w4};
  auto score = [&](size_t i, const double* w) { return w[0] * E[i].r.I[0] + w[1] * E[i].r.I[1] + w[2] * E[i].r.I[2] + w[3] * E[i].r.I[3] + E[i].pen; };
  const size_t win = ord[0];
  const char* wn[4] = {"w1 (virus)", "w2 (cytokine)", "w3 (pulse)", "w4 (exposure)"};
  std::ostringstream ws; int same = 0, tot = 0, worst = 1;
  for (int k = 0; k < 4; ++k) for (int s = 0; s < 2; ++s) {
    double w[4] = {w0[0], w0[1], w0[2], w0[3]}; w[k] *= (s ? 1.0 / c.wfactor : c.wfactor);
    size_t top = ord[0]; int rankWin = 1;
    for (size_t i : ord) { auto lt = [&](size_t a, size_t b) { return E[a].cls != E[b].cls ? E[a].cls < E[b].cls : score(a, w) < score(b, w); };
      if (lt(i, top)) top = i;
      if (i != win && lt(i, win)) ++rankWin; }
    ++tot; same += (top == win); worst = std::max(worst, rankWin);
    ws << "  " << wn[k] << (s ? " / " : " x ") << fv(c.wfactor) << " : top-1 " << (top == win ? "unchanged" : "changed (sample " + std::to_string(top) + ")")
       << ", baseline winner rank " << rankWin << "\n";
  }
```

```
  // 7. Global sensitivity: PRCC of every swept parameter against log10 peak V_W, peak C and TIP exposure.
  // PRCC
  std::vector<std::vector<double>> X(d); std::vector<double> yV, yC, yU;
  for (size_t i : ord) { for (size_t j = 0; j < d; ++j) X[j].push_back(E[i].x[j]);
    yV.push_back(std::log10(std::max(E[i].r.peakV, 1e-4))); yC.push_back(E[i].r.peakC); yU.push_back(E[i].r.I[3]); }
  auto pV = prcc(X, yV), pC = prcc(X, yC), pU = prcc(X, yU);

  // 8. Outputs: text report, CSV of every sample (for any later analysis), and two SVG figures.
  // ---- report
  const std::string rep = c.prefix + "_report.txt";
  std::ofstream o(rep);
  o << "# TIP-profile screening report\n";
  o << "samples = " << n << ", valid = " << nValid << ", feasible (C<=C_crit, H<=H_crit) = " << nFeas
    << ", feasible + WT extinction = " << nExt << ", Pareto front size = " << front.size() << "\n";
  o << "seed = " << (uint64_t)c.seed << ", t_end = " << base.t_end << " d, V_ext = " << base.v_ext << "\n";
  o << "R0_WT = " << fv(R0(base)) << "\n";
  o << "untreated reference: peak_V_W = " << fv(ru.peakV) << ", peak_C = " << fv(ru.peakC) << ", peak_H = " << fv(ru.peakH)
    << ", extinct = " << (ru.extinct ? "yes" : "no") << ", J = " << fv(Jw(base, ru)) << "\n\n";
  o << "class: A = feasible + extinction (high priority), B = feasible, C = violates safety bound\n";
  auto head = [&] { o << "rank class"; for (auto& dm : c.dims) o << " " << dm.name; o << " peakV peakC peakH t_ext U_AUC_VT F\n"; };
  auto rowf = [&](size_t id, const Eval& e) {
    o << "(" << id << ") " << "ABC"[e.cls];
    for (double v : e.x) o << " " << fv(v);
    o << " " << fv(e.r.peakV) << " " << fv(e.r.peakC) << " " << fv(e.r.peakH) << " " << (e.r.extinct ? fv(e.r.tExt) : "none")
      << " " << fv(e.r.I[3]) << " " << fv(e.F) << "\n";
  };
  o << "== Top " << c.top_k << " profiles (sampled) ==\n"; head();
  for (int k = 0; k < c.top_k && k < (int)ord.size(); ++k) { o << (k + 1) << " "; rowf(ord[k], E[ord[k]]); }
  o << "\n== Nelder-Mead refinement of the top " << refined.size() << " ==\n"; head();
  for (auto& r : refined) { o << "from " << r.from << " (" << r.evals << " evals) "; rowf(r.from, r.e); o << "  F before = " << fv(E[r.from].F) << "\n"; }
  o << "\n== Pareto front (peak V_W, peak C, exposure), sorted by peak V_W; first 25 of " << front.size() << " ==\n"; head();
  for (size_t k = 0; k < front.size() && k < 25; ++k) { o << (k + 1) << " "; rowf(front[k], E[front[k]]); }
  o << "\n== PRCC (|value| near 1 = strong influence on the output) ==\nparameter  log10_peakV  peakC  exposure_U\n";
  for (size_t j = 0; j < d; ++j) o << c.dims[j].name << "  " << fv(pV[j]) << "  " << fv(pC[j]) << "  " << fv(pU[j]) << "\n";
  o << "\n== Weight sensitivity (each weight scaled by x" << fv(c.wfactor) << " and 1/" << fv(c.wfactor) << ") ==\n" << ws.str();
  o << "  baseline winner stays top-1 in " << same << " of " << tot << " scenarios (worst rank " << worst << ")\n";
  o << "\nNote: ranking is conditional on the illustrative parameter set; all outputs are simulated, not predictions.\n";
  o.close();

  std::ofstream cs(c.prefix + "_samples.csv");
  cs << "id"; for (auto& dm : c.dims) cs << "," << dm.name; cs << ",peakV,peakC,peakH,extinct,t_ext,U,F,class,pareto\n";
  for (size_t i = 0; i < n; ++i) {
    cs << i; for (size_t j = 0; j < d; ++j) cs << "," << E[i].x[j];
    cs << "," << E[i].r.peakV << "," << E[i].r.peakC << "," << E[i].r.peakH << "," << E[i].r.extinct << "," << E[i].r.tExt
       << "," << E[i].r.I[3] << "," << E[i].F << "," << "ABC"[E[i].cls] << "," << (int)onFront[i] << "\n";
  }

  // Figure: peak WT load (log) vs peak cytokine for all samples; Pareto points dark blue; red x = untreated.
  // Pareto scatter
  { std::ostringstream s;
    const double W = 700, H = 460, L = 70, R = 20, T = 40, B = 48;
    double xl = 1e300, xh = -1e300, yh = base.C_crit * 1.05;
    auto lx = [&](double v) { return std::log10(std::max(v, 1e-4)); };
    for (size_t i : ord) { xl = std::min(xl, lx(E[i].r.peakV)); xh = std::max(xh, lx(E[i].r.peakV)); yh = std::max(yh, E[i].r.peakC * 1.02); }
    xl = std::min(xl, lx(ru.peakV)); xh = std::max(xh, lx(ru.peakV)); yh = std::max(yh, ru.peakC * 1.02);
    xl = std::floor(xl); xh = std::ceil(xh); if (xh <= xl) xh = xl + 1;
    yh = std::min(yh, base.C_crit * 2.0);
    auto px = [&](double v) { return L + (lx(v) - xl) / (xh - xl) * (W - L - R); };
    auto py = [&](double v) { return T + (1 - std::min(v, yh) / yh) * (H - T - B); };
    s << "<svg xmlns='http://www.w3.org/2000/svg' width='" << W << "' height='" << H + 30 << "' viewBox='0 0 " << W << " " << H + 30 << "' font-family='Arial,Helvetica,sans-serif'>\n<rect width='100%'
height='100%' fill='white'/>\n";
    s << "<text x='" << L << "' y='22' font-size='15' font-weight='bold'>TIP screening: peak WT viral load vs peak cytokine</text>\n";
    s << "<rect x='" << L << "' y='" << T << "' width='" << W - L - R << "' height='" << H - T - B << "' fill='none' stroke='#888'/>\n";
    for (int k = 0; k <= 4; ++k) { const double fx = xl + (xh - xl) * k / 4, fy = yh * k / 4;
      s << "<text x='" << L + (W - L - R) * k / 4 << "' y='" << H - B + 16 << "' font-size='11' text-anchor='middle'>1e" << fmt(fx) << "</text>"
        << "<text x='" << L - 7 << "' y='" << T + (1 - fy / yh) * (H - T - B) + 4 << "' font-size='11' text-anchor='end'>" << fmt(fy) << "</text>\n"; }
    s << "<text x='" << L + (W - L - R) / 2 << "' y='" << H - 12 << "' font-size='12' text-anchor='middle'>peak V_W (copies/mL, log scale)</text>\n";
    s << "<text x='16' y='" << T + (H - T - B) / 2 << "' font-size='12' text-anchor='middle' transform='rotate(-90 16 " << T + (H - T - B) / 2 << ")'>peak cytokine C (pg/mL)</text>\n";
    s << "<line x1='" << L << "' y1='" << py(base.C_crit) << "' x2='" << W - R << "' y2='" << py(base.C_crit) << "' stroke='#A32D2D' stroke-dasharray='2 3'/><text x='" << W - R - 3 << "' y='" << py(base.C_crit) - 4
<< "' font-size='11' text-anchor='end' fill='#A32D2D'>C_crit</text>\n";
    for (size_t i : ord) { if (E[i].cls == 2 || onFront[i]) continue;
      s << "<circle cx='" << px(E[i].r.peakV) << "' cy='" << py(E[i].r.peakC) << "' r='2.5' fill='" << (E[i].cls == 0 ? "#1D9E75" : "#85B7EB") << "' fill-opacity='0.6'/>\n"; }
    for (size_t i : ord) if (E[i].cls == 2) s << "<circle cx='" << px(E[i].r.peakV) << "' cy='" << py(E[i].r.peakC) << "' r='2' fill='#B4B2A9' fill-opacity='0.5'/>\n";
    for (size_t f : front) s << "<circle cx='" << px(E[f].r.peakV) << "' cy='" << py(E[f].r.peakC) << "' r='4' fill='#0C447C' stroke='white'/>\n";
    s << "<path d='M" << px(ru.peakV) - 6 << " " << py(ru.peakC) - 6 << "L" << px(ru.peakV) + 6 << " " << py(ru.peakC) + 6 << "M" << px(ru.peakV) - 6 << " " << py(ru.peakC) + 6 << "L" << px(ru.peakV) + 6 << "
" << py(ru.peakC) - 6 << "' stroke='#A32D2D' stroke-width='3'/>\n";
    const char* lg[5][2] = {{"#0C447C", "Pareto front"}, {"#1D9E75", "feasible + extinction"}, {"#85B7EB", "feasible"}, {"#B4B2A9", "violates a safety bound"}, {"#A32D2D", "untreated (x)"}};
    double lxp = L;
    for (auto& it : lg) { s << "<circle cx='" << lxp + 5 << "' cy='" << H + 14 << "' r='4' fill='" << it[0] << "'/><text x='" << lxp + 14 << "' y='" << H + 18 << "' font-size='11'>" << it[1] << "</text>\n"; lxp += 30 + 6.3 *
std::string(it[1]).size(); }
    s << "</svg>\n"; std::ofstream(c.prefix + "_pareto.svg") << s.str(); }

  // Figure: trajectory of the best profile (refined if it improved on the sampled winner) vs untreated.
  // best profile trajectory vs untreated
  { const Eval& bst = refined.empty() || refined[0].e.F >= E[win].F ? E[win] : refined[0].e;
    Params q = base;
    for (size_t j = 0; j < d; ++j) *(double*)((char*)&q + c.dims[j].off) = bst.x[j];
    validate(q, tmp);
    Traj tb; run(q, &tb, tmp);
    write_traj_svg(c.prefix + "_best.svg", "Best screened TIP profile vs untreated", q, tb, &trU); }

  const double secs = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();
  std::printf("screened %zu profiles on %d thread(s) in %.1f s: valid %zu, feasible %zu, feasible+extinct %zu, Pareto %zu\n",
              n, nt, secs, nValid, nFeas, nExt, front.size());
  std::printf("best sampled F = %.4g  (class %c)  | refined best F = %.4g\n", E[win].F, "ABC"[E[win].cls], refined.empty() ? E[win].F : refined[0].e.F);
  std::printf("wrote %s, %s_samples.csv, %s_pareto.svg, %s_best.svg\n", rep.c_str(), c.prefix.c_str(), c.prefix.c_str(), c.prefix.c_str());
  return 0;
}

// ---- Self-test of the integrator: run the same problem with step h, h/2, h/4; for a 4th-order
// method the error ratio e1/e2 is ~2^4, so order = log2(e1/e2) should lie in [3.8, 4.2].
// ------------------------------------------------------------------ self-test
// Integrate with a fixed number of equal steps (multiplied by "mult"), dosing at tip_time; no stability shrinking, so errors are pure truncation errors.
[[maybe_unused]] static State fixed_run(const Params& p, int mult) {
  State y{}; y[iT] = p.Ts; y[iVW] = p.VW0; y[iC] = p.Cs; y[iH] = p.Hs;
  const double seg1 = p.tip_time, seg2 = p.t_end - p.tip_time;
  if (seg1 > 0) { const int n = (int)std::ceil(seg1 / p.h) * mult; for (int i = 0; i < n; ++i) rk4(p, y, seg1 / n); }
  y[iVT] += p.tip_dose;
  if (seg2 > 0) { const int n = (int)std::ceil(seg2 / p.h) * mult; for (int i = 0; i < n; ++i) rk4(p, y, seg2 / n); }
```

```
  return y;
}
// Max relative difference between two states over the dynamic block.
[[maybe_unused]] static double dist(const State& a, const State& b) {
  double m = 0;
  for (int i = 0; i < kDyn; ++i) m = std::max(m, std::fabs(a[i] - b[i]) / std::max(1.0, std::fabs(b[i])));
  return m;
}


// NOTE FOR FUTURE PORTERS: the C API below is the stable boundary. A Kotlin/JNI (or Python ctypes,
// WebAssembly, ...) wrapper needs only these four functions; the model code above never changes.
// ================================================================== C API (for Kotlin / JNI)
// All text in, text out: no files needed for single runs, so it works inside an Android sandbox.
// Params text = the same "name = value" lines as input.txt (any subset; missing names use defaults).
// Every returned string must be released with ebov_free().
// Copy a std::string to malloc memory the caller releases with ebov_free() (C-compatible ownership).
static char* dup_str(const std::string& s) {
  char* p = (char*)std::malloc(s.size() + 1);
  if (p) std::memcpy(p, s.c_str(), s.size() + 1);
  return p;
}
// Helpers: does the message list contain a hard ERROR (warnings do not count)? / join messages with newlines.
static bool has_err(const std::vector<std::string>& e) { for (auto& s : e) if (s.rfind("ERROR", 0) == 0) return true; return false; }
static std::string join(const std::vector<std::string>& e) { std::string o; for (auto& s : e) o += s + "\n"; return o; }

// Unmangled names so any language can bind to them.
extern "C" {
const char* ebov_version() { return "ebov-tip-engine 1.0 (C++20)"; }
void ebov_free(char* p) { std::free(p); }

// format: 0 = summary (key=value lines + ERROR/WARNING lines), 1 = trajectory table, 2 = SVG plot
// Single run from text parameters. format 0 = key=value summary, 1 = trajectory table (space
// separated, header line first), 2 = SVG dashboard. Returns ERROR lines instead if input is invalid.
char* ebov_run(const char* params_text, int format) {
  Params p; std::vector<std::string> err;
  std::istringstream in(params_text ? params_text : "");
  load_stream(in, p, err);
  if (!has_err(err)) validate(p, err);
  if (has_err(err)) return dup_str(join(err));
  Traj tr; Result r = run(p, &tr, err);
  const bool haveU = p.tip_dose > 0;
  Traj trU; Result ru;
  if (haveU) { Params ub = p; ub.tip_dose = 0; std::vector<std::string> tmp; ru = run(ub, &trU, tmp); }
  if (format == 2) return dup_str(traj_svg_str(haveU ? "EBOV-TIP trajectory (with TIP vs untreated)" : "EBOV within-host trajectory (untreated)", p, tr, haveU ? &trU : nullptr));
  if (format == 1) {
    std::ostringstream o; o << "time T E_W I_W V_W E_Co I_Co V_T C H J P\n";
    for (size_t k = 0; k < tr.t.size(); ++k) { const State& y = tr.y[k];
      o << tr.t[k]; for (int i = 0; i < 9; ++i) o << ' ' << y[i];
      o << ' ' << p.w1 * y[iJV] + p.w2 * y[iJC] + p.w3 * y[iJH] + p.w4 * y[iU] << ' ' << y[iP] << "\n"; }
    return dup_str(o.str());
  }
  std::ostringstream o;
  o << "R0_WT=" << R0(p) << "\npeak_V_W=" << r.peakV << "\nt_peak_V_W=" << r.tPeakV << "\npeak_C=" << r.peakC
    << "\npeak_H=" << r.peakH << "\nC_within_bound=" << (r.peakC <= p.C_crit) << "\nH_within_bound=" << (r.peakH <= p.H_crit)
    << "\nextinct=" << r.extinct << "\nt_extinct=" << (r.extinct ? r.tExt : -1.0) << "\nexposure_AUC_V_T=" << r.I[3]
    << "\nobjective_J=" << Jw(p, r) << "\n";
  if (haveU) o << "untreated_peak_V_W=" << ru.peakV << "\nuntreated_peak_C=" << ru.peakC << "\nuntreated_peak_H=" << ru.peakH << "\n";
  if (R0(p) <= 1) err.push_back("WARNING: R_0 <= 1 for WT alone; not an acute-infection regime");
  o << join(err);
  return dup_str(o.str());
}

// (Output files need a writable prefix path; the report text is also returned so the UI can show it at once.)
// Screening: writes <out_prefix>_report.txt, _samples.csv, _pareto.svg, _best.svg and returns the report text
// (or ERROR lines). out_prefix example on Android: "/data/data/<pkg>/files/screen".
char* ebov_screen(const char* params_text, const char* screen_text, const char* out_prefix) {
  Params p; std::vector<std::string> err;
  std::istringstream in(params_text ? params_text : "");
  load_stream(in, p, err);
  if (!has_err(err)) validate(p, err);
  ScreenCfg c;
  std::istringstream sin(screen_text ? screen_text : "");
  load_screen_stream(sin, c, p, err);
  if (has_err(err)) return dup_str(join(err));
  if (out_prefix && *out_prefix) c.prefix = out_prefix;
  if (screen(p, c, err) != 0) return dup_str(join(err));
  std::ifstream rep(c.prefix + "_report.txt");
  std::ostringstream o; o << rep.rdbuf();
  return dup_str(join(err) + o.str());
}
}  // extern "C"

// main() is compiled out when the file is built as a library for the API test or JNI (-DEBOV_NO_MAIN).
#ifndef EBOV_NO_MAIN
// Command-line driver. Modes: default (single run), --screen, --selftest. Argument handling is
// positional with defaults; all input problems are written to errors.txt and nothing is run on error.
int main(int argc, char** argv) {
  const std::string mode = argc > 1 ? argv[1] : "";
  const bool self = mode == "--selftest", scr = mode == "--screen";
  const int a0 = (self || scr) ? 2 : 1;
  auto arg = [&](int k, const char* def) { return argc > a0 + k ? std::string(argv[a0 + k]) : std::string(def); };
  const std::string scfg = scr ? arg(0, "screen.txt") : "";
  const std::string fin = scr ? arg(1, "input.txt") : arg(0, "input.txt");
  const std::string fout = (self || scr) ? "" : arg(1, "output.txt");
  const std::string ferr = scr ? arg(2, "errors.txt") : (self ? "errors.txt" : arg(2, "errors.txt"));

  Params p;
  std::vector<std::string> err;
  load(fin, p, err);
  if (err.empty()) validate(p, err);
  auto has_error = [&] { for (auto& s : err) if (s.rfind("ERROR", 0) == 0) return true; return false; };
  auto flush_err = [&] { std::ofstream e(ferr); for (auto& s : err) e << s << "\n"; };
  if (has_error()) { flush_err(); std::fprintf(stderr, "Input rejected, see %s\n", ferr.c_str()); return 1; }

  if (self) {
    State a = fixed_run(p, 1), b = fixed_run(p, 2), c = fixed_run(p, 4);
    const double e1 = dist(a, b), e2 = dist(b, c), order = std::log2(e1 / e2);
    std::printf("RK4 order estimate: %.3f  (e1=%.3e, e2=%.3e)  %s\n", order, e1, e2, (order >= 3.8 && order <= 4.2) ? "PASS" : "CHECK");
    flush_err(); return 0;
```

```
  }
  if (scr) {
    ScreenCfg c;
    load_screen(scfg, c, p, err);
    if (has_error()) { flush_err(); std::fprintf(stderr, "Screen file rejected, see %s\n", ferr.c_str()); return 1; }
    const int rc = screen(p, c, err);
    flush_err(); return rc;
  }

  Traj tr; Result r = run(p, &tr, err);
  std::ofstream out(fout);
  out << "# time T E_W I_W V_W E_Co I_Co V_T C H J\n";
  for (size_t k = 0; k < tr.t.size(); ++k) {
    const State& y = tr.y[k];
    char buf[400];
    std::snprintf(buf, sizeof buf, "%.6g %.9g %.9g %.9g %.9g %.9g %.9g %.9g %.9g %.9g %.9g\n", tr.t[k], y[0], y[1], y[2], y[3], y[4], y[5], y[6], y[7], y[8],
                  p.w1 * y[iJV] + p.w2 * y[iJC] + p.w3 * y[iJH] + p.w4 * y[iU]);
    out << buf;
  }
  const double r0 = R0(p);
  if (r0 <= 1) err.push_back("WARNING: R_0 <= 1 for WT alone; not an acute-infection regime");
  if (r.reduced > 0) err.push_back("WARNING: step reduced for stability in " + std::to_string(r.reduced) + " output intervals");
  Traj trU; Result ru; const bool haveU = p.tip_dose > 0;
  if (haveU) { Params ub = p; ub.tip_dose = 0; std::vector<std::string> tmp; ru = run(ub, &trU, tmp); }
  out << "# ---- summary ----\n# R0_WT = " << r0 << "\n# R_eff_(entry+clearance only) = " << (1 - p.eps_ab) * r0 / (1 + p.kappa * p.eps_clr) << "\n";
  out << "# peak_V_W = " << r.peakV << " at t = " << r.tPeakV << " d" << (haveU ? "   (untreated: " + fv(ru.peakV) + ")" : "") << "\n";
  out << "# peak_C = " << r.peakC << (r.peakC > p.C_crit ? "  EXCEEDS C_crit" : "  within C_crit") << (haveU ? "   (untreated: " + fv(ru.peakC) + ")" : "") << "\n";
  out << "# peak_H = " << r.peakH << (r.peakH > p.H_crit ? "  EXCEEDS H_crit" : "  within H_crit") << (haveU ? "   (untreated: " + fv(ru.peakH) + ")" : "") << "\n";
  out << "# extinction (V_W < V_ext) = " << (r.extinct ? "YES" : "NO"); if (r.extinct) out << " , time = " << r.tExt << " d";
  out << "\n# objective_J = " << Jw(p, r) << "\n# exposure_AUC_V_T = " << r.I[3] << "\n";
  out.close();
  std::string svg = fout; if (auto dot = svg.rfind('.'); dot != std::string::npos) svg.erase(dot); svg += ".svg";
  write_traj_svg(svg, haveU ? "EBOV-TIP trajectory (with TIP vs untreated)" : "EBOV within-host trajectory (untreated)", p, tr, haveU ? &trU : nullptr);
  std::printf("R0=%.3g  peakV=%.3g (t=%.2f)  peakC=%.3g  peakH=%.3g  extinct=%s  J=%.4g  -> %s, %s\n",
              r0, r.peakV, r.tPeakV, r.peakC, r.peakH, r.extinct ? "yes" : "no", Jw(p, r), fout.c_str(), svg.c_str());
  flush_err();
  return r.ok ? 0 : 2;
}
#endif  // EBOV_NO_MAIN
```

Appendix D Adaptive-Immunity Extension: rpt_adapt.cpp

The extension adds the B, A and Z states of Section 11 to the core model, with the same fixed-step RK4 integrator and a stability guard that uses a scaled Jacobian bound. With adaptive = 0 it reproduces the core model; the figures of Sections 7.5 and 10–11 were produced with it. Build with g++ -std=c++20 -O2.

```
// ============================================================================
// rpt_adapt.cpp - Adaptive-immunity and vaccination extension of the RPT-EBOV engine
// (C++20, standard library only).                    Author: Reinford Nyachio
//
// PURPOSE
//   Adds three adaptive-immune states to the ten-state core model of code.cpp: the B-cell /
//   antibody-secreting pool B, the neutralising antibody concentration A and the cytotoxic
//   T-lymphocyte pool Z. Vaccination is represented ONLY through the initial conditions
//   (A(0), B(0), Z(0)); the equations are identical for naive and vaccinated hosts.
//   With adaptive = 0 the three new states are frozen and the programme reproduces the core
//   model exactly (this is how the extension is cross-validated against code.cpp).
//
// BUILD / RUN
//   g++ -std=c++20 -O2 rpt_adapt.cpp -o rpt_adapt
//   ./rpt_adapt name=value ... [traj=out.csv]    one run; prints a one-line summary
//   Any parameter below can be overridden as name=value on the command line.
//
// STATE (13 dynamic states):  T EW IW EC IC P VW VT C H | B A Z
// UNITS: cells/mL, copies/mL, days; antibody in ug/mL; cytokine in pg/mL; pulse in beats/min.
// ============================================================================
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <fstream>
#include <map>
#include <string>
#include <vector>

// Index names for the state vector (core states first, then the adaptive block).
enum Idx { iT, iEW, iIW, iEC, iIC, iP, iVW, iVT, iC, iH, iB, iA, iZ, N };
using State = std::array<double, N>;

// All parameters. Core values are those of the reference parameter set of the main text;
// the adaptive block holds illustrative values stated in Table 15 of the document.
struct Params {
  // ---- core (see Section 5) ----
  double lambda = 5e4, dT = 0.07, bW = 5e-7, bT = 5e-7, kW = 6, kCo = 6;
  double dW = 2, dCo = 2, pW = 500, pT = 1500, alpha = 0.2, cW = 10, cT = 10;
  double dN = 0.5, KF = 50, kappa = 1;
  double c_rest = 5, dC = 5, gamma = 3, KC = 50, rhoI = 0.005, rhoV = 1e-6, rhoT = 0;
  double sigma_v = 0.5, H0 = 70, Hmax = 130, dH = 50, KH = 100, n_hill = 2, tau_H = 0.02;
  double eps_ab = 0, eps_clr = 0;      // constant (non-immune) blocking / clearance effects
  double primed = 1, v_ext = 2e-4, ext_floor = 1;
  double guard = 1;                    // 1 = stability pre-flight shrinks the step; 0 = fixed step (convergence studies)
  double VW0 = 100, tip_dose = 0, tip_time = 0, t_end = 30, h = 0.005, dt_out = 0.25;
  // ---- adaptive block ----
  double adaptive = 1;                 // 1 = adaptive states evolve; 0 = frozen (core model)
  double KA = 10;                      // antibody half-effect concentration (ug/mL)
  double eps_max = 0.9;                // maximal antibody entry-blocking fraction
  double xiT = 1;                      // antibody cross-recognition of the TIP (0 = TIP invisible)
  double add_tip_clr = 0;              // 1 = antibody clearance of TIP is ADDITIVE (worst case)
  double piB = 0.5, KB = 1e3, Bmax = 200, mu_decay = 0.005, B_naive = 1;
  double pA = 0.0165, dA = 0.033;      // pA is calibrated so that B = 100 sustains A = 50 ug/mL
  double piZ = 0.4, KZ = 1e3, Zmax = 1e4, muZd = 0.005, Z_naive = 1, muZ = 1e-5;
  double A0 = 0, B0 = -1, Z0 = -1;     // initial adaptive state (B0, Z0 < 0: derived from A0)
  // ---- optional initial perturbations (equilibrium test) ----
  double T0f = 1, C0 = -1, H0i = -1;
  // ---- derived ----
  double Ts = 0, Cs = 0, Hs = 0, s0 = 0;
};

static double hill(double x, double K, int n) {
  const double xn = std::pow(std::max(x, 0.0), n), Kn = std::pow(K, n);
  return xn / (Kn + xn);
}
// Vagal gain m(H), Eq 9.
static double vagal(const Params& p, double H) {
  return 1.0 - p.sigma_v * std::clamp((p.Hmax - H) / (p.Hmax - p.H0), 0.0, 1.0);
}
// Saturating antibody effect a(A) = A/(K_A + A) in [0,1): drives eps_Ab(A) and eps_Clr(A) (Eqs 17-18).
static double aeff(const Params& p, double A) { return std::max(A, 0.0) / (p.KA + std::max(A, 0.0)); }

// Right-hand side: Eqs 1-10 of the core model with antibody-dependent entry blocking and clearance,
// CTL killing, and the adaptive equations 19-21 (see the document, Section 11).
static void rhs(const Params& p, const State& y, State& d) {
  const double a = aeff(p, y[iA]) * p.adaptive;
  const double epsAb = 1.0 - (1.0 - p.eps_ab) * (1.0 - p.eps_max * a);   // combined entry blocking (Eq 18)
  const double epsClr = p.eps_clr + a * (1.0 - p.eps_clr);               // combined clearance effect (Eq 18)
  const double ent = 1.0 - epsAb;
  const double cyt = p.dN * y[iC] / (p.KF + y[iC]);
  const double ctl = p.adaptive * p.muZ * y[iZ];                         // CTL killing rate of infected cells
  const double inf = ent * p.bW * y[iT] * y[iVW];
  const double tip = ent * p.bT * y[iVT];
  const double supW = ent * p.bW * y[iVW];
  d[iT]  = p.lambda - p.dT * y[iT] - inf - tip * y[iT];
  d[iEW] = inf - p.kW * y[iEW] - tip * y[iEW];
  d[iIW] = p.kW * y[iEW] - (p.dW + cyt + ctl) * y[iIW] - tip * y[iIW];
  d[iEC] = tip * y[iEW] + p.primed * supW * y[iP] - p.kCo * y[iEC];
  d[iIC] = p.kCo * y[iEC] + tip * y[iIW] - (p.dCo + cyt + ctl) * y[iIC];
  d[iP]  = p.primed * (tip * y[iT] - supW * y[iP]) - p.dT * y[iP];
  d[iVW] = p.pW * y[iIW] + p.alpha * p.pW * y[iIC] - p.cW * (1.0 + p.kappa * epsClr) * y[iVW];
  // TIP clearance: multiplicative boost (default, same structure as WT) or additive worst case.
  const double cTeff = p.add_tip_clr > 0.5 ? p.cT + p.xiT * p.kappa * epsClr * p.cW
                                           : p.cT * (1.0 + p.xiT * p.kappa * epsClr);
  d[iVT] = p.pT * y[iIC] - cTeff * y[iVT];
  d[iC]  = vagal(p, y[iH]) * (p.s0 + p.rhoI * (y[iIW] + y[iIC]) + p.rhoV * y[iVW] + p.rhoT * y[iVT]
         + p.gamma * hill(y[iC], p.KC, 2)) - p.dC * y[iC];
```

```
  d[iH]  = (p.H0 + p.dH * hill(y[iC], p.KH, (int)p.n_hill) - y[iH]) / p.tau_H;
  // ---- adaptive block (Eqs 19-21) ----
  const double Itot = y[iIW] + y[iIC];
  d[iB] = p.adaptive * (p.piB * y[iB] * y[iVW] / (p.KB + y[iVW]) * (1.0 - y[iB] / p.Bmax)
                        - p.mu_decay * (y[iB] - p.B_naive));
  d[iA] = p.adaptive * (p.pA * y[iB] - p.dA * y[iA]);
  d[iZ] = p.adaptive * (p.piZ * y[iZ] * Itot / (p.KZ + Itot) * (1.0 - y[iZ] / p.Zmax)
                        - p.muZd * (y[iZ] - p.Z_naive));
}

// Classical fixed-step RK4.
static void rk4(const Params& p, State& y, double h) {
  State k1, k2, k3, k4, s;
  rhs(p, y, k1);
  for (int i = 0; i < N; ++i) s[i] = y[i] + 0.5 * h * k1[i];
  rhs(p, s, k2);
  for (int i = 0; i < N; ++i) s[i] = y[i] + 0.5 * h * k2[i];
  rhs(p, s, k3);
  for (int i = 0; i < N; ++i) s[i] = y[i] + h * k3[i];
  rhs(p, s, k4);
  for (int i = 0; i < N; ++i) y[i] += (h / 6.0) * (k1[i] + 2 * k2[i] + 2 * k3[i] + k4[i]);
}

// Infection-free equilibrium and the basal cytokine synthesis that makes C* exact (as in the core).
static void derive(Params& p) {
  p.Ts = p.lambda / p.dT;
  p.Cs = p.c_rest;
  p.Hs = p.H0 + p.dH * hill(p.Cs, p.KH, (int)p.n_hill);
  p.s0 = p.dC * p.Cs / vagal(p, p.Hs) - p.gamma * hill(p.Cs, p.KC, 2);
  if (p.B0 < 0) p.B0 = std::max(p.B_naive, p.A0 * p.dA / p.pA);   // memory pool consistent with A0
  if (p.Z0 < 0) p.Z0 = std::max(p.Z_naive, p.B0);                 // CTL memory scaled alike
}
// Basic reproduction number (Eq 12) and its antibody-modified form (Eq 13) at antibody level A.
static double R0(const Params& p) { return p.bW * p.pW * p.Ts / (p.cW * (p.dW + p.dN * p.Cs / (p.KF + p.Cs))); }
static double Reff(const Params& p, double A) {
  const double a = aeff(p, A);
  const double eAb = 1.0 - (1.0 - p.eps_ab) * (1.0 - p.eps_max * a), eCl = p.eps_clr + a * (1.0 - p.eps_clr);
  return (1.0 - eAb) * R0(p) / (1.0 + p.kappa * eCl);
}

// STABILITY GUARD. RK4 needs h*rho < ~2.78 (real axis), where rho is the spectral radius of the Jacobian J.
// J is formed by forward differences and rho is estimated by power iteration (geometric mean of the growth
// ratio over the last iterations, which also copes with complex eigenvalue pairs). A plain matrix-norm bound
// is far too pessimistic here (units differ by 10 orders of magnitude, e.g. production p_T = 2877 against a
// state of order 1), so the true spectral radius is estimated instead and a 25% safety factor is applied.
// advance() shrinks the step when h*rho would exceed 2.5; this matters for very large TIP doses, where
// beta_T*V_T reaches hundreds per day.
static double jac_bound(const Params& p, const State& y) {
  State f0, f1, z = y;
  rhs(p, y, f0);
  double J[N][N];
  for (int j = 0; j < N; ++j) {
    const double e = 1e-6 * std::max(1.0, std::fabs(y[j]));
    z[j] = y[j] + e;
    rhs(p, z, f1);
    z[j] = y[j];
    for (int i = 0; i < N; ++i) J[i][j] = (f1[i] - f0[i]) / e;
  }
  std::array<double, N> v;
  v.fill(1.0 / std::sqrt((double)N));
  double logsum = 0;
  int cnt = 0;
  for (int it = 0; it < 300; ++it) {
    std::array<double, N> w{};
    for (int i = 0; i < N; ++i) for (int j = 0; j < N; ++j) w[i] += J[i][j] * v[j];
    double nrm = 0;
    for (int i = 0; i < N; ++i) nrm += w[i] * w[i];
    nrm = std::sqrt(nrm);
    if (nrm < 1e-300) return 0.0;
    if (it >= 250) { logsum += std::log(nrm); ++cnt; }
    for (int i = 0; i < N; ++i) v[i] = w[i] / nrm;
  }
  return 1.25 * std::exp(logsum / cnt);
}

struct Result { double peakV = 0, tPeak = 0, peakC = 0, peakH = 0, Vfinal = 0, tExt = -1; bool extinct = false, ok = true; };

// One run. Writes the trajectory (one row per output time) to 'out' when it is non-null.
static Result run(const Params& p, std::FILE* out) {
  Result r;
  State y{};
  y[iT] = p.Ts * p.T0f; y[iVW] = p.VW0;
  y[iC] = p.C0 >= 0 ? p.C0 : p.Cs;
  y[iH] = p.H0i >= 0 ? p.H0i : p.Hs;
  y[iB] = p.adaptive > 0.5 ? p.B0 : p.B_naive; y[iA] = p.adaptive > 0.5 ? p.A0 : 0; y[iZ] = p.adaptive > 0.5 ? p.Z0 : p.Z_naive;
  double t = 0, lastAbove = -1;
  bool dosed = false;
  auto track = [&] {
    if (y[iVW] > r.peakV) { r.peakV = y[iVW]; r.tPeak = t; }
    r.peakC = std::max(r.peakC, y[iC]); r.peakH = std::max(r.peakH, y[iH]);
    if (y[iVW] >= p.v_ext) lastAbove = t;
  };
  auto dose = [&] { if (!dosed && t >= p.tip_time - 1e-12) { y[iVT] += p.tip_dose; dosed = true; } };
  auto emit = [&] {
    if (!out) return;
    std::fprintf(out, "%.6g", t);
    for (int i = 0; i < N; ++i) std::fprintf(out, ",%.15g", y[i]);
    std::fprintf(out, "\n");
  };
  auto advance = [&](double to) {
    const double rho = jac_bound(p, y);
    const double hmax = p.guard > 0.5 ? std::min(p.h, 2.5 / std::max(rho, 1e-12)) : p.h;   // pre-flight stability bound
    const int n = std::max(1, (int)std::ceil((to - t) / hmax - 1e-9));
    const double hs = (to - t) / n;
    for (int s = 0; s < n; ++s) {
      rk4(p, y, hs); t += hs;
      for (int i = 0; i < N; ++i)
        if (!std::isfinite(y[i]) || y[i] < -1e-6 * p.Ts) { r.ok = false; return; }
      if (p.ext_floor > 0.5 && y[iVW] < p.v_ext && y[iEW] + y[iIW] + y[iEC] + y[iIC] < p.v_ext)
        y[iVW] = y[iEW] = y[iIW] = y[iEC] = y[iIC] = 0;
```

```
    track();
   }
   t = to;
  };
  if (out) std::fprintf(out, "t,T,EW,IW,EC,IC,P,VW,VT,C,H,B,A,Z\n");
  dose(); track(); emit();
  const int nOut = (int)std::ceil(p.t_end / p.dt_out - 1e-9);
  for (int k = 1; k <= nOut && r.ok; ++k) {
   const double target = std::min(k * p.dt_out, p.t_end);
   if (!dosed && p.tip_time > t && p.tip_time < target) { advance(p.tip_time); dose(); }
   if (!r.ok) break;
   advance(target); dose(); emit();
  }
  r.Vfinal = y[iVW];
  r.extinct = r.ok && y[iVW] < p.v_ext;
  if (r.extinct) r.tExt = std::max(lastAbove, 0.0);
  return r;
}

// Command-line parameter table: name -> member.
static std::map<std::string, double*> table(Params& p) {
  return {{"lambda", &p.lambda}, {"d_T", &p.dT}, {"beta_W", &p.bW}, {"beta_T", &p.bT}, {"k_W", &p.kW}, {"k_Co", &p.kCo},
          {"delta_W", &p.dW}, {"delta_Co", &p.dCo}, {"p_W", &p.pW}, {"p_T", &p.pT}, {"alpha", &p.alpha}, {"c_W", &p.cW},
          {"c_T", &p.cT}, {"delta_N", &p.dN}, {"K_F", &p.KF}, {"kappa", &p.kappa}, {"C_rest", &p.c_rest}, {"d_C", &p.dC},
          {"gamma", &p.gamma}, {"K_C", &p.KC}, {"rho_I", &p.rhoI}, {"rho_V", &p.rhoV}, {"rho_T", &p.rhoT},
          {"sigma_v", &p.sigma_v}, {"H_0", &p.H0}, {"H_max", &p.Hmax}, {"Delta_H", &p.dH}, {"K_H", &p.KH}, {"n_hill", &p.n_hill},
          {"tau_H", &p.tau_H}, {"eps_Ab", &p.eps_ab}, {"eps_Clr", &p.eps_clr}, {"primed", &p.primed}, {"V_ext", &p.v_ext},
          {"ext_floor", &p.ext_floor}, {"guard", &p.guard}, {"V_W0", &p.VW0}, {"tip_dose", &p.tip_dose}, {"tip_time", &p.tip_time},
          {"t_end", &p.t_end}, {"h", &p.h}, {"dt_out", &p.dt_out}, {"adaptive", &p.adaptive}, {"K_A", &p.KA},
          {"eps_max", &p.eps_max}, {"xi_T", &p.xiT}, {"add_tip_clr", &p.add_tip_clr}, {"pi_B", &p.piB}, {"K_B", &p.KB},
          {"B_max", &p.Bmax}, {"mu_decay", &p.mu_decay}, {"B_naive", &p.B_naive}, {"p_A", &p.pA}, {"d_A", &p.dA},
          {"pi_Z", &p.piZ}, {"K_Z", &p.KZ}, {"Z_max", &p.Zmax}, {"mu_Zdecay", &p.muZd}, {"Z_naive", &p.Z_naive},
          {"mu_Z", &p.muZ}, {"A0", &p.A0}, {"B0", &p.B0}, {"Z0", &p.Z0}, {"T0_factor", &p.T0f}, {"C0", &p.C0}, {"H0_init", &p.H0i}};
}

int main(int argc, char** argv) {
  Params p;
  auto tab = table(p);
  std::string trajFile;
  for (int i = 1; i < argc; ++i) {
   const char* eq = std::strchr(argv[i], '=');
   if (!eq) { std::fprintf(stderr, "bad argument: %s\n", argv[i]); return 2; }
   const std::string k(argv[i], eq - argv[i]), v(eq + 1);
   if (k == "traj") { trajFile = v; continue; }
   auto it = tab.find(k);
   if (it == tab.end()) { std::fprintf(stderr, "unknown parameter: %s\n", k.c_str()); return 2; }
   *it->second = std::atof(v.c_str());
  }
  derive(p);
  std::FILE* out = trajFile.empty() ? nullptr : std::fopen(trajFile.c_str(), "w");
  const Result r = run(p, out);
  if (out) std::fclose(out);
  std::printf("ok=%d peakV=%.9g tPeak=%.5g peakC=%.9g peakH=%.9g Vfinal=%.9g extinct=%d tExt=%.5g R0=%.6g Reff0=%.6g\n",
              r.ok, r.peakV, r.tPeak, r.peakC, r.peakH, r.Vfinal, r.extinct, r.tExt, R0(p), Reff(p, p.A0));
  return r.ok ? 0 : 1;
}
```

Appendix E Analysis Scripts (Python)

The Python drivers below call the compiled engine for the threshold searches and implement the tail-stochastic simulation and the branching-process validation of Section 12. They use only numpy and the standard library. The figures are drawn by make_figures.py and make_figures_analysis.py from the engine output stored in figure_data/.

```
# analysis_scripts.py - drivers used for Sections 10-12 (Python 3 + numpy; the engine itself stays dependency-free)
# ---- exp.py: thin wrapper around the compiled engine (rpt_adapt) and a bisection for the persistence threshold ----
import subprocess, re, json, numpy as np, os
EXE='/home/claude/build/rpt_adapt'
BEST=dict(beta_T=8.129e-7,alpha=0.05018,p_T=2877,tip_dose=3.794e7,tip_time=0.2602)   # screened-best sampled profile
REFINED=dict(beta_T=2.0e-6,alpha=0.05,p_T=1779,tip_dose=2.24e6,tip_time=0.02865)    # Nelder-Mead refined profile
def run(over, traj=None, adaptive=0):
    args=[f"{k}={v}" for k,v in over.items()]+[f"adaptive={adaptive}"] if 'adaptive' not in over else [f"{k}={v}" for k,v in over.items()]
    if traj: args.append(f"traj={traj}")
    o=subprocess.run([EXE]+args,capture_output=True,text=True).stdout
    d={k:float(v) for k,v in re.findall(r'(\w+)=([\d.eE+-]+|nan|inf)',o)}
    return d
def ext(over,**kw): return run(over,**kw)['extinct']==1
def cT_star(prof, extra=None, lo=0.01, hi=3.0, iters=22, adaptive=0):
    extra=extra or {}
    f=lambda c: ext({**prof,**extra,'c_T':c},adaptive=adaptive)
    if not f(lo): return np.nan
    if f(hi): return hi
    for _ in range(iters):
        mid=(lo*hi)**0.5
        if f(mid): lo=mid
        else: hi=mid
    return lo


# ---- tail-stochastic extinction check (Section 12.2): Gillespie simulation of the low-count WT subsystem in an ODE environment ----
import numpy as np
rng=np.random.default_rng(20261109)
VOL=5000.0   # host blood volume, mL
def tail_sim(env, pars, N_start, reps, t_end=30.0, esc=1500):
    t=env[:,0]; T=env[:,1]; EW=env[:,2]; IW=env[:,3]; EC=env[:,4]; IC=env[:,5]; P=env[:,6]; VW=env[:,7]; VT=env[:,8]; C=env[:,9]
    tot=VOL*(VW+EW+IW+EC+IC); ipk=int(np.argmax(t>=0.3)); tmin=float(tot[ipk:].min())
    N_start=max(N_start,1.5*tmin)
    k0=next(k for k in range(ipk,len(t)) if tot[k]<N_start)
    bW,bT,kW,kCo,dW,dCo,pW,al,cW,dN,KF=[pars[x] for x in ('bW','bT','kW','kCo','dW','dCo','pW','alpha','cW','dN','KF')]
    ext=0; esc_n=0
    for _ in range(reps):
        X=[rng.poisson(VOL*EW[k0]),rng.poisson(VOL*IW[k0]),rng.poisson(VOL*EC[k0]),rng.poisson(VOL*IC[k0]),rng.poisson(VOL*VW[k0])]  # E,I,Ec,Ic,V
        k=k0; tc=t[k0]; done=False
        while k<len(t)-1 and not done:
            dt=t[k+1]-t[k]; cyt=dN*C[k]/(KF+C[k]); tt=0.0
            while True:
                E,I,Ec,Ic,V=X
                r=[bW*T[k]*V, kW*E, bT*VT[k]*E, (dW+cyt)*I, bT*VT[k]*I, bW*P[k]*V, kCo*Ec, (dCo+cyt)*Ic, pW*I+al*pW*Ic, cW*V]
                R=sum(r)
                if R<=0: tt=dt; break
                tau=rng.exponential(1.0/R)
                if tt+tau>dt: break
                tt+=tau
                u=rng.random()*R; acc=0
                for j,rj in enumerate(r):
                    acc+=rj
                    if u<acc: break
                if j==0: X[0]+=1
                elif j==1: X[0]-=1; X[1]+=1
                elif j==2: X[0]-=1; X[2]+=1
                elif j==3: X[1]-=1
                elif j==4: X[1]-=1; X[3]+=1
                elif j==5: X[2]+=1
                elif j==6: X[2]-=1; X[3]+=1
                elif j==7: X[3]-=1
                elif j==8: X[4]+=1
                else: X[4]-=1
                if X[4]+X[0]+X[1]+X[2]+X[3] > esc: done=True; break
            if sum(X)==0: done=True
            k+=1
        if sum(X)==0: ext+=1
        elif sum(X)>esc: esc_n+=1
    return ext, esc_n, float(t[k0]), int(tot[k0])

# ---- branching-process establishment probability and its Monte Carlo validation (Section 12.1) ----
def q_theory(p,dl,R): return min(1.0,(p+dl)/(p+R*dl))
def mc(p,dl,c,R,n,thr=400):
    bTs=R*c*dl/p; est=0
    for _ in range(n):
        V,I=1,0
        while True:
            r1=c*V; r2=bTs*V; r3=dl*I; r4=p*I; R_=r1+r2+r3+r4
            if R_==0: break
            u=rng.random()*R_
            if u<r1: V-=1
            elif u<r1+r2: I+=1
            elif u<r1+r2+r3: I-=1
            else: V+=1
            if V>=thr: est+=1; break
    return est/n
```